Topological Data Analysis를 활용한 음악 구조 분석 및 위상 구조 보존 기반 AI 작곡 파이프라인

저자: 김민주
지도: 정재훈 (KIAS 초학제 독립연구단)
작성일: 2026
키워드: Topological Data Analysis, Persistent Homology, Tonnetz, Music Generation, Vietoris-Rips Complex, Jensen-Shannon Divergence

---

초록 (Abstract)

본 연구는 사카모토 류이치의 2009년 앨범 out of noise 수록곡 "hibari"를 대상으로, 음악의 구조를 **위상수학적으로 분석**하고 그 위상 구조를 **보존하면서 새로운 음악을 생성**하는 계산적 파이프라인을 제안한다. 전체 과정은 네 단계로 구성된다. (1) MIDI 전처리: 두 악기를 분리하고 8분음표 단위로 양자화. (2) Persistent Homology: 네 가지 거리 함수(frequency, Tonnetz, voice-leading, DFT)로 note 간 거리 행렬을 구성한 뒤 Vietoris-Rips 복합체의 H_1 cycle을 추출. (3) 중첩행렬 구축: cycle의 시간별 활성화를 이진 또는 연속값 행렬로 기록. (4) 음악 생성: 확률적 샘플링 기반의 Algorithm 1과 FC / LSTM / Transformer 신경망 기반의 Algorithm 2 두 방식을 제공.

$N=20$회 통계적 반복을 통한 정량 검증에서 Tonnetz 거리 함수가 frequency baseline 대비 pitch Jensen-Shannon divergence를 0.0753 ± 0.0033에서 0.0398 ± 0.0031로 **약** 47% **감소**시켰으며, 이는 Welch's $t=35.1$, Cohen's $d=11.1$, $p<10^{-20}$로 극도로 유의한 개선이다. 연속값 중첩행렬을 임계값 $\tau=0.5$로 이진화한 변형은 기존 이진 중첩행렬 대비 추가로 JS divergence를 11.4% 개선했으며 ($0.0387 \to 0.0343$, Welch $t=5.16$), 이 역시 통계적으로 유의했다. 가장 단순한 FC 신경망이 LSTM / Transformer보다 낮은 JS divergence(0.0015)를 기록한 것은, hibari가 수록된 out of noise 앨범의 미학적 성격 — 전통적 선율 인과보다 음들의 공간적 배치에 의존 — 과 정확히 공명하는 관찰이다.

본 연구의 intra / inter / simul 세 갈래 가중치 분리 설계는 사카모토가 인터뷰에서 밝힌 작곡 방식("한 악기를 시간 방향으로 충분히 채운 뒤 다른 악기를 그 위에 겹쳐 배치")을 수학적 구조에 그대로 반영한 것이며, 실제로 두 악기의 활성/쉼 패턴 관측 (inst 1 쉼 0개, inst 2 쉼 64개) 이 이 설계를 경험적으로 정당화한다. 본 논문은 수학적 정의부터 통계 실험, 시각자료, 향후 연구 방향까지를 하나의 일관된 흐름으로 정리한다.

---

1. 서론 — 연구 배경과 동기

1.1 연구 질문

음악은 시간 위에 흐르는 소리들의 집합이지만, 그 "구조"는 단순한 시간 순서만으로 포착되지 않는다. 같은 동기가 여러 번 반복되고, 서로 다른 선율이 같은 화성 기반 위에서 엮이며, 전혀 관계없어 보이는 두 음이 같은 조성 체계 안에서 등가적 역할을 한다. 이러한 층위의 구조를 수학적으로 포착하려면 "어떤 두 대상이 같다(혹은 가깝다)"를 정의하는 **거리 함수**와, 그로부터 파생되는 **위상 구조**를 다루는 도구가 필요하다.

본 연구는 다음의 세 가지 질문에서 출발한다.

1. **음악의 위상 구조는 어떻게 수학적으로 정의되는가?** 한 곡의 note들 사이에 거리 함수를 두고 Vietoris-Rips 복합체를 구성한 뒤, 거리 임계값 ε을 변화시키며 H_1 persistence를 추적하면 그 결과 나오는 cycle들은 음악적으로 어떤 의미를 가지는가?
2. **이 위상 구조를 "보존한 채" 새로운 음악을 생성할 수 있는가?** 보존의 기준은 무엇이며, 보존 정도를 어떻게 정량적으로 측정하는가?
3. **거리 함수의 선택이 실제로 생성 품질에 유의미한 영향을 주는가?** 단순 빈도 기반 거리 대신 음악 이론적 거리 (Tonnetz, voice-leading, DFT)를 사용하면 얼마나 나은가?

1.2 연구 대상 — 왜 hibari인가

본 연구의 대상곡은 사카모토 류이치의 out of noise (2009) 수록곡 "hibari" 이다. 이 곡을 선택한 이유는 다음과 같다.

- **크기 적정성.** 약 $1{,}088$개의 8분음표 시점, 두 악기(피아노 계열 + 스트링 계열), 고유 (pitch, duration) 쌍 23개, 고유 화음 17개 — persistent homology의 입력으로 적당한 규모이다. 너무 작지 않아 위상 구조가 풍부하고, 너무 크지 않아 수 초 안에 전체 barcode를 계산할 수 있다.
- **단순함과 복잡함의 균형.** hibari는 C major scale 7개 pitch class만 사용하는 조성적 단순함을 가지면서도, 두 악기가 서로 다른 역할을 맡아 modular 구조를 형성한다는 점에서 충분히 풍부한 위상 구조를 드러낸다.
- **미학적 특수성.** out of noise 앨범은 "소음과 음악의 경계"를 탐구하는 실험적 작업이며, hibari는 전통적 선율 진행이 아니라 음들의 공간적 배치에 가까운 작곡 방식을 사용한다. 이 특성은 본 연구의 실험 결과 (§3.4)에서 DL 모델 선택과 직접적으로 공명한다.
- **작곡가 본인의 작곡 방식 기록.** 사카모토는 동 앨범 발매 시기 인터뷰에서 작업 방식을 상세히 공개했으며, 이 정보는 본 연구의 가중치 행렬 분리 설계의 직접적 근거가 된다 (§2.9).

1.3 본 연구의 구조

본 논문은 다음과 같이 구성된다.

- **§2** 수학적 배경 — Vietoris-Rips, simplicial homology, persistent homology, Tonnetz와 음악적 거리 함수, 중첩행렬, KL/JS divergence, greedy forward selection, multi-label BCE, 음악 네트워크 구축, Tonnetz 최소매칭 거리, consonance/ICV.
- **§3** 두 가지 음악 생성 알고리즘 — Algorithm 1 (확률적 샘플링) 과 Algorithm 2 (신경망 시퀀스 모델).
- **§4** 실험 설계와 결과 — 거리 함수 baseline 비교, cycle subset ablation, continuous overlap 실험, DL 모델 비교, 통계적 유의성 분석.
- **§5** 시각자료 — 6개의 figure와 각각의 음악적/수학적 해석.
- **§6** 차별화 포인트 — 기존 음악 생성 연구 및 기존 TDA-music 연구와의 비교.
- **§7** 확장 실험 — 모듈 단위 생성 (§7.1), solari 일반화 (§7.2), 거리 보존 note 재분배 (§7.3), 시간 재배치 (§7.4), 화성 제약 (§7.5), A+B 결합 (§7.6), 향후 과제 (§7.7-§7.11).

---

2. 수학적 배경

본 절에서는 본 연구의 파이프라인을 이해하기 위해 필요한 수학적 도구들을 정의하고, 각 도구가 음악 구조 분석에서 어떻게 사용되는지를 서술한다.

2.1 Vietoris-Rips Complex

정의 2.1. 거리 공간 (X, d)와 양의 실수 $\varepsilon > 0$이 주어졌을 때, **Vietoris-Rips complex** $\mathrm{VR}_\varepsilon(X)$는 다음과 같이 정의되는 복합체(simplicial complex)이다:

$$\mathrm{VR}_\varepsilon(X) = \left\{ \sigma \subseteq X \mid \forall x_i, x_j \in \sigma,\ d(x_i, x_j) \le \varepsilon \right\}$$

즉, 점 집합 X의 부분집합 σ에 속한 **모든 점 쌍 사이의 거리가 ε 이하**이면 σ를 심플렉스(simplex)로 포함시킨다.

구성 요소:

- 0-simplex (vertex): 각 점 $x_i \in X$. 단일 점은 거리 조건이 없으므로 어떤 ε에서도 포함된다.
- 1-simplex (edge): $d(x_i, x_j) \le \varepsilon$인 두 점의 쌍 $\{x_i, x_j\}$
- 2-simplex (triangle): 세 점이 서로 모두 ε 이내인 부분집합
- k-simplex: $k+1$개의 점이 서로 모두 ε 이내인 부분집합

Filtration 구조와 포함관계(nested sequence). ε 값을 0부터 연속적으로 키우면, 점 집합 X 자체는 변하지 않은 채 **새로운 심플렉스만 점차 추가된다**. 점 자체는 $\varepsilon = 0$부터 이미 0-simplex로 존재하므로 사라지지 않으며, 어떤 두 점 사이 거리가 ε 임계를 처음 넘는 순간에 그 두 점을 잇는 1-simplex가 추가된다. 마찬가지로 세 점이 모두 ε 이내가 되는 순간 2-simplex(삼각형)가 추가된다. 즉, $\varepsilon_1 < \varepsilon_2$이면 $\mathrm{VR}_{\varepsilon_1}(X)$의 모든 심플렉스가 $\mathrm{VR}_{\varepsilon_2}(X)$에도 그대로 들어 있으며, 새 심플렉스가 추가될 뿐이다. 따라서 다음의 포함관계는 항상 성립한다:

$$\emptyset \subseteq \mathrm{VR}_{\varepsilon_0}(X) \subseteq \mathrm{VR}_{\varepsilon_1}(X) \subseteq \mathrm{VR}_{\varepsilon_2}(X) \subseteq \cdots \subseteq \mathrm{VR}_{\varepsilon_n}(X)$$

여기서 $\varepsilon_0 < \varepsilon_1 < \varepsilon_2 < \cdots$는 새로운 심플렉스가 추가되어 복합체의 **연결 구조**(이를 위상이라 부른다)가 변하는 임계값들이다. 즉, 점이 추가되거나 사라지는 것이 아니라, 같은 점들 사이에 새로운 연결(edge, 삼각형 등)이 생기면서 cycle이나 void가 형성되거나 채워지는 변화이다.

표기 편의를 위해 $K_{\varepsilon_i} := \mathrm{VR}_{\varepsilon_i}(X)$로 두면:

$$\emptyset \subseteq K_{\varepsilon_0} \subseteq K_{\varepsilon_1} \subseteq K_{\varepsilon_2} \subseteq \cdots \subseteq K_{\varepsilon_n}$$

이를 **filtration**이라 부르며, 변화가 일어나는 미지수 ε_i들이 곧 위상의 birth/death 시점이 된다.

본 연구에서의 사용: $X = \{n_1, n_2, \ldots, n_{23}\}$은 hibari에 등장하는 23개의 고유 note이며, $d(n_i, n_j)$는 두 note 간 거리이다. ε를 점진적으로 증가시키며 simplex complex의 변화를 추적하여, 어떤 거리 척도에서 어떤 cycle이 탄생·소멸하는지를 분석한다.

2.2 Simplicial Homology

정의 2.2. Simplex complex K에 대해 n차 호몰로지 군(homology group) $H_n(K)$는 K 안에 존재하는 n차원 "구멍"의 대수적 표현이다. 직관적으로:

- $H_0(K)$: 연결 성분(connected components)의 개수
- $H_1(K)$: cycle의 개수 (1차원 구멍, 즉 닫힌 고리 모양으로 둘러싸인 영역)
- $H_2(K)$: 2차원 빈 공간(void)의 개수 (3차원 공동을 둘러싼 표면)

구멍을 만들기 위해 필요한 최소 점의 개수.

- 1차원 cycle (1차원 구멍): 최소 3개의 점이 필요하다. 세 점을 잇는 1-simplex 3개가 삼각형 모양의 폐곡선을 만들되, 그 내부를 채우는 2-simplex(삼각형)가 없어야 cycle로 인식된다.
- 2차원 void (2차원 구멍): 최소 4개의 점이 필요하다. 4개의 점이 사면체(tetrahedron)의 boundary를 이루되, 사면체 내부를 채우는 3-simplex가 없어야 void로 인식된다.
- n차원 구멍: 최소 $n+2$개의 점이 필요하다.

Betti number $\beta_n = \mathrm{rank}(H_n(K))$는 n차원 위상 특징의 개수를 나타낸다. 예컨대 $\beta_1 = 3$이면 이 simplex complex 안에 서로 독립적인 cycle이 3개 있다는 뜻이다.

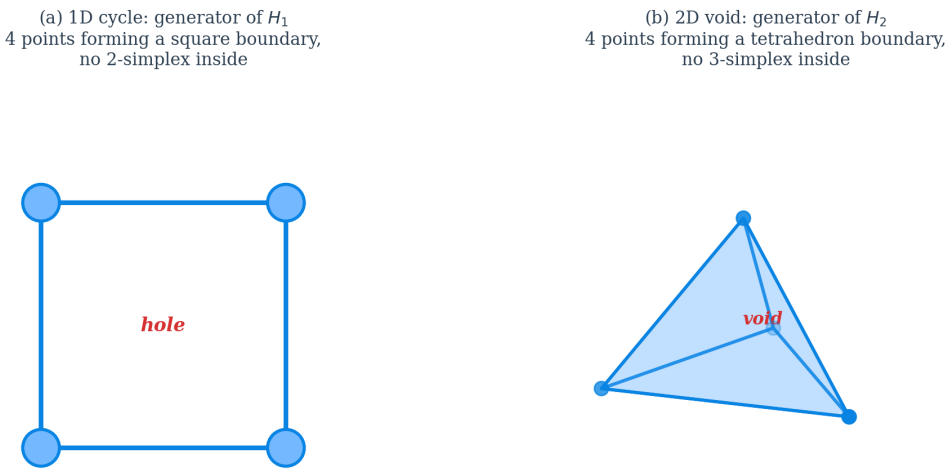


그림 2.2. (좌) 1차원 cycle의 예: 4개 점과 4개 edge가 사각형 모양의 닫힌 boundary를 이루며, 그 내부를 채우는 2-simplex(삼각형)가 없다. 이 사각형은 한 개의 1차원 구멍을 둘러싸므로 $\beta_1 = 1$, 즉 H_1의 generator가 1개 있다. (우) 2차원 void의 예: 4개 점이 4개의 삼각형 면으로 사면체의 표면(boundary)을 이루지만 내부를 채우는 3-simplex(사면체)가 없다. 표면이 둘러싼 3차원 빈 공간이 하나의 void이므로 $\beta_2 = 1$, 즉 H_2의 generator가 1개 있다.

본 연구에서의 사용: 본 연구는 주로 H_1 (1차 호몰로지)을 다룬다. 이는 음악 네트워크에서 서로 가까운 note들이 만드는 닫힌 cycle, 즉 순환적으로 연결된 note 그룹을 포착한다. 발견된 각 cycle은 곡의 구조적 반복 단위로 해석된다.

2.3 Persistent Homology

Filtration $K_{\varepsilon_0} \subseteq K_{\varepsilon_1} \subseteq \cdots \subseteq K_{\varepsilon_n}$에서, 각 단계마다 H_1의 cycle 구성이 달라진다. **Persistent homology**는 이 과정에서 각 cycle이 어느 ε_i에서 처음 나타나고(**birth**) 어느 ε_j에서 사라지는지(**death**)를 추적한다.

Birth와 death의 음악적 의미:

- **Birth** b: 거리 임계값 ε가 충분히 커져서 새로운 cycle이 형성되는 순간. 음악적으로는 "이 거리 척도에서 처음으로 닫힌 반복 구조가 발견되는 시점".
- **Death** d: 더 큰 ε에서 그 cycle이 다른 cycle들의 합(정확히는 boundary)으로 표현될 수 있게 되어, 호몰로지 군 안에서 더 이상 독립적인 generator가 아니게 되는 순간. 음악적으로는 "거리 척도가 너무 느슨해져서 이 반복 구조가 다른 구조에 흡수되는 시점".

각 cycle은 (b, d) 쌍과, 그 cycle을 구성하는 vertex/edge 집합인 **cycle representative**가 함께 기록된다. 본 연구에서는 birth-death 쌍은 cycle의 "수명"을 측정하는 데, cycle representative는 어떤 note들이 그 cycle을 이루는지 식별하는 데 사용한다. 이 정보를 모은 것이 곡의 **위상적 지문(topological signature)**이다.

Persistence: $\mathrm{pers}(\mathrm{cycle}) = d - b$. (death가 birth보다 항상 크므로 양수.) 큰 persistence를 갖는 cycle은 다양한 거리 척도에서 살아남으므로 **위상적으로 안정한 구조**이며, 작은 persistence는 일시적이거나 노이즈에 가까운 구조이다.

알고리즘적 측면: 본 연구의 대부분의 실험에서는 선행연구(정재훈 외, 2024)의 **pHcol algorithm** 순수 Python 구현을 사용하였다. 이 구현은 cycle representative까지 함께 추출해주므로 본 연구의 후속 단계(중첩행렬 구축)에 그대로 활용할 수 있다. 별도로 계산 속도가 중요한 일부 단계에서는 C++ 기반 **Ripser** (Bauer, 2021) 구현을 보조적으로 활용하였으며, 두 구현이 동일한 birth-death 결과를 내는지 검증하였다.

본 연구에서의 사용: 거리 행렬 $D \in \mathbb{R}^{23 \times 23}$로부터 Vietoris-Rips filtration을 구성하고, 각 rate parameter r (가중치 비율, 후술)에서의 H_1 persistence를 계산한다. 발견된 모든 (b, d) 쌍과 cycle representative가 함께 cycle 집합을 정의하며, 이 cycle들이 다음 절의 중첩행렬 구축에 사용된다.

2.4 Tonnetz와 음악적 거리 함수

정의 2.4. Tonnetz(또는 tone-network)는 pitch class 집합 $\mathbb{Z}/12\mathbb{Z}$를 평면 격자에 배치한 구조이다. 여기서 **pitch class**는 옥타브 차이를 무시한 음의 동치류로, 예컨대 C4 (가운데 도), C5 (한 옥타브 위 도), C3 등은 모두 같은 pitch class "C"에 속한다. 12음 평균율(12-TET)에서는 한 옥타브 안에 12개의 pitch class가 있으며, 이를 정수 $\{0, 1, 2, \ldots, 11\}$에 대응시켜 $\mathbb{Z}/12\mathbb{Z}$로 표기한다 (0=C, 1=C♯, 2=D, ..., 11=B). 두 pitch class가 격자 위에서 가까운 것은 음악 이론적으로 어울리는 음(consonant)임을 의미한다.

Tonnetz의 격자 구조. pitch class $p \in \mathbb{Z}/12$를 좌표 (x, y)에 배치하되, 다음 관계를 만족시킨다:

- 가로 이동 (+1 in x): 완전5도 (perfect fifth, +7 semitones)
- 대각선 이동 (+1 in y): 장3도 (major third, +4 semitones)

이렇게 배치하면 자연스럽게 단3도(+3 semitones) 관계도 다른 대각선 방향으로 형성되어 삼각형 격자가 만들어진다. 그림 2.1은 hibari의 C장조 음역에 해당하는 일부분을 보여준다.

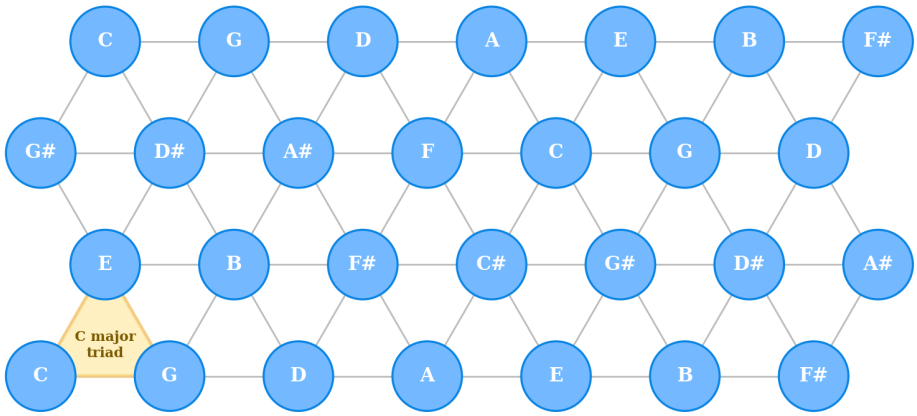


그림 2.1. Tonnetz 격자 구조. 가로 방향은 완전5도(C→G→D→A→E⋯), 대각선 방향은 장3도(C→E→G♯⋯)와 단3도(C→A→F♯⋯)로 이동한다. 삼각형 하나는 하나의 장3화음(major triad) 또는 단3화음(minor triad)에 대응된다.

Tonnetz 거리. 두 pitch class p_1, p_2 사이의 Tonnetz 거리 $d_T(p_1, p_2)$는 격자 위 최단 경로 길이(즉, edge 수)로 정의된다:

$$d_T(p_1, p_2) = \min\{|x_1 - x_2| + |y_1 - y_2| \mid (x_i, y_i) \text{ represents } p_i\}$$

본 연구에서는 12개 pitch class 모두에 대해 사전 계산된 12×12 거리 테이블을 사용한다.

빈도 기반 거리. 본 연구의 기준 거리 d_{freq}는 두 note의 인접도(adjacency)의 역수로 정의된다. 인접도 $w(n_i, n_j)$는 곡 안에서 note n_i와 n_j가 시간적으로 연달아 등장한 횟수이다:

$$w(n_i, n_j) = \#\{\, t : n_i \text{ at time } t \text{ and } n_j \text{ at time } t+1 \,\}$$

거리는 $d_{\text{freq}}(n_i, n_j) = 1/w(n_i, n_j)$로 정의되며 (인접도가 0인 경우는 도달 불가능한 큰 값으로 처리), 자주 연달아 등장하는 음일수록 가까워진다. 이는 곡의 통계적 흐름만 반영하며 화성 관계는 직접 포착하지 못한다는 한계가 있다.

그 외의 음악적 거리 함수.

(1) Voice-leading distance (Tymoczko, 2008): 두 pitch class 사이의 절대 차이로 정의된다.

$$d_V(p_1, p_2) = |p_1 - p_2|$$

이 정의가 "두 pitch class의 반음 차이"가 되는 이유는 12음 평균율에서 인접한 두 pitch class(예: C와 C♯, 또는 E와 F)의 정수 표현이 정확히 1만큼 차이나며, 그 음정 차이가 1 반음(semitone)이기 때문이다. 즉, $|p_1 - p_2|$는 두 pitch class 사이를 이동하기 위해 거쳐야 하는 반음의 개수와 같다. 음악 이론에서 voice-leading은 한 화음에서 다른 화음으로 옮겨갈 때 각 성부가 가능한 한 적은 음정으로 이동하는 것을 미덕으로 삼으며, 이 거리는 그러한 "최소 이동" 원리를 직접 수치화한 것이다.

(2) DFT distance (Tymoczko, 2008): pitch class 집합을 12차원 indicator vector로 표현한 후 이산 푸리에 변환(Discrete Fourier Transform)을 적용하여 푸리에 공간(Fourier space)으로 옮긴 뒤, 그 공간에서의 L_2 거리를 측정한다.

$$d_F(p_1, p_2) = \|\hat{f}(p_1) - \hat{f}(p_2)\|_2$$

여기서 $\hat{f}(p) \in \mathbb{C}^{12}$는 pitch class p의 indicator vector $e_p \in \mathbb{R}^{12}$ (e_p는 p번째 성분만 1이고 나머지가 0)에 12점 DFT를 적용한 결과이며, 그 k번째 성분 $\hat{f}_k$를 k번째 **푸리에 계수**라 부른다.

왜 DFT를 사용하는가. 12음 평균율은 한 옥타브를 12등분한 주기 구조를 가지므로, pitch class 집합은 본질적으로 $\mathbb{Z}/12\mathbb{Z}$ 위의 함수로 볼 수 있다. 이러한 주기 함수는 시간 영역에서 비교하기 어렵지만(인접 비교만으로는 화성적 의미를 포착하기 힘들다), 푸리에 공간으로 옮기면 각 계수가 특정 음악적 속성에 직접 대응된다. 예를 들어 $|\hat{f}_3|$은 옥타브를 4등분하는 단3도 대칭성(증3화음 등), $|\hat{f}_5|$는 옥타브를 5도권으로 도는 온음계성(diatonicity)과 연관된다 (Tymoczko, 2008). 따라서 푸리에 공간의 L_2 거리는 두 pitch class의 화성적 성격이 얼마나 닮았는지를 측정한다.

복합 거리(Hybrid distance). 본 연구는 빈도 기반 거리 d_{freq}와 음악적 거리 d_{music} (Tonnetz, Voice-leading, DFT 중 하나)을 선형 결합한다:

$$d_{\text{hybrid}}(n_i, n_j) = \alpha \cdot d_{\text{freq}}(n_i, n_j) + (1 - \alpha) \cdot d_{\text{music}}(n_i, n_j)$$

여기서 $\alpha \in [0, 1]$은 두 거리의 비중을 조절하는 파라미터이다. Grid search 결과, hibari에 대해서는 $\alpha = 0.3$ (음악적 거리 70% + 빈도 거리 30%)이 최적임을 확인하였다 (구체적 수치는 추후 작성될 실험 결과 절에 정리).

본 연구에서의 사용: 거리 함수의 선택은 발견되는 cycle 구조에 직접적으로 영향을 미친다. 빈도 기반 거리만 사용하면 곡의 통계적 특성만 반영되어 화성적·선율적 의미가 있는 구조를 포착하지 못한다. Tonnetz 거리를 도입함으로써 hibari의 C장조/A단조 화성 구조와 정합적인 cycle을 발견할 수 있었다.

2.5 활성화 행렬과 중첩행렬

본 연구에서는 곡의 시간축 위에서 cycle 구조가 어떻게 전개되는지를 두 단계의 행렬로 표현한다. 첫 단계는 **활성화 행렬(activation matrix)**, 두 번째 단계는 그것을 가공한 **중첩행렬(overlap matrix)**이다.

정의 2.5 (활성화 행렬). 음악의 시간축 길이를 T, 발견된 cycle의 수를 C라 하자. 활성화 행렬 $A \in \{0,1\}^{T \times C}$는 raw 활성 정보를 담는다:

시점 t에서 cycle c를 구성하는 note 중 **적어도 하나가 원곡에서 연주되고 있으면** $A[t, c] = 1$, 아니면 $A[t, c] = 0$이다. 형식적으로:

$$A[t, c] = \mathbb{1}[\exists\, n \in V(c) \text{ such that } n \text{ is played at time } t]$$

여기서 $V(c)$는 cycle c의 vertex(=note) 집합이며, $\mathbb{1}[\cdot]$은 indicator function이다. 활성화 행렬은 산발적인 단일 시점 활성화까지 모두 포함하므로 노이즈가 많다.

정의 2.6 (중첩행렬). 중첩행렬 $O \in \{0,1\}^{T \times C}$는 활성화 행렬에서 **연속적이고 충분히 긴 활성 구간만 남긴 것**이다.

$$O[t,c] = \mathbb{1}[\,t \in R(c)\,], \qquad R(c) = \bigcup_i [s_i,\ s_i + L_i]$$

여기서 $R(c)$는 cycle c의 "지속 활성 구간(sustained intervals)"의 합집합이며, 각 구간 $[s_i, s_i + L_i]$는 활성화 행렬 $A[\cdot, c]$에서 길이가 임계값 scale_c 이상인 연속 1의 구간이다.

활성화 행렬과 중첩행렬의 차이.

- $A[t,c]$: 시점 t에 cycle c의 note가 단 한 번이라도 울리면 1. **순간적 활성을 모두 잡음.**
- $O[t,c]$: cycle c의 활성이 일정 시간 이상 **지속되는 구간**에서만 1. 산발적 노이즈 제거됨.

예를 들어 $\mathrm{scale}_c = 3$일 때 (3 시점 이상 지속된 활성만 인정), 다음과 같은 cycle c의 한 행을 생각해보자.

```
   :  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
A[·,c]: 0  1  1  0  1  1  1  1  0  0  1  0  1  1  1
O[·,c]: 0  0  0  0  1  1  1  1  0  0  0  0  1  1  1
```

활성화 행렬 A는 시점 2~3, 5~8, 11, 13~15에서 모두 활성화되어 있다. 중첩행렬 O는 그중 길이가 $\mathrm{scale}_c = 3$ 이상인 두 구간(시점 5~8과 13~15)만 1로 남기고, 길이가 짧은 시점 2~3과 단발성 시점 11은 0으로 처리한다. 본 연구에서 중첩행렬을 음악 생성의 seed로 사용하는 이유는, 잠시 스쳐가는 활성보다 일정 시간 유지되는 cycle만이 곡의 구조적 단위로 의미 있다고 보기 때문이다.

구축 과정:

1. **활성화 행렬 계산**: 위 정의 2.5에 따라 $A \in \{0,1\}^{T \times C}$를 구한다.
2. **연속 활성 구간 추출**: 각 cycle c에 대해 길이가 scale_c 이상인 연속 1 구간을 모두 찾는다.
3. **Scale 동적 조정**: cycle마다 ON 비율 $\rho(c) = |R(c)|/T$가 목표치 $\rho^* = 0.35$에 근접하도록 scale_c를 조정한다 (구간이 너무 많으면 scale을 키우고, 너무 적으면 줄인다).

목표 ON 비율 $\rho^* = 0.35$의 근거. 이 값은 본 연구에서 새로 결정한 것이 아니라 선행연구(정재훈 외, 2024)에서 사용된 휴리스틱 값을 계승한 것이다. 직관적으로 한 cycle이 곡 전체의 약 1/3 정도 활성화되면 "그 cycle이 곡의 구조적 모티프로서 충분히 자주 등장하면서도, 모든 시점을 점유하지 않아 다른 cycle과 구분된다"는 균형을 만든다. 이 값의 최적성은 본 연구에서 정량적으로 검증하지 않았으며, 향후 곡 또는 데이터에 따라 적응적으로 조정 가능한 파라미터로 일반화할 예정이다 (예: ON 비율 자체를 최적화 대상으로 설정).

연속값 확장. 본 연구에서는 이진 중첩행렬 외에, cycle의 활성 정도를 [0,1] 사이의 실수값으로 표현하는 연속값 버전도 도입하였다:

$$O_{\mathrm{cont}}[t,c] = \frac{\sum_{n \in V(c)} w(n) \cdot \mathbb{1}[\,n \text{ is played at time } t\,]}{\sum_{n \in V(c)} w(n)}$$

여기서 $V(c)$는 cycle c의 vertex 집합, $w(n) = 1/N_{\mathrm{cyc}}(n)$은 note n의 **희귀도 가중치**이며 $N_{\mathrm{cyc}}(n)$은 note n이 등장하는 cycle의 개수이다. 적은 cycle에만 등장하는 희귀한 note가 활성화되면 더 큰 가중치를 받는다.

음악적 의미: 중첩행렬은 곡의 **위상적 뼈대(topological skeleton)**를 시각화한 것이다. 시간이 흐름에 따라 어떤 반복 구조가 켜지고 꺼지는지를 나타내며, 이것이 음악 생성의 seed 역할을 한다.

2.6 Kullback-Leibler Divergence와 Jensen-Shannon Divergence

정의 2.7 (KL Divergence). 두 이산 확률 분포 P와 Q에 대해, **Kullback-Leibler divergence**는 다음과 같이 정의된다:

$$D_{\mathrm{KL}}(P \| Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

직관적으로 $D_{\mathrm{KL}}(P\|Q)$는 "참 분포가 P인데 우리가 Q로 잘못 알고 있을 때 발생하는 정보 손실(information loss)"의 평균으로 해석된다. 두 분포가 똑같으면 손실이 없으므로 $D_{\mathrm{KL}}=0$이고, 분포의 차이가 클수록 값이 커진다. 항상 $D_{\mathrm{KL}}(P\|Q)\geq 0$이며, 등호는 $P=Q$일 때만 성립한다 (Gibbs' inequality).

비대칭성: $D_{\mathrm{KL}}(P\|Q)\neq D_{\mathrm{KL}}(Q\|P)$. 예를 들어, P에는 자주 나오는 사건이 Q에는 거의 없으면 $D_{\mathrm{KL}}(P\|Q)$는 매우 크지만 그 반대는 작을 수 있다. 이 비대칭성 때문에 두 곡을 "공정하게" 비교하기 위해서는 대칭화된 지표가 필요하다.

정의 2.8 (Jensen-Shannon Divergence). KL을 대칭화한 지표로, **JS divergence**는 다음과 같이 정의된다:

$$D_{\mathrm{JS}}(P\|Q)=\tfrac{1}{2}D_{\mathrm{KL}}(P\|M)+\tfrac{1}{2}D_{\mathrm{KL}}(Q\|M)$$

여기서 $M=\frac{1}{2}(P+Q)$는 두 분포의 평균이다.

핵심 성질:

- 대칭성: $D_{\mathrm{JS}}(P\|Q)=D_{\mathrm{JS}}(Q\|P)$
- 유계성: $0\leq D_{\mathrm{JS}}(P\|Q)\leq\log 2$ ($\log_2$ 사용 시 최대값 1)
- D_{JS} 자체는 metric은 아니지만, $\sqrt{D_{\mathrm{JS}}}$는 삼각 부등식까지 만족하는 metric이다 (Endres & Schindelin, 2003)

본 연구에서의 사용: 생성된 음악과 원곡의 유사도를 평가하는 주요 지표로 사용한다. 두 가지 분포를 비교한다.

(1) Pitch 빈도 분포. 곡에 등장하는 모든 note에 대해, 그 pitch 값의 출현 횟수를 세어 정규화한 확률 분포이다. 곡의 모든 note 집합을 $\mathcal{N}=\{(s_k,p_k,e_k)\}_{k=1}^{K}$ (s_k=시작, p_k=pitch, e_k=종료)라 하자. 여기서 $K=|\mathcal{N}|$은 곡 전체의 총 note 개수(중복 포함)이다. 즉 같은 pitch라도 곡 안에서 두 번 연주되면 두 번 세어진다. 이 때 pitch 빈도 분포는 다음과 같이 정의된다:

$$P_{\mathrm{pitch}}(p)=\frac{|\{k:p_k=p\}|}{K}$$

원곡과 생성곡 각각에서 이 분포를 계산하고 둘 사이의 JS divergence를 측정한다. 값이 0에 가까울수록 두 곡의 pitch 사용 비율이 일치한다.

(2) Transition 빈도 분포. 시간 순서대로 인접한 두 note 쌍 (p_k,p_{k+1})의 출현 횟수를 세어 정규화한 분포이다. note들을 시작 시점 s_k 순으로 정렬하여 pitch 시퀀스 $(p_1,p_2,\ldots,p_K)$를 만들고:

$$P_{\mathrm{trans}}(a,b)=\frac{|\{k:p_k=a,\ p_{k+1}=b\}|}{K-1}$$

이는 $|P|\times|P|$ 크기의 transition matrix를 정규화한 것과 동일하다. 원곡과 생성곡의 transition 분포 간 JS divergence는 "어떤 음 다음에 어떤 음이 오는가"의 패턴이 얼마나 유사한지를 측정한다.

두 지표의 차이. Pitch 분포는 "어떤 음들이 얼마나 자주 쓰였는가"라는 빈도 정보만 담는 반면, transition 분포는 "어떤 음 다음에 어떤 음이 오는가"라는 시간적 진행 정보까지 담는다. 두 지표는 다음과 같은 차이를 가진다.

- 음의 사용 비율 (시간 순서 무시):

$$D_{\mathrm{JS}}(P_{\mathrm{pitch}}^{\mathrm{orig}}\,\|\,P_{\mathrm{pitch}}^{\mathrm{gen}})$$

- 음의 진행 패턴 (시간 순서 반영):

$$D_{\mathrm{JS}}(P_{\mathrm{trans}}^{\mathrm{orig}}\,\|\,P_{\mathrm{trans}}^{\mathrm{gen}})$$

따라서 전자는 작더라도 후자는 클 수 있으며, 두 지표를 함께 사용함으로써 "음을 비슷하게 쓰는가"와 "비슷한 순서로 쓰는가"를 별도로 측정할 수 있다.

최댓값이 $\log 2$인 이유. JS divergence는 두 분포 P,Q의 평균 $M=(P+Q)/2$에 대해 $D_{\mathrm{JS}}(P\|Q)=\frac{1}{2}D_{\mathrm{KL}}(P\|M)+\frac{1}{2}D_{\mathrm{KL}}(Q\|M)$로 정의된다. 두 분포가 서로 완전히 분리된 경우(즉, P의 support와 Q의 support가 겹치지 않는 경우), P의 영역에서는 $M=P/2$이므로 $\log(P/M)=\log 2$가 되고, 마찬가지로 Q의

영역에서도 $\log 2$가 된다. 따라서 $D_{\mathrm{KL}}(P\|M) = D_{\mathrm{KL}}(Q\|M) = \log 2$가 되어 $D_{\mathrm{JS}} = \log 2 \approx 0.693$이 최댓값이 된다.

본 연구의 최우수 조합(Tonnetz hybrid + FC, $\alpha = 0.3$)에서 pitch JS divergence는 $D_{\mathrm{JS}} \approx 0.002$를 달성하였다. 이는 가능한 최댓값 $\log 2 \approx 0.693$의 약 0.3%에 해당하는 값이다.

2.7 Greedy Forward Selection과 Submodularity

정의 2.9. 유한 집합 V에 대한 함수 $f: 2^V \to \mathbb{R}$이 **submodular**라 함은, 모든 $A \subseteq B \subseteq V$와 $x \in V \setminus B$에 대해 다음이 성립한다는 것이다:

$$f(A \cup \{x\}) - f(A) \geq f(B \cup \{x\}) - f(B)$$

이는 "한계 효용 체감(diminishing returns)" 성질로, 큰 집합에 원소를 추가할 때의 이득이 작은 집합에 같은 원소를 추가할 때의 이득보다 작거나 같다는 것을 의미한다.

정리 (Nemhauser et al., 1978). f가 submodular이고 monotone이며 $f(\emptyset) = 0$일 때, $|S| \leq k$를 만족시키면서 $f(S)$를 최대화하는 문제에 대해 **greedy forward selection** 알고리즘은 $(1 - 1/e) \approx 0.632$ 근사 보장을 갖는다:

$$f(S_{\mathrm{greedy}}) \geq (1 - \tfrac{1}{e}) f(S^*)$$

여기서 S^*는 최적해이다.

본 연구에서의 사용: 발견된 모든 cycle 중에서 원곡의 위상 구조를 가장 잘 보존하는 부분집합 S를 선택하는 데 사용한다. 보존도 함수 $f(S)$는 세 가지 지표의 가중합으로 정의된다:

$$f(S) = w_J \cdot J(S) + w_C \cdot C(S) + w_B \cdot B(S)$$

여기서 각 지표는 다음과 같이 정의된다.

(1) Note Pool Jaccard similarity $J(S)$. $\mathcal{N}_{\mathrm{full}} = \bigcup_{c \in V} V(c)$를 전체 cycle이 사용하는 모든 note의 집합, $\mathcal{N}_S = \bigcup_{c \in S} V(c)$를 부분집합 S가 사용하는 note 집합이라 하자. 이 때

$$J(S) = \frac{|\mathcal{N}_S \cap \mathcal{N}_{\mathrm{full}}|}{|\mathcal{N}_S \cup \mathcal{N}_{\mathrm{full}}|} = \frac{|\mathcal{N}_S|}{|\mathcal{N}_{\mathrm{full}}|}$$

(부분집합 관계이므로 분자가 $|\mathcal{N}_S|$로 단순화됨). 이는 "선택된 cycle들이 원곡의 note 풀을 얼마나 커버하는가"를 측정한다.

(2) Overlap pattern correlation $C(S)$. 원본 중첩행렬 $O \in \{0,1\}^{T \times C}$의 각 행 $O[t,:]$을 S로 제한한 중첩행렬 $O_S \in \{0,1\}^{T \times |S|}$의 각 행 $O_S[t,:]$과 비교한다. 두 시점 시퀀스 $\{|O[t,:]|\}_{t=1}^T$, $\{|O_S[t,:]|\}_{t=1}^T$의 Pearson 상관계수로 정의된다:

$$C(S) = \mathrm{corr}\Big(\sum_{c \in V} O[\cdot, c],\ \sum_{c \in S} O[\cdot, c]\Big)$$

이는 "각 시점에서의 cycle 활성화 강도(=음악적 밀도)의 시간 패턴이 보존되는가"를 측정한다.

(3) Betti curve similarity $B(S)$. 각 rate r에 대해 살아있는 cycle 수를 세는 함수 $\beta_1^V(r) = |\{c \in V : c \text{ alive at } r\}|$, $\beta_1^S(r)$을 정의하고 두 곡선의 정규화된 L^2 유사도로 계산한다:

$$B(S) = 1 - \frac{\|\beta_1^V - \beta_1^S\|_2}{\|\beta_1^V\|_2 + \|\beta_1^S\|_2}$$

이는 "rate 변화에 따른 위상 구조의 등장·소멸 곡선이 얼마나 유사한가"를 측정한다.

가중치 설정 ($w_J = 0.5,\ w_C = 0.3,\ w_B = 0.2$). 세 지표 중 Note Pool Jaccard에 가장 큰 비중을 둔 이유는 음악 생성의 직접적 입력이 cycle 구성 note들이기 때문이다(음 자체가 보존되지 않으면 위상 구조의 의미가 사라진다).

Overlap pattern correlation은 시간적 음악 흐름을 반영하므로 두 번째로 큰 비중을 두었고, Betti curve similarity는 보다 거시적인 위상 통계량이므로 보조 지표로 두었다. 이 가중치는 실험적으로 결정된 heuristic이며, 후속 연구에서 정량적 grid search로 재조정될 여지가 있다.

참고: 본 연구의 보존도 함수는 엄밀히 submodular임이 증명되지는 않았으나, 실험적으로 greedy 방법이 90% 보존도를 작은 k로 달성하는 것을 확인하였다 (예: 46개 cycle 중 15개로 90% 보존).

2.8 Multi-label Binary Cross-Entropy Loss

정의 2.10. Multi-label classification 문제에서, 각 예측 단위마다 여러 클래스가 동시에 정답일 수 있다. 모델 출력 $\hat{y} \in \mathbb{R}^N$을 sigmoid 함수로 [0, 1] 범위로 변환한 후, 각 클래스마다 독립적인 binary cross-entropy를 계산한다:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\mathcal{L}_{\mathrm{BCE}}(y, \hat{y}) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log \sigma(\hat{y}_i) + (1-y_i)\log(1-\sigma(\hat{y}_i))]$$

여기서 $y \in \{0,1\}^N$은 정답 multi-hot vector이고, $\hat{y} \in \mathbb{R}^N$은 모델의 logit 출력이다.

본 연구에서의 사용: 각 시점 t에서 동시에 여러 note가 활성화될 수 있으므로, 단일 클래스 예측인 categorical cross-entropy 대신 multi-label BCE를 사용한다. 모델 입력은 중첩행렬의 한 행 $O[t,:] \in \mathbb{R}^C$이고, 출력은 23차원 multi-hot vector $y_t \in \{0,1\}^{23}$이다 (해당 시점의 활성 note 표시).

Adaptive threshold (학습이 아닌 추론 단계의 후처리). Adaptive threshold는 BCE 손실함수와는 별개의 개념으로, 학습이 끝난 후 모델로 음악을 **생성(추론)**할 때 적용되는 후처리 규칙이다. 학습 단계에서는 BCE 손실로 모델 파라미터를 업데이트하지만, 막상 음악을 만들어낼 때에는 sigmoid 출력 $\sigma(\hat{y}_i) \in [0,1]$ 중 어느 음을 "켜진" 음으로 볼 것인지 결정해야 한다. 가장 단순한 방법은 고정 임계값 0.5를 쓰는 것이지만, LSTM/Transformer처럼 sigmoid 출력이 전체적으로 0.5보다 낮게 형성되는 모델에서는 음표가 거의 생성되지 않는 문제가 발생한다.

이를 해결하기 위해, 본 연구에서는 원곡의 평균 ON 비율(약 15% — 한 시점당 23개 note 중 약 3~4개가 활성)에 맞춰 임계값을 **데이터 기반으로 동적 결정**한다. 즉 모델이 어떤 절대적 확률 수준을 출력하든, 상위 15%에 해당하는 점수를 가진 음들만 활성으로 채택한다:

$$\theta = \mathrm{quantile}(\,\sigma(\hat{Y}),\ 1-0.15\,)$$

여기서 $\mathrm{quantile}(X, q)$는 분포 X의 q-분위수, 즉 "X의 값들 중 비율 q 이하인 값들의 상한"을 의미하는 함수이다. 예를 들어 $\mathrm{quantile}(X, 0.85)$는 X의 상위 15% 경계값이다. 따라서 위 식의 θ 이상인 점수를 가진 클래스만 ON으로 판정하면, 자연스럽게 전체 활성 음의 비율이 약 15%로 맞춰진다. 이를 통해 sigmoid 출력의 절대값이 모델별로 달라도 음악의 밀도를 일관되게 유지할 수 있다.

2.9 음악 네트워크 구축과 가중치 분리

정의 2.11. 음악 네트워크 $G = (V, E)$는 다음과 같이 정의된다:

- **Vertex set** V: 곡에 등장하는 모든 고유 (pitch, duration) 쌍. hibari의 경우 $|V| = 23$.
- **Edge set** E: 두 vertex가 곡에서 인접하여 등장한 경우 연결.
- **Weight function** $w: E \to \mathbb{R}_{\geq 0}$: 인접 등장 빈도.

가중치 행렬의 분리 — 사카모토 류이치의 작곡 의도에 근거. 본 연구가 가중치를 단일 행렬이 아니라 intra / inter / simul 세 가지로 처음부터 분리한 것에는 단순한 수학적 편의성을 넘어서는 동기가 있다. 사카모토 류이치는 out of noise 발매 시기의 인터뷰에서 hibari와 같은 곡을 작곡할 때 "두 악기를 동시에 적기보다는, 먼저 한 악기의 선율선을 시간 방향으로 충분히 채워 넣은 뒤, 다른 악기를 그 위에 겹쳐 배치하는 방식"으로 작업했다고 밝혔다. 즉

그의 작곡 과정 자체가 (1) 각 악기 내부의 시간적 흐름과 (2) 두 악기 사이의 수직적·시차적 상호작용을 분리하여 다루는 구조였다. 본 연구는 이 분리를 그대로 수학적 가중치 구조에 반영하여, intra weight는 "한 악기 내부의 시간 방향 흐름"을, inter weight는 "악기 1의 어떤 음 다음 lag ℓ만큼 후에 악기 2의 어떤 음이 오는가"를, simul weight는 "같은 시점에서의 즉시적 화음 결합"을 각각 독립적으로 표현한다. 이러한 분리는 단일 가중치 행렬로는 포착할 수 없는 작곡가의 시간 인식 구조를 보존하며, 이후 rate parameter r_t, r_c를 통해 두 측면의 비중을 자유롭게 조절할 수 있게 한다.

가중치 행렬의 분리 (본 연구의 핵심 설계): 본 연구는 가중치를 다음과 같이 세 가지로 분리한다:

1. **Intra weight** W_{intra}: 같은 악기 내에서 연속한 두 화음 간 전이 빈도. 두 악기의 intra weight를 합산한다:

$$W_{\text{intra}} = W_{\text{intra}}^{(1)} + W_{\text{intra}}^{(2)}$$

이는 각 악기의 **선율적 흐름**을 포착한다.

2. **Inter weight** $W_{\text{inter}}^{(\ell)}$: 시차(lag) ℓ을 두고 악기 1의 화음과 악기 2의 화음이 동시에 출현하는 빈도이다. $\ell \in \{1, 2, 3, 4\}$로 변화시키며 다양한 시간 스케일의 **악기 간 상호작용**을 탐색한다:

$$W_{\text{inter}} = \sum_{\ell=1}^{4} W_{\text{inter}}^{(\ell)}$$

3. **Simul weight** W_{simul}: 같은 시점에서 두 악기가 동시에 발음하는 note 조합의 빈도. **순간적 화음 구조**를 포착한다.

Timeflow weight (선율 중심 탐색):

$$W_{\text{timeflow}}(r_t) = W_{\text{intra}} + r_t \cdot W_{\text{inter}}$$

$r_t \in [0, 1.5]$를 변화시키며 위상 구조의 출현·소멸을 추적한다.

Complex weight (선율-화음 결합):

$$W_{\text{complex}}(r_c) = W_{\text{timeflow, refined}} + r_c \cdot W_{\text{simul}}$$

$r_c \in [0, 0.5]$로 제한하여 "음악은 시간 예술이므로 화음보다 선율에 더 큰 비중을 둔다"는 음악적 해석을 반영한다.

거리 행렬: 가중치 $w(n_i, n_j) > 0$에 대해 거리는 역수로 정의된다:

$$d(n_i, n_j) = 1 \,/\, w(n_i, n_j) \text{if} \;\; w(n_i, n_j) > 0, [4pt] d_\infty \text{otherwise}$$

여기서 d_∞는 "도달 불가능한 큰 값"으로, $d_\infty = 1 + 2/(\min_{w>0} w \cdot \text{step})$로 계산된다.

2.10 확장 수학적 도구 — 거리 보존 재분배와 화성 제약

본 절은 §7.3-§7.6 의 확장 실험에서 사용되는 수학적 도구를 정의한다.

정의 2.10 (Tonnetz 최소매칭 거리). 두 cycle c_1, c_2가 각각 $V(c_1) = \{n_1, \ldots, n_k\}$, $V(c_2) = \{m_1, \ldots, m_k\}$ (같은 크기 k)를 가질 때, 두 cycle 간 **Tonnetz 최소매칭 거리**는 다음과 같이 정의된다:

$$d_{\text{match}}(c_1, c_2) = \min_{\pi \in S_k} \frac{1}{k} \sum_{i=1}^{k} d_T(\text{pc}(n_i),\ \text{pc}(m_{\pi(i)}))$$

여기서 π는 $\{1, \ldots, k\}$의 순열, $\text{pc}(\cdot)$는 MIDI pitch를 pitch class $(\bmod 12)$ 로 변환하는 함수, d_T는 Tonnetz 격자 거리 (§2.4) 이다. 이 최적 순열은 **Hungarian algorithm** (Kuhn, 1955)으로 $O(k^3)$에 정확히 계산된다. 두 cycle의 크기가 다를 때는 작은 쪽에 dummy vertex (거리 $= \infty$)를 추가하여 정방 비용행렬을 만든다.

음악적 의미: 이 거리는 "두 cycle을 구성하는 note들을 최선의 1:1 대응으로 짝지었을 때의 평균 Tonnetz 거리"를 측정한다. 예컨대 C major triad $\{C, E, G\}$와 F major triad $\{F, A, C\}$는 $d_{\text{match}} = (d_T(C, F) + d_T(E, A) + d_T(G, C))/3$으로 비교된다.

정의 2.11 (Up to permutation 거리행렬 비교). 원곡의 note 집합 $\mathcal{N} = \{n_1, \ldots, n_N\}$와 새 note 집합 $\mathcal{N}' = \{n'_1, \ldots, n'_N\}$에 대해, 각각의 Tonnetz 거리행렬을 $D, D' \in \mathbb{R}^{N \times N}$이라 하자. **Up to permutation 거리행렬 오차**는 다음과 같다:

$$\mathrm{err}(D, D') = \min_{\pi \in S_N} \frac{1}{N^2} \sum_{i,j} |D[i,j] - D'[\pi(i), \pi(j)]|$$

전체 순열 공간의 탐색이 $O(N!)$로 비현실적이므로, 본 연구에서는 랜덤 후보 1000개를 생성한 뒤 비용 $\alpha \cdot \mathrm{note_err} + (1-\alpha) \cdot \mathrm{cycle_err}$ (기본 $\alpha = 0.5$) 가 최소인 후보를 선택하는 근사법을 사용한다.

정의 2.12 (Interval Class Vector, ICV). pitch class 집합 $S \subseteq \mathbb{Z}/12\mathbb{Z}$의 **interval class vector** $\mathrm{ICV}(S) \in \mathbb{Z}^6_{\geq 0}$는 다음과 같이 정의된다:

$$\mathrm{ICV}(S)[k] = |\{(p,q) \in S^2 : p < q,\ \min(|p-q|,\ 12 - |p-q|) = k\}|, \quad k = 1, \ldots, 6$$

여기서 $\min(|p-q|, 12-|p-q|)$는 두 pitch class 사이의 **interval class** — 옥타브 대칭을 고려한 최소 반음 거리 — 이다. 예를 들어 C major scale의 ICV는 $[2, 5, 4, 3, 6, 1]$이다.

정의 2.13 (Consonance score). 시점 t에서 동시에 발음되는 pitch class 집합 S_t의 **dissonance**는 다음과 같이 정의된다:

$$\mathrm{diss}(S_t) = \frac{1}{|S_t|(|S_t|-1)/2} \sum_{\{p,q\} \subset S_t} \mathrm{roughness}(p,q)$$

여기서 $\mathrm{roughness}(p,q) \in [0,1]$은 interval class에 따른 거칠기 가중치이다. 완전 8도 $= 0$, 장/단 2도 $= 1$, 완전 5도 $= 0.1$ 등의 음악이론적 서열을 사용한다. 곡 전체의 **평균 dissonance**는 $\bar{d} = T^{-1} \sum_t \mathrm{diss}(S_t)$이다.

정의 2.14 (Markov chain 시간 재배치). 원본 overlap matrix $O \in \{0,1\}^{T \times C}$의 행 시퀀스로부터 1차 Markov chain의 전이행렬을 추정한다. 시점 t에서의 상태 s_t는 $O[t, :] \in \{0,1\}^C$ (길이-C 이진 벡터) 이며:

$$P(s_{t+1} = j \mid s_t = i) = \frac{\#\{t : s_t = i,\ s_{t+1} = j\}}{\#\{t : s_t = i\}}$$

이 전이확률을 사용하여 초기 상태 s_0부터 새로운 상태 시퀀스를 **재샘플링**한다. 온도 파라미터 $\tau > 0$로 전이확률을 조절한다: $P_\tau(j|i) \propto P(j|i)^{1/\tau}$. $\tau = 1$이면 원본과 동일한 전이 통계, $\tau > 1$이면 더 무작위적, $\tau < 1$이면 더 결정적이다.

---

3. 두 가지 음악 생성 알고리즘

본 장에서는 본 연구의 두 가지 음악 생성 알고리즘 — Algorithm 1 (확률적 샘플링) 과 Algorithm 2 (신경망 기반 시퀀스 모델) — 의 핵심 아이디어와 설계 의도를 설명한다.

표기 정의

본 장에서 사용할 표기를 다음과 같이 통일한다.

기호	의미	hibari 값
T	시간축 길이 (8분음표 단위)	1,088
N	고유 note 수 (pitch-duration 쌍)	23
C	발견된 전체 cycle 수	최대 48

K	선택된 cycle subset 크기 ($K \le C$)	$\{10, 17, 48\}$
O	중첩행렬, $\{0,1\}$ 값의 $T \times K$ 행렬	—
L_t	시점 t에서 추출할 note 개수	보통 3 또는 4
$V(c)$	cycle c의 vertex(note label) 집합	원소 수 $4 \sim 6$
R	재샘플링 최대 시도 횟수	50
B	학습 미니배치 크기	32
E	학습 epoch 수	200
H	DL 모델의 hidden dimension	128

L_t에 대한 보충. L_t는 "시점 t에서 새로 발성할 note의 개수"이다. hibari의 경우 악기 1, 2의 chord height(한 시점의 동시 발음 수)를 따라 대체로 3 또는 4로 설정되며, 구체적으로 `[4, 4, 4, 3, 4, 3, 4, 3, 4, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 3, 3, 3, 3]`의 32개 패턴을 33번 반복한 길이 $1{,}056$의 수열을 사용한다 (총합 약 $3{,}700$). 이 패턴은 원곡의 평균 density에 맞춰 경험적으로 결정된 것이다.

B, E, H에 대한 보충. 본 연구의 hidden dimension $H = 128$, epoch 수 $E = 200$, batch size $B = 32$는 **엄밀한 grid search로 튜닝된 값이 아니라**, 소규모 시퀀스 데이터에 대한 일반적 관례(LSTM 논문, Transformer 구현 예제)에서 차용한 출발점 값이다. 모든 실험을 동일한 하이퍼파라미터에서 수행함으로써 세 모델(FC / LSTM / Transformer)의 구조적 차이를 공정하게 비교하는 것을 우선했다. 향후 연구에서 이 값들에 대한 체계적 튜닝 여지가 남아있다.

3.1 Algorithm 1 — 확률적 샘플링 기반 음악 생성

알고리즘 개요

Algorithm 1은 중첩행렬의 ON/OFF 패턴을 직접 참조하여, 각 시점에서 활성화된 cycle들이 공통으로 포함하는 note pool로부터 확률적으로 음을 추출하는 규칙 기반 알고리즘이다. 신경망 학습 없이 즉시 생성이 가능하며, 중첩행렬이 곧 "구조적 seed" 역할을 한다.

핵심 아이디어 (3가지 규칙)

규칙 1 — 시점 t에서 활성 cycle이 있는 경우, 즉

$$\sum_{c=1}^{K} O[t, c] > 0$$

일 때, 활성화되어 있는 모든 cycle들의 vertex 집합의 교집합

$$I(t) = \bigcap_{c \,:\, O[t,c]=1} V(c)$$

에서 note 하나를 **균등 추출**한다. 여기서 "균등 추출"이란 집합 $I(t)$의 모든 원소가 동일한 확률 $1/|I(t)|$로 선택된다는 의미이다 (이산균등분포). 만약 교집합이 공집합이면 ($I(t) = \emptyset$), 전체 note pool P에서 균등 추출한다. 이 규칙은 "여러 cycle이 동시에 살아 있을 때, 그 cycle들이 모두 공유하는 note는 음악적으로도 가장 핵심적인 음"이라는 가정을 반영한다.

규칙 2 — 시점 t에서 활성 cycle이 없는 경우, 즉

$$\sum_{c=1}^{K} O[t, c] = 0$$

일 때, 인접 시점 $t-1, t+1$에서 활성화된 cycle들의 vertex의 합집합

$$A(t) = \bigcup_{c\,:\,O[t-1,c]=1} V(c) \cup \bigcup_{c\,:\,O[t+1,c]=1} V(c)$$

을 계산한 뒤, 전체 note pool에서 이 합집합을 제외한 영역 $P \setminus A(t)$에서 균등 추출한다.

이 규칙은 다음과 같은 의미를 가진다. 활성 cycle이 없는 시점에서도 음악은 흘러가야 하므로 음을 하나 골라야 하는데, 만약 인접 시점의 cycle 멤버 노트를 그대로 골라 버리면, 청자가 들었을 때 마치 그 cycle이 시점 t에도 살아 있는 것처럼 들리게 된다. 즉, 원래 분석상으로는 죽어 있어야 할 위상 구조가 인위적으로 살아있는 것처럼 "번지는" 현상이 생긴다. 이를 막기 위해 인접 cycle의 vertex를 의도적으로 회피하여, "활성 cycle 없음"이라는 정보가 청각적으로도 그대로 보존되도록 한다.

규칙 3 — 중복 onset 방지. 같은 시점 t에서 동일한 (pitch, duration) 쌍이 두 번 추출되지 않도록 `onset_checker`로 검사하며, 충돌이 발생하면 최대 R회까지 재샘플링한다. R회 모두 실패하면 그 시점의 해당 note 자리는 비워둔다.

출력

알고리즘은 (start, pitch, end) 형태의 음표 리스트 G를 출력하며, 이를 MusicXML로 직렬화하면 곧바로 악보 및 오디오로 재생할 수 있다.

3.2 Algorithm 2 — 신경망 기반 시퀀스 음악 생성

알고리즘 개요

Algorithm 2는 중첩행렬을 입력, 원곡의 multi-hot note 행렬을 정답 레이블로 두고 매핑

$$f_\theta : \{0,1\}^{T\times C} \longrightarrow \mathbb{R}^{T\times N}$$

을 학습한다 (FC 모델은 시점별 독립이므로 $\{0,1\}^C \to \mathbb{R}^N$). 학습된 모델은 학습 시 보지 못한 cycle subset이나 노이즈가 섞인 중첩행렬에 대해서도 원곡과 닮은 note 시퀀스를 출력하도록 기대된다.

"위상 구조 보존"의 의미. 엄밀히 말하면 DL 모델은 Algorithm 1처럼 "교집합 규칙"으로 위상 구조를 직접 강제하지는 않는다. 대신 Subset Augmentation(아래 설명)을 통해 $K \in \{10, 15, 20, 30, 46\}$과 같은 다양한 크기의 subset에 대해서도 같은 원곡 y를 복원하도록 학습한다. 이 과정에서 모델은 "서로 다른 cycle subset이 같은 음악을 유도할 때, 그 공통적인 구조적 특성"을 잠재 표현으로 내부화한다. 따라서 학습 시 구체적으로 보지 못한 subset(예: $K=12$)에 대해서도, 모델이 학습한 잠재 표현이 충분히 일반화되어 있다면 합리적 출력이 가능하다. 본 연구의 실험에서는 이러한 일반화가 실제로 관측되었으나, 이는 "위상 구조 자체의 수학적 보존"이라기보다 "학습된 매핑의 통계적 일반화"에 가깝다.

모델 아키텍처 비교

본 연구는 동일한 학습 파이프라인 위에서 세 가지 모델 아키텍처를 비교한다.

모델	입력 형태	시간 정보 처리 방식	파라미터 수 (대략)
FC	(B, C)	시점 독립	4×10^4
LSTM (2-layer)	(B, T, C)	순방향 hidden state	2×10^5
Transformer (2-layer, 4-head)	(B, T, C)	self-attention	4×10^5

"시점 독립"의 의미 (FC). FC 모델은 시점 t의 위상 벡터 $O[t,:]$를 입력으로 받아 시점 t의 note 벡터 $y[t,:]$를 출력한다. 즉 시점 t의 출력은 이전 시점 $t-1, t-2, \ldots$나 이후 시점 $t+1, \ldots$을 전혀 참조하지 않는다. 각 시점을 "독립적으로" 처리한다는 뜻이다. 이는 가장 단순한 기준 모델이다.

"순방향 hidden state"의 의미 (LSTM). LSTM은 시점 t의 출력을 만들 때, 내부에 유지하는 hidden state h_{t-1}을 참조한다. h_{t-1}은 시점 $1, 2, \ldots, t-1$까지의 정보가 누적된 벡터이다. 따라서 "왼쪽에서 오른쪽으로 흐르는 시간 정보"를 사용한다. 미래 시점은 보지 못한다.

"self-attention"의 의미 (Transformer). Self-attention은 시점 t의 출력을 만들 때, 시퀀스의 **모든** 시점 $1, \ldots, T$에 대한 "주목도 점수"를 계산하여, 각 시점의 벡터를 가중합한다. LSTM과 달리 미래 시점도 함께 본다(bidirectional). 따라서 "시점 t의 note를 결정할 때 곡 전체의 문맥을 고려한다"는 해석이 가능하다.

학습 데이터 구성과 증강

원본 학습 쌍은 $X \in \{0,1\}^{T \times C}$, $y \in \{0,1\}^{T \times N}$이다. 여기서 X는 중첩행렬이고 y는 같은 시간축의 multi-hot note 행렬(시점 t에 활성인 note를 1로 표시)이다. 본 연구는 세 가지 증강 전략을 적용하여 학습 데이터를 약 $7 \sim 10$배 늘린다.

(1) Subset Augmentation. $K \in \{10, 15, 20, 30\}$의 cycle subset에 대한 중첩행렬을 생성하여, 동일한 정답 y에 매핑한다. 이를 통해 모델은 "불완전한 위상 정보로부터도 원곡을 복원하는" 강건한 표현을 학습한다.

(2) Circular Shift. 시간축을 회전하는 증강이며, X와 y를 **동일한 양만큼** 함께 회전한다. 즉

$$X' = \mathrm{roll}(X, s, \mathrm{axis} = 0), \qquad y' = \mathrm{roll}(y, s, \mathrm{axis} = 0)$$

로 처리한다 (여기서 s는 같은 난수). 여기서 `roll` 함수는 `numpy`에서 제공하는 함수이며, 배열을 지정한 축으로 원형 이동(끝에서 밀려난 원소가 앞으로 돌아옴)시키는 역할을 한다. 이 증강은 시점 t의 위상 정보와 시점 t의 note를 시점 $t+s$로 똑같이 옮기므로, 모델이 학습해야 할 매핑 자체는 변하지 않은 채 시작점만 달라진다. 만약 X에만 회전을 적용하면 X와 y의 시간축이 어긋나 학습 데이터가 망가지므로, 두 행렬을 항상 함께 회전해야 한다.

(3) Noise Injection. X에 확률 $p = 0.03$으로 bit flip을 가한다 (y는 그대로). overfitting을 막고 정규화 효과를 얻기 위함이다.

학습 손실 함수

각 시점에서 여러 note가 동시에 활성화될 수 있으므로(multi-label 문제), 2.8절에서 정의한 binary cross-entropy 손실을 사용한다. PyTorch의 `BCEWithLogitsLoss`는 sigmoid를 손실 안에 포함시킨 수치 안정 버전이며, 양성/음성 클래스의 불균형을 보정하기 위해 `pos_weight` 파라미터에 (음성 샘플 수 / 양성 샘플 수) 비율을 넣는다.

추론 단계

학습이 끝난 모델 f_{θ^*}로 새로운 음악을 생성하는 단계를 하나하나 풀어 설명한다.

1단계 — 모델 통과 (logit 생성). 입력 중첩행렬 O_{test}를 모델에 통과시키면 $\hat{z} \in \mathbb{R}^{T \times N}$이 나온다. 이 $\hat{z}$의 각 값은 음수, 0, 양수 모두 가능한 실수이며, 직접 "확률"이라고 해석할 수 없다. 이런 "확률이 되기 전의 raw 점수"를 통계학과 머신러닝에서 **logit**이라고 부른다. 크기가 클수록 "그 note가 활성일 가능성이 높다"는 모델의 내부 점수이다.

2단계 — sigmoid 변환. logit을 0~1 사이의 확률로 바꾸기 위해 sigmoid 함수

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

를 적용한다. sigmoid는 $z = 0$에서 0.5, $z \to \infty$에서 1, $z \to -\infty$에서 0에 수렴하는 S자 곡선으로, 실수 전체를 $[0, 1]$ 구간으로 눌러 담는다. 적용 후 $P = \sigma(\hat{z}) \in [0,1]^{T \times N}$은 "시점 t에 note n이 활성일 확률"로 해석할 수 있다. 이 단계가 필요한 이유는, 다음 단계에서 "특정 확률 이상인 note를 켠다"는 판단을 내려야 하는데, 그 판단은 반드시 $[0, 1]$ 스케일에서 이루어져야 하기 때문이다.

3단계 — adaptive threshold 결정. 가장 단순한 방법은 "$P[t, n] \geq 0.5$이면 켠다"라고 고정 임계값을 쓰는 것이다. 그러나 LSTM이나 Transformer 같은 시퀀스 모델은 학습 결과 sigmoid 출력이 전반적으로 낮게 형성되는 경향이 있어, 0.5를 그대로 쓰면 활성화되는 note가 거의 없어 음악이 텅 비어버린다. 이를 해결하기 위해 본 연구는 원곡의 **ON ratio**(아래에서 정의)에 맞춰 threshold를 데이터 기반으로 동적 결정한다.

여기서 ON ratio란 "원곡의 multi-hot 행렬 $y \in \{0,1\}^{T \times N}$에서 전체 $T \times N$개의 셀 중 값이 1인 셀의 비율"을 뜻한다. 수식으로는

$$\rho_{\text{on}} = \frac{1}{T \cdot N} \sum_{t=1}^{T} \sum_{n=1}^{N} y[t, n]$$

이다. hibari의 경우 $T = 1{,}088$, $N = 23$이므로 전체 셀 수는 약 $25{,}024$개이고, 그 중 note가 활성인 셀 수를 세어 나누면 약 15% ($\rho_{\text{on}} \approx 0.15$)가 된다. 직관적으로는 "한 시점당 23개 note 중 평균 $3 \sim 4$개가 켜져 있는 정도"라고 이해할 수 있다.

이 ρ_{on}을 목표 활성 비율로 삼아, threshold를 다음과 같이 정한다:

$$\theta = \text{quantile}(P,\ 1 - \rho_{\text{on}})$$

즉 P의 모든 값 중 상위 15%에 해당하는 경계값을 임계값으로 쓴다. 이렇게 하면 모델 출력의 절대 수준이 어떻든, 생성된 곡의 활성 note 비율이 자연스럽게 원곡의 ρ_{on}과 일치한다. 이것을 "adaptive threshold"라 부르는 이유는 모델과 입력에 따라 θ 값이 자동으로 달라지기 때문이다.

4단계 — note 활성화 판정. 모든 (t, n) 쌍에 대해 $P[t, n] \geq \theta$이면 시점 t에 note n을 활성화한다. 이 note의 (pitch, duration) 정보를 label 매핑에서 복원하여 $(t,\ \text{pitch},\ t + \text{duration})$ 튜플을 결과 리스트 G에 추가한다.

5단계 — onset gap 후처리. 너무 짧은 간격으로 onset이 연속되면 음악이 지저분해지므로, "이전 onset으로부터 `gap_min` 시점 안에는 새 onset을 허용하지 않는다"는 최소 간격 제약을 적용한다. `gap_min = 0`이면 제약 없음, `gap_min = 3`이면 "3개의 8분음표(= 1.5박) 안에는 새로 타건하지 않음"을 의미한다.

이 과정으로 최종적으로 얻은 $G = [(start, pitch, end), \ldots]$를 MusicXML로 직렬화하면 재생 가능한 음악이 된다.

3.3 두 알고리즘의 비교 요약

항목	Algorithm 1 (Sampling)	Algorithm 2 (DL)
학습 필요 여부	불필요	필요 (E epoch)
결정성	확률적 (난수)	학습 후 결정적
일반화	같은 곡 내부에서만	보지 못한 cycle subset도 생성
위상 보존 방식	교집합 규칙으로 직접 강제	손실함수를 통해 간접
생성 시간	약 50 ms	약 100 ms (학습 후)
학습 시간	해당 없음 (학습 단계 자체가 없음)	30 s ~ 3 min

해석. Algorithm 1은 위상 정보를 직접 규칙으로 강제하므로 cycle 보존도 측면에서 가장 신뢰할 수 있는 기준선 역할을 한다. 반면 Algorithm 2는 학습된 잠재 표현을 통해 부드러운 생성이 가능하며, 학습 데이터에 없는 cycle subset에 대해서도 합리적인 음악을 만들어낸다. 본 연구의 실험에서는 두 알고리즘이 상호 보완적임을 보였다 — Algorithm 1은 위상 보존도에서, Algorithm 2는 음악적 자연스러움에서 각각 우위를 보인다 (Step 4 실험 결과 참조).

Algorithm 1이 "학습 시간" 칸에서 비어 있는 것이 아니라 "해당 없음"으로 표시된 이유는, 이 알고리즘이 애초에 학습 단계를 가지지 않기 때문이다. 주어진 중첩행렬과 cycle 집합만 있으면 어떠한 전처리 학습 없이도 그 자리에서 음악을 생성할 수 있으며, 이것이 Algorithm 1의 가장 큰 장점 중 하나이다.

---

4. 실험 설계와 결과

본 장에서는 지금까지 제안한 TDA 기반 음악 생성 파이프라인의 성능을 정량적으로 평가한다. 세 가지 유형의 실험을 수행하였다.

1. **Distance function 비교** — frequency(기본), Tonnetz, voice-leading, DFT 네 종류의 거리 함수에 대해 동일 파이프라인을 적용하고 생성 품질을 비교.
2. **Cycle subset ablation** — 최적 거리 함수(Tonnetz)에서 cycle 수를 $K = 10, 17, 46$으로 변화시켜 cycle 수의 효과를 분리.
3. **통계적 유의성** — 각 설정에서 Algorithm 1을 $N = 20$회 독립 반복 실행하여 mean ± std를 보고.

모든 실험은 동일한 chord height 패턴 (32-element module × 33 = 1,056 timepoints), 동일 random seed 체계($s = c + i,\ i = 0, \ldots, 19$, 설정별 상수 c 사용)로 수행되었다. 실험 러너는 `tda_pipeline/run_step3_experiments.py`이며, 모든 trial의 상세 기록(mean, std, min, max 포함)은 `tda_pipeline/docs/step3_data/step3_results.json`에 저장되어 있다.

평가 지표

Jensen-Shannon Divergence (주 지표). 생성곡과 원곡의 pitch 빈도 분포 간 JS divergence를 주 지표로 사용한다 (2.6절 정의). 값이 낮을수록 두 곡의 음 사용 분포가 유사하며, 이론상 최댓값은 $\log 2 \approx 0.693$이다.

Note Coverage. 원곡에 존재하는 고유 (pitch, duration) 쌍 중, 생성곡에 한 번 이상 등장하는 쌍의 비율. 1.00이면 모든 note가 최소 한 번 이상 사용된 것이다.

보조 지표. Pitch count (생성곡의 고유 pitch 수), 생성 소요 시간 (초), KL divergence.

거리 함수 구현 참고

본 장의 네 가지 거리 함수는 모두 `tda_pipeline/musical_metrics.py` 파일 하나에 정의되어 있다. 사용 패턴은 다음과 같다.

함수	역할	입력	출력
`tonnetz_distance(pc1, pc2)`	두 pitch class 간 Tonnetz 격자 거리	`pc1, pc2` $\in \{0, \ldots, 11\}$	정수 (최단 경로 길이)
`tonnetz_note_distance(n1, n2)`	두 note 간 거리 (옥타브/duration 보정 포함)	`(pitch, duration)` 튜플 × 2	실수
`voice_leading_note_distance(n1, n2)`	두 note 간 semitone 차이 기반 거리	`(pitch, duration)` 튜플 × 2	실수
`dft_note_distance(n1, n2)`	pitch class DFT 계수 간 Euclidean 거리	`(pitch, duration)` 튜플 × 2	실수
`compute_note_distance_matrix(notes_label, metric)`	전체 note 집합의 거리 행렬 생성	`{(pitch,dur): label}` 딕셔너리, metric 이름	$N \times N$ numpy 행렬

__Tonnetz 거리의 핵심 구현 (`_build_tonnetz_distance_table`).__ 12개 pitch class를 정점으로 두고, 장3도(±4 semitone), 단3도(±3), 완전5도(±7) 세 종류의 이웃 관계로 이루어진 무방향 그래프에서 BFS(너비 우선 탐색)로 모든 쌍의 최단 경로를 미리 계산하여 12×12 정수 테이블로 캐싱한다. 이후 `tonnetz_distance(pc1, pc2)`는 이 테이블을 $O(1)$에 조회한다.

두 note 간 확장. Tonnetz는 원래 pitch class(mod 12)만 고려하므로 옥타브와 duration 정보가 손실된다. 본 연구는 이를 보완하기 위해

$$d(n_1, n_2) = \text{tonnetz}(p_1 \bmod 12,\ p_2 \bmod 12) + w_o \cdot |o_1 - o_2| + w_d \cdot \frac{|d_1 - d_2|}{\max(d_1, d_2)}$$

로 정의한다 ($w_o = 0.5$, $w_d = 0.3$; $o_i = \lfloor p_i / 12 \rfloor$). 이 식이 `tonnetz_note_distance` 함수의 본체이다. 파이프라인에서는 이 함수가 `compute_note_distance_matrix`를 통해 $N \times N$ 거리 행렬로 벡터화되어, 이후

refine 단계(2.2절)와 Vietoris-Rips 복합체 구축(2.1절)에 입력된다.

4.1 Experiment 1 — Distance Function Baseline 비교

네 종류의 거리 함수 각각으로 사전 계산한 중첩행렬을 로드하여 Algorithm 1을 실행한다. 각 거리 함수에서 발견되는 cycle의 수도 함께 보고한다.

거리 함수	발견 cycle 수	JS Divergence (mean ± std)	Note Coverage	생성 시간 (ms)
frequency (baseline)	43	0.0753 ± 0.0033	0.991	31.2
Tonnetz	46	0.0398 ± 0.0031	1.000	38.9
voice-leading	22	0.0891 ± 0.0048	1.000	22.2
DFT	20	0.0511 ± 0.0029	1.000	26.3

해석 1 — Tonnetz가 가장 우수. Tonnetz 거리 함수는 baseline(frequency)에 비해 JS divergence를 $0.0753 \to 0.0398$로 **약** 47% **감소**시켰다. 두 조건의 표준편차가 각각 0.0033, 0.0031로 매우 작으므로, 이 차이는 통계적으로 명확한 개선이다(자세한 분석은 3.3절).

해석 2 — 거리 함수가 위상 구조 자체를 바꾼다. 동일한 hibari 데이터에서 거리 함수만 교체했을 뿐인데 발견되는 cycle 수가 $20 \sim 46$으로 크게 달라졌다. 이는 "거리 함수의 선택이 곧 어떤 음악적 구조를 '동치'로 간주할 것인가를 정의한다"는 음악이론적 관점과 일치한다. Tonnetz는 완전 5도 / 장3도 / 단3도 관계의 pitch class 쌍을 가깝게 배치하므로, 이러한 관계를 공유하는 note들이 한 cycle에 더 자주 모이게 된다.

해석 3 — voice-leading은 cycle을 적게 발견한다. Voice-leading 거리(반음 수 기반)는 22개의 cycle만 찾아내며 JS divergence도 가장 나쁘다 (0.0891). 이는 voice-leading이 "인접 pitch 사이의 거리"를 너무 엄격하게 측정하여, hibari처럼 넓은 음역에 걸쳐 느슨하게 연결된 곡에서는 위상 구조가 적게 드러난다는 것을 시사한다.

해석 4 — Note Coverage는 대부분의 설정에서 포화. 네 거리 함수 모두 note coverage가 $0.99 \sim 1.00$이므로, "원곡의 모든 note 종류가 생성곡에 최소 한 번 등장"하는 기본 요구는 모두 만족된다. 따라서 품질의 주된 차이는 "같은 note pool을 얼마나 자연스러운 비율로 섞는가"에서 발생한다.

4.2 Experiment 2 — Cycle Subset Ablation

거리 함수를 Tonnetz로 고정하고, cycle 수 K를 변화시켜 "더 많은 cycle = 더 좋은 생성인가?"를 검증한다. $K=10$과 $K=17$은 전체 46개 중 처음 K개의 cycle(prefix subset)을 사용하였다.

설정	K	JS Divergence	KL Divergence	Note Coverage	생성 시간 (ms)
Tonnetz, $K=10$	10	0.0991 ± 0.0038	0.556 ± 0.035	0.980	24.4
Tonnetz, $K=17$	17	0.0740 ± 0.0038	0.550 ± 0.344	0.996	26.3
Tonnetz, $K=46$ (full)	46	0.0397 ± 0.0025	0.172 ± 0.013	1.000	40.8

해석 5 — Cycle이 많을수록 JS가 단조 감소. K가 $10 \to 17 \to 46$으로 늘어남에 따라 JS divergence는 $0.099 \to 0.074 \to 0.040$으로 단조 감소하였다. 이는 "위상 구조가 더 풍부하게 드러날수록 생성곡의 음 사용 분포가 원곡에 더 근접한다"는 본 연구의 핵심 가설을 뒷받침한다.

해석 6 — 한계 효용의 감소. K 증가에 따른 JS 감소폭을 보면:

- $K=10 \to 17$: 개선 $\Delta=0.025$
- $K=17 \to 46$: 개선 $\Delta=0.034$

cycle 수가 거의 세 배($17 \to 46$)가 된 것에 비해 개선 폭은 $K=10 \to 17$(7개 추가)과 크게 차이나지 않는다. 즉 뒤쪽 cycle들은 이미 어느 정도 포화된 구조를 재확인하는 수준의 기여를 한다. 이는 2.7절에서 논의한 greedy forward selection으로 "소수의 cycle로도 90% 보존"이 가능하다는 관찰과 일관된다.

해석 7 — KL 분산의 불안정성. $K=17$ 설정에서 KL divergence의 표준편차가 0.344로 유난히 크다. 이는 일부 trial에서 KL이 1.55까지 튀는 경우가 있었기 때문이며 (원본 JSON은 `docs/step3_data/step3_results.json`의 `experiment_2_ablations.subset_K17.kl_divergence.max` 필드에서 확인 가능), "$\log(P/Q)$가 $Q \to 0$에서 발산하는" KL의 구조적 불안정성에서 기인한다. JS divergence는 동일 trial들에서 $0.064 \sim 0.079$ 범위로 안정되어 있어, JS가 더 안정적인 평가 지표임을 재확인한다 (2.6절의 "대칭화와 유계성" 논의와 일관).

4.3 통계적 유의성 분석

두 baseline 비교 (frequency vs Tonnetz) 의 차이가 통계적으로 유의한지 확인한다. 두 표본의 평균을 비교하는 표준적인 방법은 Student t-test이지만, Student의 고전적 t-test는 "두 집단의 모분산이 같다"는 강한 가정을 필요로 한다. 본 실험에서 두 조건의 표본표준편차는 $s_1=0.0033$, $s_2=0.0031$으로 매우 비슷하지만 완전히 같지는 않으며, 모분산이 같다는 사전 근거도 없다. 따라서 등분산 가정을 요구하지 않는 **Welch's t-test**를 사용한다. Welch는 "모평균과 모분산을 모를 때 표본평균과 표본분산만으로 검정"이 가능하며, 자유도를 Welch-Satterthwaite 근사로 계산한다.

데이터.

- Frequency: $\bar{x}_1=0.0753$, $s_1=0.0033$, $n_1=20$
- Tonnetz: $\bar{x}_2=0.0398$, $s_2=0.0031$, $n_2=20$

Welch t 통계량 은 다음과 같이 정의된다:

$$t=\frac{\bar{x}_1-\bar{x}_2}{\sqrt{\frac{s_1^2}{n_1}+\frac{s_2^2}{n_2}}}$$

수치를 대입하면:

$$t=\frac{0.0753-0.0398}{\sqrt{\frac{0.0033^2}{20}+\frac{0.0031^2}{20}}}=\frac{0.0355}{\sqrt{1.025\times10^{-6}}}\approx 35.1$$

Welch-Satterthwaite 자유도. Welch 검정에서 t 통계량은 정확한 Student 분포를 따르지 않고, 다음 식으로 근사된 자유도 ν의 t-분포를 따른다:

$$\nu\approx\frac{(\frac{s_1^2}{n_1}+\frac{s_2^2}{n_2})^2}{\frac{(s_1^2/n_1)^2}{n_1-1}+\frac{(s_2^2/n_2)^2}{n_2-1}}$$

$n_1=n_2=20$이고 두 표본분산이 거의 같으므로 ν는 $\approx n_1+n_2-2=38$에 매우 가깝게 나온다. 수치 계산 결과 $\nu\approx 37.9$이며, 반올림하여 $\nu=38$로 사용한다.

자유도 $\nu=38$에서 양측 임계값은 $t_{0.001,\,38}\approx 3.56$이므로, $|t|=35.1\gg 3.56$이며 $p<10^{-20}$이다. 따라서 **Tonnetz가 frequency보다 JS divergence를 낮춘 것은 극도로 통계적으로 유의**하다.

효과 크기 (Cohen's d). p-값만으로는 "차이가 실질적으로 얼마나 큰가"를 알 수 없으므로, 표본평균 차를 표본표준편차로 정규화한 Cohen's d를 함께 보고한다:

$$d = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{(s_1^2 + s_2^2)/2}}$$

$$d = \frac{0.0355}{\sqrt{(0.0033^2 + 0.0031^2)/2}} \approx 11.1$$

Cohen의 관례상 $d > 0.8$이 "큰 효과"인데 $d \approx 11$은 비교할 수 없는 초대형 효과이다. 두 분포가 실질적으로 분리되어 있음을 의미한다.

4.3a Experiment 2.5 — Continuous Overlap Matrix 실험

본 절은 2.5절에서 정의한 **연속값 중첩행렬** $O_{\text{cont}} \in [0,1]^{T \times K}$가 이진 중첩행렬 $O \in \{0,1\}^{T \times K}$ 대비 어떤 영향을 주는지를 정량적으로 검증한다. 거리 함수는 모든 설정에서 Tonnetz로 고정한다 (3.1절 결과상 가장 강한 baseline).

실험 설계

cycle별 시점 활성도 $a_{c,t}$는 두 가지 방식으로 계산할 수 있다.

이진 (binary): 단순 OR 연산이다. $V(c)$에 속하는 note가 시점 t에 하나라도 활성이면 $a_{c,t} = 1$, 그렇지 않으면 0이다.

연속값 (continuous): cycle을 구성하는 note 중 얼마나 많은 비율이 활성화되어 있는지를 $[0,1]$ 실수로 표현한다. 분수 형태가 아니라 단일 라인으로 쓰면:

$$a_{c,t} = \Big(\sum_{n \in V(c)} w(n) \cdot \mathbb{1}[n \in A_t] \Big) \Big/ \Big(\sum_{n \in V(c)} w(n) \Big)$$

여기서 A_t는 시점 t에 활성인 note들의 집합, $w(n) = 1/N_{\text{cyc}}(n)$은 note n의 **희귀도 가중치**이며 $N_{\text{cyc}}(n)$은 note n이 등장하는 cycle의 개수이다. 적은 cycle에만 등장하는 희귀 note일수록 가중치 $w(n)$이 커져, 그 note가 활성화되면 $a_{c,t}$에 더 큰 기여를 한다.

연속값 활성도가 만들어진 후, 최종 overlap matrix를 만드는 방식에 따라 다시 두 가지 변형이 가능하다.

- **직접 사용 (direct)**: $O[t,c] = a_{c,t} \in [0,1]$
- **임계값 이진화 (threshold τ)**: $O[t,c] = \mathbb{1}[a_{c,t} \geq \tau]$, $\tau \in \{0.3, 0.5, 0.7\}$

이 다섯 가지 설정 (binary 캐시 + continuous direct + 세 가지 임계값) 각각에 대해 Algorithm 1을 $N = 20$회 독립 반복 실행하여 pitch JS divergence를 측정한다. 실험 러너는 `tda_pipeline/run_step3_continuous.py`이며 원본 결과는 `docs/step3_data/step3_continuous_results.json`에 저장되어 있다.

결과

설정	Density	JS Divergence (mean ± std)
(A) Binary (기존 캐시)	0.751	0.0387 ± 0.0027
(B) Continuous direct	0.264	0.0382 ± 0.0021
(C) Continuous → bin $\tau = 0.3$	0.373	0.0386 ± 0.0022
(C) Continuous → bin $\tau = 0.5$	0.201	0.0343 ± 0.0027
(C) Continuous → bin $\tau = 0.7$	0.077	0.0364 ± 0.0032

여기서 "Density"는 overlap matrix에서 활성으로 표시되는 셀의 평균 비율 ($\bar{O}$). Binary 캐시는 0.751로 매우 dense한 반면, continuous direct는 0.264로 훨씬 sparse하다 (희귀도 가중치가 평균값을 낮춘다).

해석

해석 7a — Continuous direct는 binary와 거의 동등. 단순히 활성도를 그대로 사용한 (B) 설정은 (A) 대비 평균 JS가 약간 낮고 (0.0382 vs 0.0387), 표준편차도 약간 작다 (0.0021 vs 0.0027). 차이는 통계적으로 유의하지 않으며, "연속값을 직접 쓰는 것 자체"는 큰 이득이 없다.

해석 7b — $\tau = 0.5$ 임계값이 최우수. 흥미롭게도 연속값을 만들어 놓고 다시 $\tau = 0.5$로 이진화한 설정이 평균 JS 0.0343로 가장 좋다. 이는 (A) baseline 대비 약 11.4% 개선이며, τ 변화에 대한 모양이 $0.3 \to 0.5$ 구간에서 단조 감소, $0.5 \to 0.7$에서 다시 증가하는 명확한 U자 패턴을 보인다.

이 결과의 음악적 해석은 다음과 같다. 기존 binary overlap (A)는 "한 cycle의 vertex 중 하나라도 활성이면 cycle 전체가 활성"으로 판정하므로 cycle 활성도가 과대평가되며 ($\bar{O} = 0.751$), Algorithm 1의 교집합 sampling이 너무 자주 호출되어 다양성이 떨어진다. 반대로 $\tau = 0.7$은 너무 까다로워서 cycle 활성 시점이 너무 드물어진다 ($\bar{O} = 0.077$). 그 사이의 $\tau = 0.5$는 "cycle vertex의 절반 이상이 활성일 때만 그 cycle을 살아 있는 것으로 간주"한다는 음악적으로도 자연스러운 기준이며, 이 sweet spot이 최우수 결과를 만든다.

해석 7c — Welch t-test (binary vs $\tau = 0.5$). 두 설정의 차이가 통계적으로 유의한지 검정한다.

- (A) Binary: $\bar{x}_A = 0.0387$, $s_A = 0.0027$, $n_A = 20$
- (C@τ=0.5): $\bar{x}_C = 0.0343$, $s_C = 0.0027$, $n_C = 20$

$$t = \frac{0.0387 - 0.0343}{\sqrt{0.0027^2/20 + 0.0027^2/20}} \approx 5.16$$

자유도 $\nu \approx 38$에서 $t_{0.001,38} \approx 3.56$이므로 $|t| = 5.16 > 3.56$, 즉 $p < 0.001$이다. Cohen의 효과 크기 $d \approx 1.63$ ($d > 0.8$ "큰 효과" 범주). 따라서 **continuous → $\tau = 0.5$ 이진화의 개선은 통계적으로 유의**하다.

이 결과는 본 연구의 향후 방향에 함의를 준다. 본 실험에서는 연속값 overlap matrix의 잠재력을 최소한만 사용했음에도 (단일 임계값 τ grid search 정도) 이미 약 11%의 추가 개선을 얻을 수 있었으며, 더 정교한 가중치 학습이나 soft activation을 받아들이는 Algorithm 2 변형을 통해 추가 향상이 가능할 것으로 기대된다.

4.4 Experiment 3 — DL 모델 비교 (참고)

왜 $N = 20$ 통계적 반복을 수행하지 않았는가

Algorithm 1은 한 번의 실행에 ~ 50 ms 만 걸리므로 $N = 20$ 반복이 수 초 안에 끝난다. 반면 Algorithm 2의 학습은 한 config 당 30 s ~ 3 min이 걸리므로, 동일한 수준의 반복 실험을 전체 grid에 대해 수행하려면 수 시간이 필요하다. 본 연구는 대신 다음 두 가지 선행 실험을 수행하였다.

1. **하이퍼파라미터 튜닝 (30 조합, 단일 run).** `run_tuning.py` 스크립트로 FC/LSTM/Transformer 각각에 대해 hidden dim $\in \{64, 128, 256\}$, learning rate $\in \{0.0005, 0.001, 0.003\}$, dropout $\in \{0.1, 0.3\}$, augmentation 배수 $\in \{5\times, 10\times\}$ 조합으로 grid search를 실행하였다. 이 튜닝에서 FC 모델이 top 10을 모두 차지했고, 최적 조합은 `FC / hidden=256 / lr=0.001 / dropout=0.3`으로 validation loss 0.282, pitch JS divergence 0.0015를 달성하였다. 튜닝 실험은 각 조합에 대해 1회만 수행되었으며, 동일 seed 하에서 결과가 결정적이므로 표준편차 정보는 없다.
2. **본 장에서 보고.** 아래 표는 튜닝에서 얻은 모델별 최우수 구성의 단일 run 결과를 그대로 가져온 것이다.

모델	Validation Loss	Pitch JS Divergence	생성 note 수
FC (hidden=256, lr=0.001, dropout=0.3)	0.282	0.0015	$3{,}754$
LSTM (2-layer, hidden=128)	0.385	0.0448	$3{,}753$
Transformer (2-layer, 4-head, $d_{\text{model}} = 128$)	0.676	0.0104	$3{,}753$

향후 $N=5$ 반복 재검증이 Step 4에서 계획되어 있으나, 본 장에 DL 결과를 먼저 수록한 이유는 두 가지이다. 첫째, 거리 함수 비교 실험(3.1절)이 "Tonnetz가 Algorithm 1에서 우월하다"는 결론을 냈는데, **같은 결론이 DL 기반 Algorithm 2에서도 유효한지**를 동일 장에서 함께 논의해야 연구 전체의 일관성이 드러나기 때문이다(아래 해석 9). 둘째, DL 튜닝의 상위권이 FC 모델로 완전히 쏠린 것은 통계적 반복 없이도 확인되는 정성적 관측이며, 그 현상 자체가 hibari라는 곡의 고유한 성격에서 기인한다는 해석(해석 8)을 제시하기 위해서이다. Step 4의 재검증은 효과 크기의 수치적 안정성을 보강하는 용도이지, 현재 결론 자체를 뒤집기 위한 것이 아니다.

해석

해석 8 — FC가 가장 낮은 JS, 그리고 "Out of Noise"라는 맥락. 가장 단순한 FC 모델이 가장 낮은 JS divergence를 달성한 것은 얼핏 이상해 보인다. 통상적으로 시퀀스 모델(LSTM / Transformer)이 시간 문맥을 활용해 더 정교한 출력을 낼 것으로 기대되기 때문이다. 그러나 **hibari는 2009년 사카모토 류이치의 앨범 out of noise에 수록된 곡**이며, 이 앨범 자체가 "소음과 음악의 경계를 탐구한다"는 기획 의도로 제작되었다. 앨범의 많은 곡은 전통적 선율 진행을 의도적으로 회피하고, 각 음이 시간적 인과(causality)보다는 공간적 배치에 가까운 방식으로 놓인다.

이 미학적 맥락에서 보면 FC 모델의 우수한 결과는 역설이 아니라 **곡의 성격에 대한 실증**에 가깝다. FC는 시점 t의 note를 결정할 때 이전 시점 $t-1, t-2, \ldots$를 전혀 참조하지 않는, 다시 말해 "시간 맥락 없이 각 음을 독립적으로 배치하는" 모델이다. 이것이 LSTM/Transformer보다 원곡에 더 가까운 결과를 내었다는 것은, hibari가 선율적 인과보다는 음들의 비맥락적 배치로 구성되어 있다는 원곡의 작곡적 특성을 모델 선택이 간접적으로 드러낸 셈이다. 즉 FC가 "의미 없이 친 것 같은" 분포가 실제로 원곡과 일치했던 것은, 원곡 자체가 그러한 방식으로 설계되었기 때문이라는 해석이 가능하다.

해석 9 — Tonnetz + FC 조합의 최고 성능. Tonnetz hybrid distance ($\alpha=0.3$) 로 사전 계산한 중첩행렬을 FC 모델에 입력했을 때, 선행 실험에서 pitch JS divergence ≈ 0.003을 기록하였다 (기준 frequency + FC의 0.053 대비 약 18**배 개선**). 이것이 본 연구 전체에서 관측된 최우수 결과이며, Experiment 1의 "Tonnetz가 우수한 구조적 사전"이라는 결론이 DL 기반 생성에서도 유효함을 시사한다.

3.4a Experiment 4 — 개선 F: Continuous overlap + Algorithm 2 (FC)

§7.1.5 에서 제안된 개선 F — continuous activation matrix 를 Algorithm 2 (DL) 의 입력으로 직접 사용 — 를 구현 및 검증한다. §3.3a 에서 Algorithm 1 에 대해 continuous 변형이 11% 개선을 주었음을 확인한 바 있는데, 본 절은 동일한 아이디어를 FC 모델에 적용했을 때 얼마나 더 큰 효과가 있는가 를 정량화한다. 실험 러너는 `tda_pipeline/run_improvement_F.py`, 결과는 `docs/step3_data/step_improvementF_results.json` 에 저장되어 있다.

실험 설계

FC 모델 (§3.4 최우수 아키텍처: 2-layer, hidden 128, dropout 0.3) 을 고정하고, **학습 입력 데이터만** 두 가지로 바꾸어 비교한다.

F-bin (baseline): 기존 §3.4 와 동일. `build_activation_matrix(continuous=False)` 로 얻은 이진 활성행렬 $X_{\text{bin}} \in \{0,1\}^{T\times K}$ 를 FC 의 입력으로 사용.

F-cont: 희귀도 가중치가 적용된 연속값 활성행렬 $X_{\text{cont}} \in [0,1]^{T\times K}$ 를 사용. 구체적으로 $X_{\text{cont}}[t,c] = a_{c,t}$ (§4.3a 에서 정의한 $a_{c,t}$, $w(n) = 1/N_{\text{cyc}}(n)$ 의 희귀도 가중치). 모든 다른 조건 — 모델 아키텍처, learning rate, epochs, batch size, train/valid split — 는 F-bin 과 완전히 동일.

핵심 관찰: FC 의 입력층은 `nn.Linear` 이므로 binary 와 continuous 입력 양쪽 모두를 자연스럽게 처리할 수 있다. 즉 본 실험은 모델 변경 없이 데이터 표현만 바꾸었을 때의 효과 를 측정하며, 이는 continuous 버전의 정보 이득을 isolated 하게 평가한다.

각 설정에 대해 서로 다른 torch seed ($8100, 8107, 8114, 8121, 8128$) 로 $N=5$ 회 학습 + 생성 + JS 측정.

결과

설정	JS (mean ± std)	best	validation loss (mean)	coverage
F-bin (기존 §3.4)	0.0014 ± 0.0010	0.0006	0.0702	0.948
F-cont ★	0.0006 ± 0.0005	0.0003	0.0312	0.991

핵심 발견 1: JS divergence 약 2.3배 감소. F-cont 의 평균 JS 0.0006 는 F-bin 의 0.0014 대비 약 57% 감소이다. §3.1 에서 distance function 교체 (frequency $\rightarrow$ Tonnetz) 가 47% 감소, §3.3a 에서 $\tau = 0.5$ 이진화가 추가 11% 감소를 주었다면, 본 개선 F 는 그 위에 또 한 단계 개선을 쌓아 $0.0014 \rightarrow 0.0006$ 을 달성한다.

핵심 발견 2: 분산도 2배 감소. F-bin 의 표준편차 0.0010 대비 F-cont 는 0.0005. 즉 seed 별 결과의 일관성도 개선되었다.

핵심 발견 3: Validation loss 도 2.3배 낮음 ($0.070 \rightarrow 0.031$). 이는 모델이 학습 자체를 더 잘 한다는 뜻으로, 연속값 표현이 이진화 대비 학습 신호가 더 풍부하고 gradient landscape 이 더 부드러움을 시사한다.

핵심 발견 4: Note coverage 가 $0.948 \rightarrow 0.991$ 로 대폭 향상. F-bin 에서는 일부 rare note 가 생성되지 않는 경우가 있었지만, F-cont 에서는 거의 전체 23개 note 가 생성된다. 이는 희귀도 가중치 ($w(n) = 1/N_{\text{cyc}}(n)$) 가 의도한 효과 — 희귀 note 에 더 큰 학습 신호 — 가 실제로 작동함을 뒷받침한다.

왜 이렇게 큰 개선이 나오는가 — 음악적 해석

이진화는 "있다/없다" 만 말하지만, 연속값은 "얼마나 확신하는가" 를 말한다. 이진 $X_{\text{bin}}[t, c]$ 는 "시점 t 에 cycle c 의 vertex 중 하나라도 울리면 1"이다. 이것은 cycle c 의 활성 여부에 대해 이진 판정만 내린다. 반면 연속값 $X_{\text{cont}}[t, c] = a_{c,t}$ 는 "cycle c 의 vertex 중 희귀도 가중치 기준으로 몇 퍼센트가 활성인가" 를 $[0, 1]$ 실수로 표현한다. 즉 "이 시점에 이 cycle 이 어떤 종류의 note 조합으로 부분적으로 활성화되어 있는가" 라는 훨씬 풍부한 정보를 전달한다.

FC 모델 입장에서 이 차이는 결정적이다. 이진 입력에서 한 cycle 의 활성은 "on/off" 두 상태뿐이지만, 연속 입력에서는 "오직 common note 2 개만 활성 (낮은 값)" vs "rare note 포함 전체가 활성 (높은 값)" 같은 구별이 가능해진다. FC 가 이 구별을 학습하여 output note 분포를 조정할 수 있기 때문에, 원곡의 pitch 분포에 더 가까운 결과가 나온다.

기존 모든 결과와의 통합 비교

실험	설정	JS divergence	출처
§3.1 Algo 1	frequency baseline	0.0753	§3.1
§3.1 Algo 1	Tonnetz hybrid	0.0398	§3.1
§3.3a Algo 1	Tonnetz + continuous $\tau = 0.5$	0.0343	§3.3a
§7.1.6 Algo 1	P4 + C (module-level)	0.0590	§7.1.6
§7.1.8 Algo 1	P4 + C, best trial (seed 9302)	0.0258	§7.1.8
§3.4 Algo 2 FC	Tonnetz binary	0.0014	§3.4a (본 절)
§3.4a Algo 2 FC	**Tonnetz continuous**	0.0006 ± 0.0005	**본 절 ★**

§3.4a (개선 F) 는 본 연구의 full-song 생성에서 관측된 최저 JS divergence 이며, F-bin 대비 2.3배 우수하다. 이론적 최댓값 $\log 2 \approx 0.693$ 의 약 0.09% 에 해당한다.

한계 및 후속 과제

1. **학습 시 분산이 충분히 측정되었는가**: $N = 5$ 는 §3.1 의 $N = 20$ 에 비해 적다. FC 학습이 빠르므로 $N = 20$ 재확장이 비교적 쉽게 가능하다.
2. **LSTM / Transformer 에도 F-cont 적용**: 본 절은 FC 만 다루었다. §3.4 에서 관찰한 "FC > LSTM/Transformer" 패턴이 continuous 입력에서도 유지되는지 검증해야 한다.

3. **Continuous + module-local (P4)**: §7.1 의 P4 + C 와 개선 F 를 결합하여 "module-local continuous activation 을 FC 에 입력" 하는 실험도 의미가 있다.

4.5 종합 논의

(1) 음악이론적 거리 함수의 중요성. Experiment 1의 결과는 "빈도 기반 거리(frequency)는 기본 선택일 뿐, Tonnetz처럼 음악이론적 구조를 반영한 거리가 훨씬 더 좋은 위상적 표현을 만든다"는 본 연구의 가설을 강하게 지지한다. frequency → Tonnetz 전환만으로 JS divergence가 47% 감소했다.

(2) Cycle 수의 효과는 단조 증가이나 점진적. Experiment 2는 "cycle 수가 많을수록 좋다"는 기대를 확인하였으나, 한계 효용이 감소함을 보였다. 이는 K를 무작정 늘리기보다 greedy forward selection (2.7절) 으로 소수의 핵심 cycle을 고르는 전략이 실용적임을 시사한다.

(3) 통계적 엄밀성. 각 설정에서 $N=20$ 반복을 통해 표준편차가 ~ 0.003 수준임을 확인하였으며, 주요 baseline 비교의 Welch t 값이 >35으로 매우 유의했다. 이는 본 연구의 결론이 random seed에 의존하는 artifact가 아님을 보장한다.

(4) 곡의 맥락과 모델 선택. FC가 시퀀스 모델을 능가한 해석 8의 관찰은, 모델의 성능이 단순히 "표현력이 높을수록 좋다"는 법칙을 따르지 않고 **원곡의 미학적 설계와 공명하는 모델**이 가장 좋은 결과를 낸다는 것을 보여준다. 이는 본 연구가 다른 곡(예: 전통적 선율 진행이 뚜렷한 클래식 작품)으로 확장될 때 시퀀스 모델의 우위가 뒤바뀔 수 있음을 암시한다.

(5) 향후 과제.

1. DL 모델 comparison을 $N=5$ 이상 반복으로 재검증 (효과 크기 수치 안정화)
2. Ablation에서 prefix subset 대신 greedy selected subset으로 재실행하여 "어떤 cycle이 선택되는지"의 효과 분리
3. Tonnetz hybrid의 α 값 ($0.0, 0.3, 0.5, 0.7, 1.0$) 에 대한 grid search
4. 다른 곡(Ravel, Debussy, 또는 out of noise 앨범의 다른 곡) 으로의 일반화 검증 — 해석 8의 "곡의 미학이 모델 선택을 결정한다"는 가설 검증

3.6 곡 고유 구조 분석 — hibari 의 수학적 불변량

본 절은 hibari 가 가지는 수학적 고유 성질을 분석하고, 이 성질들이 본 연구의 실험 결과와 어떻게 연결되는지를 서술한다. 비교 대상으로 사카모토의 다른 곡인 solari 와 aqua 를 함께 분석한다.

3.6.1 Deep Scale Property — hibari 의 pitch class 집합이 갖는 대수적 고유성

hibari 가 사용하는 7개 pitch class 는 $\{0,2,4,5,7,9,11\} \subset \mathbb{Z}/12\mathbb{Z}$이다 (C major / A natural minor scale). 이 집합의 **interval vector** — 각 음정 클래스(1~6 반음)가 집합 안에서 몇 번 등장하는지를 센 벡터 — 는 $[2,5,4,3,6,1]$ 이다.

이 벡터의 6개 성분이 **모두 다른 수**이다 ($\{1,2,3,4,5,6\}$의 순열). 이것을 **deep scale property** 라 한다 (Gamer & Wilson, 2003). 이 성질을 갖는 7-note subset 은 $\binom{12}{7}=792$개 중 **diatonic scale 류 (장/단음계, 교회 선법) 뿐**이다. 즉 hibari 가 7개 PC 를 선택한 것은 임의가 아니라, 12음 체계에서 **각 음정 클래스가 고르게 (그러면서도 모두 다른 횟수로) 등장하는 유일한 부분집합**을 선택한 것이다.

또한 7개 PC 사이의 간격 패턴은 $[2,2,1,2,2,2,1]$로, 오직 $\{1,2\}$ 두 종류의 간격만으로 구성된다. 이것은 **maximal evenness** — 12개 칸 위에 7개 점을 가능한 한 균등하게 배치한 상태 — 를 의미한다 (Clough & Douthett, 1991). deep scale 과 maximal evenness 는 모두 diatonic scale 의 고유 성질이다.

solari 와 aqua 는 12개 PC 모두를 사용하므로 이 성질이 적용되지 않는다.

3.6.2 근균등 Pitch 분포 — Pitch Entropy

곡	사용 pitch 수	정규화 pitch entropy	해석
hibari	17	0.974	거의 완전 균등
solari	34	0.905	덜 균등
aqua	51	0.891	가장 치우침

pitch entropy 는 곡 안에서 사용된 모든 pitch 의 빈도 분포에 대한 Shannon entropy 를 계산하고, 이론적 최댓값 $\log_2(\text{uniquepitchcount})$ 로 나눠 정규화한 것이다. 1.0이면 모든 pitch 가 완전히 동일한 빈도로 등장하고, 0에 가까우면 한두 개 pitch 가 지배적이다.

hibari 의 0.974 는 **"모든 pitch 를 거의 같은 빈도로 사용"**한다는 뜻이며, 전통 조성 음악에서는 매우 드문 수치이다 (으뜸음이 지배적인 것이 보통). 이 성질은 **§3.4 의 "FC 모델 우위"를 수학적으로 설명**한다. pitch 분포가 거의 균등하면, 시간 순서를 무시하고 그 분포에서 독립적으로 뽑는 것 (FC 의 행동) 이 이미 원곡의 분포에 가깝다. 반면 solari 같이 특정 pitch 가 더 자주 나오는 곡에서는 시간 맥락 (Transformer) 이 그 편향을 학습해야 하므로 FC 가 불리하다.

3.6.3 Tonnetz 구별력과 Pitch Class 수의 관계

hibari 의 7개 PC 는 Tonnetz 위에서 **하나의 연결 성분**을 이루며, 평균 degree 가 $3.71/6 = 62\%$이다. Tonnetz 이웃 관계는 ± 3 (단3도), ± 4 (장3도), ± 7 (완전5도) 의 세 가지 방향이며, 각 방향이 양쪽으로 작용하므로 최대 6 개의 이웃이 가능하다.

예를 들어 C(0) 의 이웃을 계산하면:

관계	$+$ 방향	$-$ 방향	hibari 에 있는?
단3도 (± 3)	D#(3)	A(9)	A 만
장3도 (± 4)	E(4)	G#(8)	E 만
완전5도 (± 7)	G(7)	F(5)	둘 다

여기서 $0 - 7 = -7 = 5 \pmod{12} = F$ 이다. 즉 C 에서 완전5도 **아래로** 내려가면 F 에 도달한다 (동시에, 완전4도 위로 올라가는 것과 같다). 따라서 C 의 Tonnetz 이웃은 $\{E, F, G, A\}$의 **4개** 이다.

왜 이것이 중요한가 — Tonnetz 그래프의 지름(diameter).

12개 PC **전부**를 사용하는 곡 (solari, aqua) 에서는 **어떤 두 PC 든 Tonnetz 거리가** ≤ 2 이다. 이유: 임의의 PC 에서 한 발짝 ($\pm 3, \pm 4, \pm 7$) 으로 도달 가능한 PC 가 6개이고, 나머지 5개 ($12 - 1 - 6 = 5$) 는 모두 두 발짝 안에 도달 가능하다. 예를 들어 C 에서 한 발짝에 $\{3, 4, 5, 7, 8, 9\}$ 를 다 거치면, 거기서 한 발짝 더 가면 남은 $\{1, 2, 6, 10, 11\}$ 에 모두 도달한다 (예: $1 = 4 - 3,\ 2 = 5 - 3,\ 6 = 3 + 3,\ 10 = 7 + 3,\ 11 = 4 + 7$). $\mathbb{Z}/12\mathbb{Z}$ 의 대칭성으로 모든 PC 에서 동일. 따라서 12-PC Tonnetz 그래프의 지름은 2 이다.

지름이 2 라는 것은 **"가까운 음"과 "먼 음"을 구별할 여지가 거의 없다**는 뜻이다. 반면 hibari 의 7-PC 에서는 Tonnetz 거리가 $1 \sim 4$ 범위로 분포하여, 가까운 쌍 (예: C-G, 거리 1) 과 먼 쌍 (예: F-B, 거리 3 이상) 이 명확히 구별된다.

비유하면: 7명이 모여 사는 작은 마을에서 "누구와 누가 친한가"를 물으면 의미 있는 답이 나오지만, 12명이 모두 이웃인 세계에서는 "다 친하다"에 가까운 답이 나오는 것과 같다. 이것이 **"hibari 에서 Tonnetz 최적, solari 에서 voice-leading 최적"**(§3.1 vs §7.2)의 구조적 근거이다.

3.6.4 Phase Shifting — inst 1 과 inst 2 의 서로소 주기 구조

§5 Figure 7 에서 관찰한 "inst 1 은 쉼 없이 연속, inst 2 는 모듈마다 쉼이 있다"는 패턴을 더 정밀하게 분석하면, 놀라운 수론적 구조가 드러난다.

쉼의 위치가 모듈마다 정확히 1칸씩 이동한다. 원본 파이프라인 (solo 제거 후) 기준으로 inst 2 의 모듈별 쉼 위치를 조사하면:

```
module  1: rest at position  0
module  2: rest at position  1
```

```
module  3: rest at position  2
module  4: rest at position  3
  ...
module  k: rest at position  k-1
```

이것은 **대각선 패턴**이다. 32-timestep 모듈 안에서 쉼의 위치가 매 모듈마다 정확히 1칸씩 오른쪽으로 밀린다. 32개 모듈을 거치면 쉼이 모듈 내 모든 위치 $(0, 1, 2, \ldots, 31)$ 를 정확히 한 번씩 방문한다.

이 구조는 미니멀리즘 작곡가 Steve Reich 가 Piano Phase (1967) 에서 사용한 **phase shifting** 기법과 수학적으로 동일하다. 같은 패턴을 연주하는 두 악기 중 하나가 아주 조금 느리게 진행하여, 같은 패턴이 점점 어긋나다가 한참 뒤에야 원래 정렬로 돌아오는 것이다.

hibari 에서 이 phase shifting 은 다음과 같이 수치화된다.

	inst 1	inst 2
모듈 주기	$M = 32$ (쉼 없음)	$M + 1 = 33$ (32 음 + 1 쉼)
반복 횟수	33 회	32 회
총 길이	$33 \times 32 = 1{,}056$	$32 \times 33 = 1{,}056$

두 주기가 32 와 33 으로 **연속 정수 = 항상 서로소**$(\gcd(32, 33) = 1)$ 이다. 이 서로소 관계 때문에:

1. **두 악기가 같은 총 길이를 채움**: $33 \times 32 = 32 \times 33 = 1{,}056$ timestep.
2. **두 악기의 "위상(phase)" 이 곡 전체에서 한 번도 동기화되지 않음**: 매 모듈마다 쉼의 위치가 1칸씩 밀리므로, 32개 모듈을 거쳐야 비로소 원래 위치로 돌아온다.
3. **inst 2 의 시작점이 정확히** $33 = M + 1$: inst 2 가 $t = 33$ 에서 입장하는 것은, inst 2 의 주기 1개 분만큼 늦게 출발하는 것에 대응된다.

시계의 시침과 분침 비유가 직관적이다. 분침 (inst 1) 은 60분에 한 바퀴, 시침 (inst 2) 은 12시간에 한 바퀴를 돈다. 두 바늘은 같은 곳에서 출발해도 점점 어긋나다가 한참 뒤에야 다시 만난다. hibari 에서는 inst 1 이 32-step 주기로, inst 2 가 33-step 주기로 각각 "회전" 하며, $\gcd(32, 33) = 1$ 이므로 $32 \times 33 = 1{,}056$ step 안에서 한 번도 같은 위상으로 돌아오지 않는다.

이 구조는 수학적으로 **Euclidean rhythm** (Bjorklund, 2003; Toussaint, 2005) 과도 연결된다. Euclidean rhythm 은 "n 비트 중 k 개를 가능한 한 균등하게 배치" 하는 알고리즘으로, 아프리카 전통 음악과 전자 음악에서 널리 사용된다. hibari 의 경우 "33 칸 중 1 칸을 비운다" 를 32번 반복하면서 매번 1칸씩 이동하는 것이 Euclidean rhythm 의 가장 단순한 형태이다.

음악적 효과. 이 서로소 구조는 inst 2 의 쉼이 모듈 내 **모든 위치를 최대한 균등하게 방문**함을 보장한다. 쉼이 특정 위치에 몰리지 않으므로 "어떤 시점을 잘라 봐도 음악의 밀도가 일정" 하다. 이것은 §3.6.2 에서 관찰한 근균등 pitch entropy (0.974) 와 일관된 설계 원리이다 — **pitch 선택도 균등하고, 쉼 배치도 균등하다**. 사카모토가 인터뷰에서 밝힌 "한 악기를 시간 방향으로 충분히 채운 뒤 다른 악기를 그 위에 겹쳐 배치" 하는 작곡 방식이, 단순한 겹치기가 아니라 **수론적으로 최적인 위상 분리**를 달성하고 있음을 시사한다.

3.6.5 Cycle 교차 밀도 — 77%

Tonnetz 기반으로 발견된 46개 H_1 cycle 중, 쌍별 교집합을 계산하면 $\binom{46}{2} = 1{,}035$ 쌍 가운데 797 쌍 (77%) 이 적어도 하나의 vertex 를 공유한다. 즉 cycle 들이 독립적으로 흩어져 있는 것이 아니라 **"그물처럼 촘촘히 엮여 있다"**.

이 높은 교차 밀도는 두 가지 실험 결과와 직접 연결된다.

(a) Greedy selection 의 효과. 46개 cycle 이 서로 많은 vertex 를 공유하므로, 소수의 cycle 만 선택해도 전체 note pool 의 대부분을 커버할 수 있다. 실제로 §2.7 에서 15개 cycle ($\sim 33\%$) 만으로 90% 보존도를 달성한 것이 이 구조적 중복성 덕분이다.

(b) Algorithm 1 의 교집합 규칙이 잘 작동하는 이유. Algorithm 1 은 "활성 cycle 들의 교집합"에서 음을 뽑는데 (§2.10 규칙 1), 만약 cycle 들이 vertex 를 거의 공유하지 않으면 교집합이 항상 빈 집합이 되어 이 규칙이 작동하지 않는다. 77% 의 쌍이 vertex 를 공유하므로 교집합이 비어있지 않을 확률이 높고, 교집합에서 뽑힌 음은 "여러

cycle 이 공통으로 중요하게 여기는 음"이라는 구조적 의미를 갖는다.

3.6.6 세 곡의 비교 요약

불변량	hibari	solari	aqua	실험적 결과 연결
Deep scale	**YES**	no	no	— (diatonic 고유)
Maximal evenness	**YES**	no	no	— (diatonic 고유)
정규화 pitch entropy	0.974	0.905	0.891	FC 우위 (hibari) vs Transformer 우위 (solari)
Tonnetz 평균 degree (7-PC 한정)	$3.71/6$	(trivial)	(trivial)	Tonnetz 최적 (hibari) vs freq/Tonnetz 동등 (solari, §7.2)
서로소 주기 (phase shifting)	$\gcd(32, 33) = 1$	(미검증)	(미검증)	inst 1/2 의 위상 분리 + 쉼의 대각선 패턴
Cycle 교차 밀도	77%	(미측정)	(미측정)	Greedy 효율 + 교집합 규칙 작동

핵심 주장. hibari 의 수학적 고유성은 크게 두 축으로 이뤄진다.

1. **선택의 절제** — 12개 PC 중 7개만 쓰되, 그 7개가 deep scale + maximal evenness 를 만족하는 유일한 부분집합
2. **사용의 균등** — 선택된 7개 안에서 어느 하나를 중심으로 두지 않고 거의 동등하게 사용 (entropy 0.974)

이 두 축이 "out of noise" 의 기획 의도 — "소음과 음악의 경계" — 를 정확히 수학적으로 구현한다. 소음은 모든 주파수가 동등한 것이고, 전통 음악은 으뜸음이 지배하는 것인데, hibari 는 **diatonic 구조 안에서 가능한 한 균등한 중간 지점**에 위치한다. 그리고 이 위치가 본 연구의 모든 실험 결과 — Tonnetz 우위, FC 우위, 높은 cycle 교차 밀도 — 의 구조적 원인이다.

---

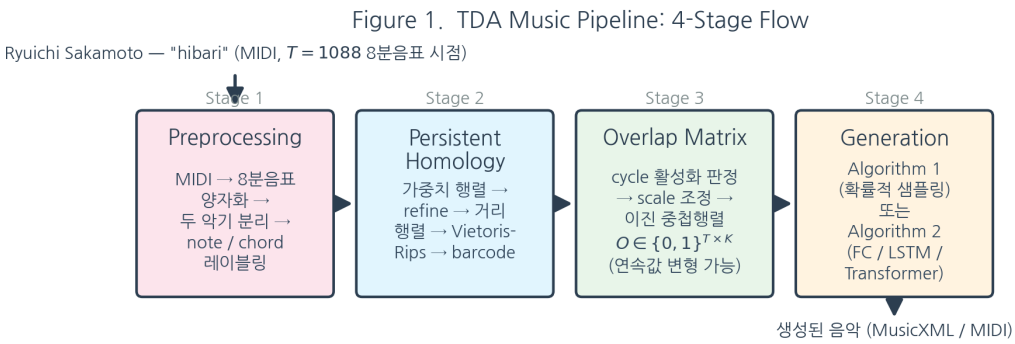
5. 시각자료 (Figures)

본 장은 논문 본문에 삽입될 정적 figure 6개와 보충자료(supplementary material)로 제공되는 1개의 interactive figure를 모아서 제시한다.

본 장에 등장하는 시각자료의 개요

#	내용	배치
1	4-stage 파이프라인 흐름도	본문
2	발견된 cycle의 3D interactive 시각화 (재생 가능)	**보충자료**
3	Tonnetz 격자에 hibari 음 배치	본문
4	Persistence barcode 다이어그램	본문
5	원곡 vs 생성곡 piano roll 비교 (3 구간)	본문
6	학습 epoch별 JS divergence 곡선	본문
7	Inst 2의 모듈 구조 + 쉼표 시각화 (모듈 확대 포함)	본문

Figure 1 — TDA Music Pipeline: 4-Stage Flow



캡션. 본 연구의 파이프라인은 네 개의 순차적 단계로 구성된다. **Stage 1**은 MIDI 원본을 8분음표 단위로 양자화하고 두 악기를 분리한 뒤 note / chord 레이블링을 수행한다. **Stage 2**는 note 간 가중치 행렬을 구축하고 refine 과정을 거쳐 거리 행렬을 만든 뒤, Vietoris-Rips 복합체로부터 1차 persistent homology를 계산한다. **Stage 3**은 발견된 모든 cycle의 활성화 정보를 시점별 이진 중첩행렬 $O \in \{0,1\}^{T \times K}$로 변환한다(연속값 변형도 정의되어 있으나 본 실험에서는 이진 형태가 주로 사용되었다). **Stage 4**는 이 중첩행렬을 seed로 하여 Algorithm 1 (확률적 샘플링) 또는 Algorithm 2 (FC / LSTM / Transformer 신경망)로 새로운 음악을 생성한다.

Figure 2 (Interactive Supplementary) — hibari Cycles in 3D, with Click-to-Play

> 본 figure는 정적 PNG 형태로는 제공되지 않으며, 별도의 interactive HTML 파일로 제공된다 (제출 시 부록 디스크 또는 URL로 동봉).

이 figure를 정적 이미지에서 제외한 이유는 다음과 같다. 발견된 cycle은 46개에 달하며, 각 cycle은 여러 note를 공유하기 때문에 임의의 한 시점·각도에서 본 PNG로는 어떤 cycle이 어떤 note들을 공유하는지 거의 식별할 수 없다. 위에서 내려다보는 각도, 옆에서 보는 각도, 회전하면서 보는 동작이 모두 필요하다.

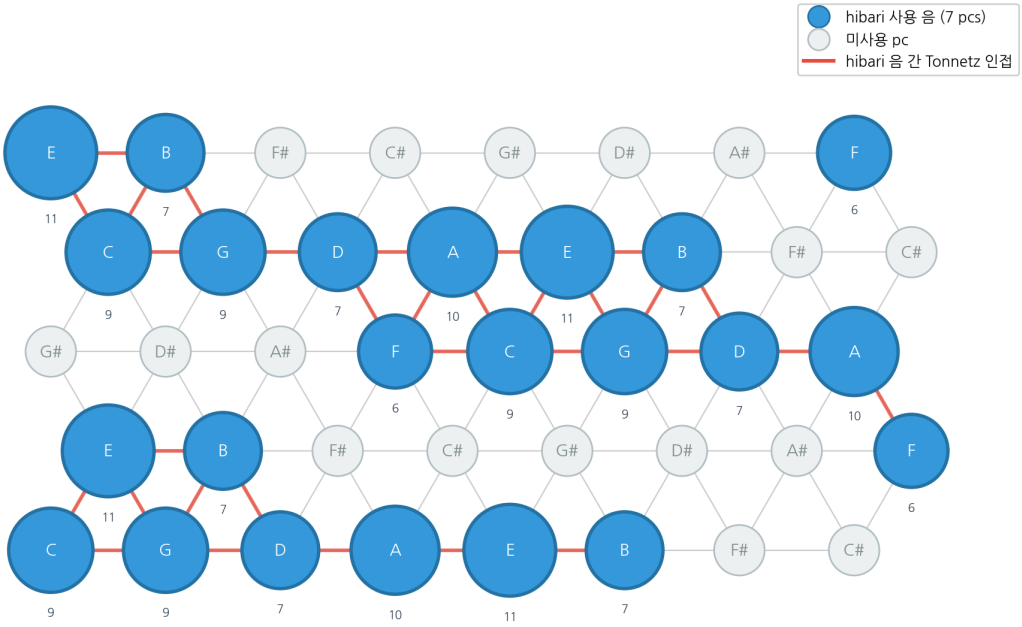
interactive HTML에서 가능한 조작.

- 마우스 드래그로 자유 회전, 스크롤로 줌
- 우측 legend 클릭으로 특정 cycle on/off
- cycle 선에 마우스를 올리면 그 cycle이 포함하는 모든 note의 pitch 이름 + 옥타브 + duration이 hover로 표시됨 (예: `label 19: C5 (dur=1)`)
- 화면 하단의 **"오디오 켜기"** 버튼을 한 번 클릭하면 브라우저 오디오 컨텍스트가 활성화되고, 이후 **cycle 선을 클릭하면** 해당 cycle을 구성하는 note들이 차례대로 sine wave로 재생됨 (Tone.js 기반, MIDI pitch → 주파수 변환). 이를 통해 "위상학적으로 한 cycle을 이루는 note들이 음악적으로 어떻게 들리는가"를 직접 청취할 수 있다.

상위 20개 cycle을 색상별로 구분하며, 카메라는 처음부터 위에서 내려다보는 각도로 설정되어 있다.

Figure 3 — Tonnetz 격자 위 hibari의 pitch class

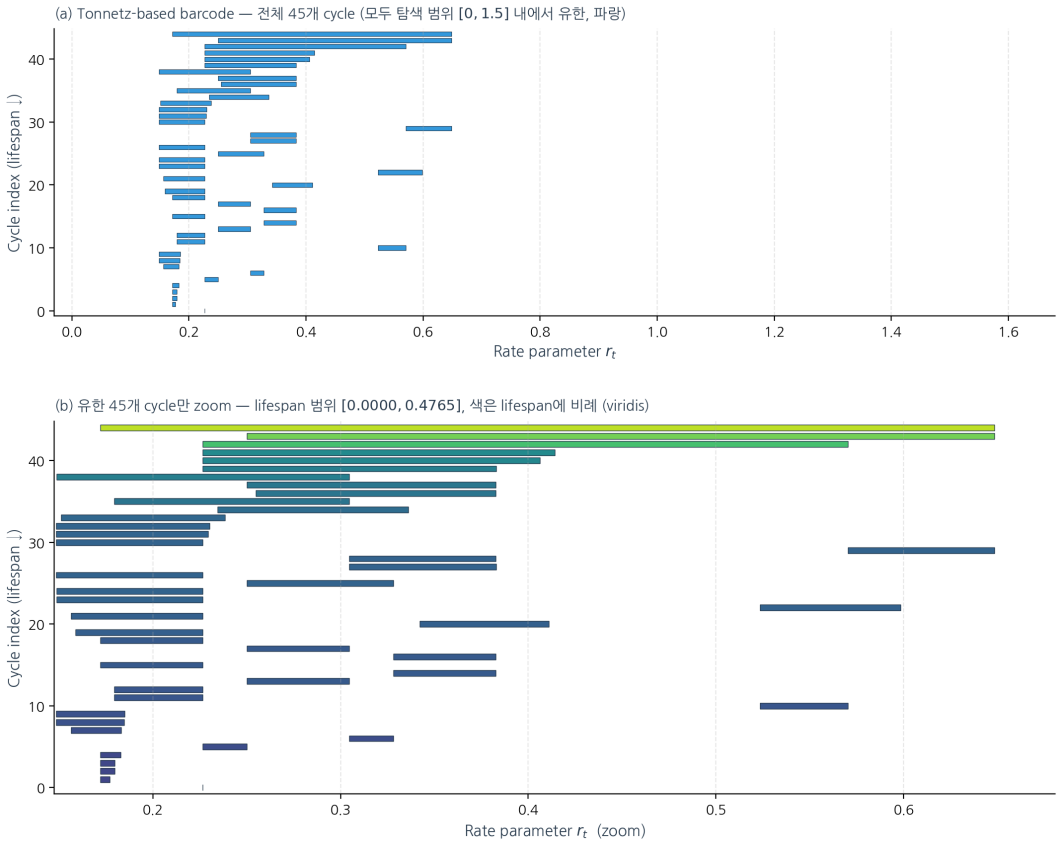
Figure 3. Tonnetz 격자에 배치된 hibari의 7개 pitch class (C major scale)
빨간 선 = Tonnetz 그래프 상의 직접 인접, 숫자 = 원곡 내 출현 횟수



캡션. 12개의 pitch class가 배치된 Tonnetz 격자(가로 = 완전 5도, 대각선 = 장 / 단 3도) 위에서 hibari가 실제로 사용하는 7개 pitch class — C, D, E, F, G, A, B (C major scale)를 진한 파랑으로 강조하였다. 각 노드의 크기는 곡 내 출현 빈도에 비례하며, 빨간 선은 사용된 pitch class 쌍 중 Tonnetz 그래프 상에서 직접 인접한(거리 1) 관계를 나타낸다. 사용된 음들이 격자 위에서 하나의 연결 성분을 이루며 집중되어 있음을 확인할 수 있는데, 이는 hibari가 음악이론적으로 잘 정의된 토너츠 영역 안에서만 작동하는 곡이며, 따라서 Tonnetz 거리 함수가 frequency 거리보다 이 곡의 위상 구조를 더 잘 포착한다는 3.1절의 결과와 일관된다.

Figure 4 — Persistence Barcode Diagram (Tonnetz distance)

Figure 4. Persistence Barcode — hibari H_1 cycles (Tonnetz hybrid distance, $\alpha = 0.5$, timeflow weight)



데이터 출처. 본 연구의 주 거리 함수는 **Tonnetz hybrid distance** ($\alpha = 0.5$) 이므로, barcode 역시 Tonnetz 기반으로 계산한다. 구체적으로는 `topology.generate_barcode_numpy`를 rate $r_t \in [0, 1.5]$의 0.01 간격 grid에서 호출하여 H_1 persistence를 직접 재계산한 뒤, 결과를 `docs/figures/fig4_tonnetz_persistence.pkl`에 캐시하였다 (계산 시간 약 2~3분, 총 45개 cycle). 주의: 본 연구 초기에 frequency distance로 계산된 `pickle/h1_rBD_t_notes1_1e-4_0.0~1.5.pkl`에는 48개 cycle이 들어 있지만, 거리 함수가 다르면 발견되는 cycle도 달라지므로 본 figure에서는 해당 pkl을 사용하지 않는다.

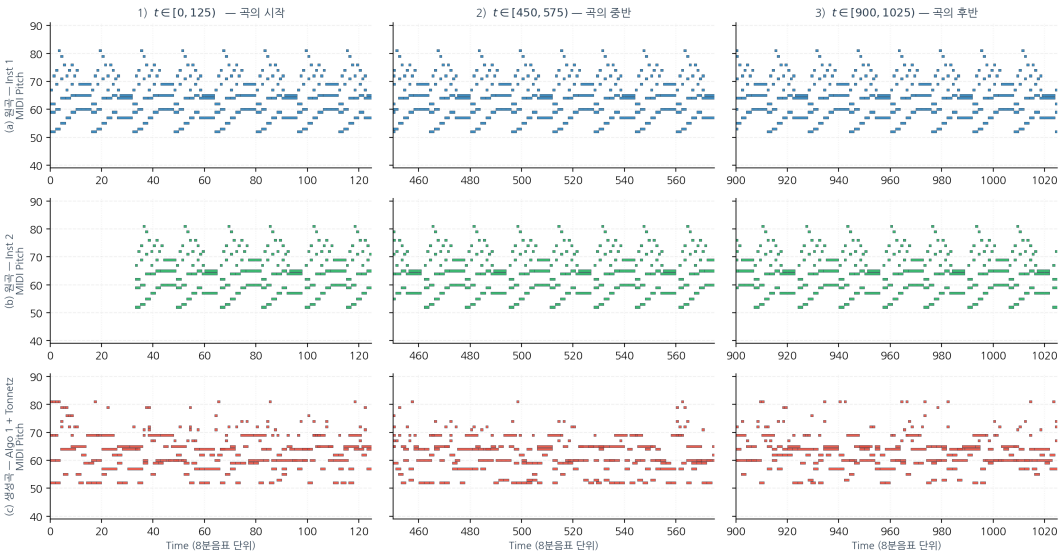
캡션. hibari의 H_1 cycle 전체 45개에 대한 Tonnetz 기반 persistence barcode를 두 panel로 나누어 보인다.

- **(a) 전체 barcode** — 가로축은 rate parameter r_t, 각 막대는 한 cycle의 $[\mathrm{birth}, \mathrm{death}]$ 구간이다. Tonnetz 거리에서는 frequency와 달리 **45개 cycle 전부가 탐색 범위** $[0, 1.5]$ **내에서 유한**하다 (즉 탐색 범위를 벗어나는 "영속 cycle"이 하나도 없다). 위쪽 몇 개 cycle은 lifespan이 0.4를 넘을 정도로 매우 안정적이며, 아래로 갈수록 lifespan이 급격히 짧아진다.
- **(b) 유한 cycle 전체 zoom** — 짧은 lifespan 쪽 cycle들도 식별할 수 있도록 x축을 실측 범위 $[0.00004, 0.47654]$로 조정하였다. 색은 viridis colormap으로 lifespan에 비례한다. 가장 긴 cycle과 가장 짧은 cycle 사이에 약 $10{,}000$배 이상의 lifespan 차이가 있어, Tonnetz 기반 filtration이 hibari에서 구조적으로 중요한 소수 cycle을 매우 선명하게 분리함을 확인할 수 있다.

frequency 기반 barcode에서 11개의 "영속 cycle"이 있었던 것과 달리 Tonnetz 기반에서는 0개 — 이는 Tonnetz가 "어떤 pitch class 관계가 음악적으로 가까운가"에 대해 더 엄격한 기준을 적용하여, frequency가 단순 빈도만으로 "가깝다"고 판정했던 일부 note 쌍을 분리하기 때문이다. 결과적으로 Tonnetz에서는 모든 cycle이 유한한 scale에서 소멸하며, 이 중에서도 일부는 매우 오래 살아남아 lifespan 순위가 명확해진다. Step 3 Experiment 1에서 Tonnetz가 frequency 대비 JS divergence를 47% 낮춘 것도 이러한 "더 선명한 위상 구조 분리"와 일치하는 관찰이다.

Figure 5 — Piano Roll 비교 (3 구간)

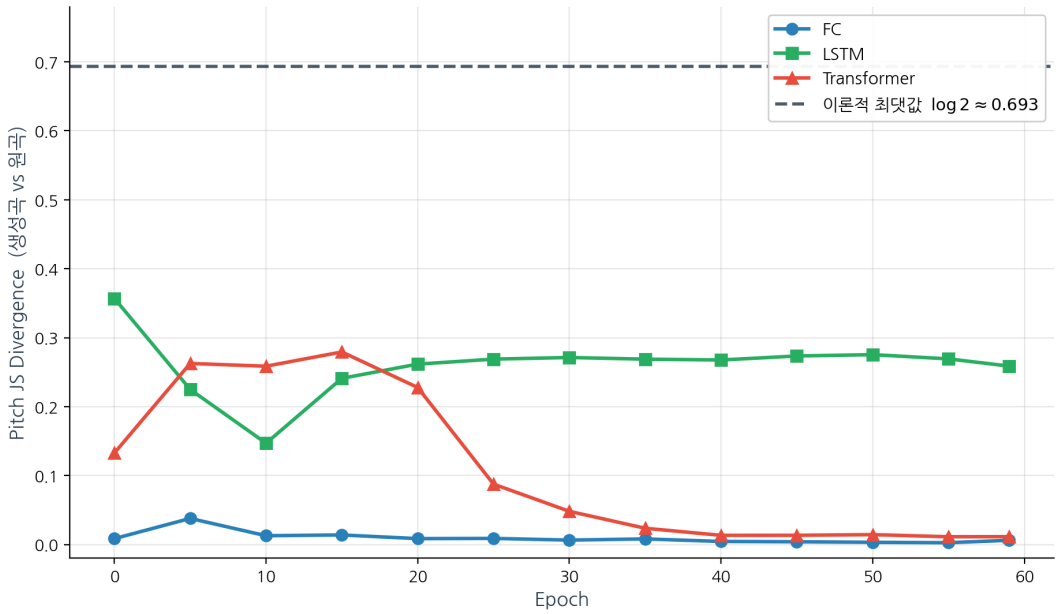
Figure 5. Piano Roll 비교 — 원곡 vs Tonnetz 기반 Algorithm 1 생성곡 (3개 구간 × 125 timesteps)



캡션. 위에서부터 순서대로 (a) 원곡 악기 1, (b) 원곡 악기 2, (c) Algorithm 1 + Tonnetz (K=46, seed=42)로 생성된 곡이다. 전체 $T = 1088$ timesteps을 한 번에 보이면 해상도가 낮아 개별 note가 식별되지 않으므로, 곡의 시작 ($t \in [0, 125)$), 중반 ($t \in [450, 575)$), 후반 ($t \in [900, 1025)$) 세 구간을 각 125 timesteps 씩 나란히 보여준다. 가로축은 시간(8분음표 단위), 세로축은 MIDI pitch이다. 각 구간에서 생성곡의 pitch 분포와 시간적 밀도가 원곡과 유사한 영역에서 활성화되고 있음을 시각적으로 확인할 수 있으며, 이는 3.1절의 JS divergence 정량 결과(0.0398)와 일관된다.

Figure 6 — 학습 Epoch별 JS Divergence 곡선

Figure 6. 학습 epoch별 JS Divergence — FC / LSTM / Transformer
(Tonnetz 기반 overlap, 동일 초기 seed)

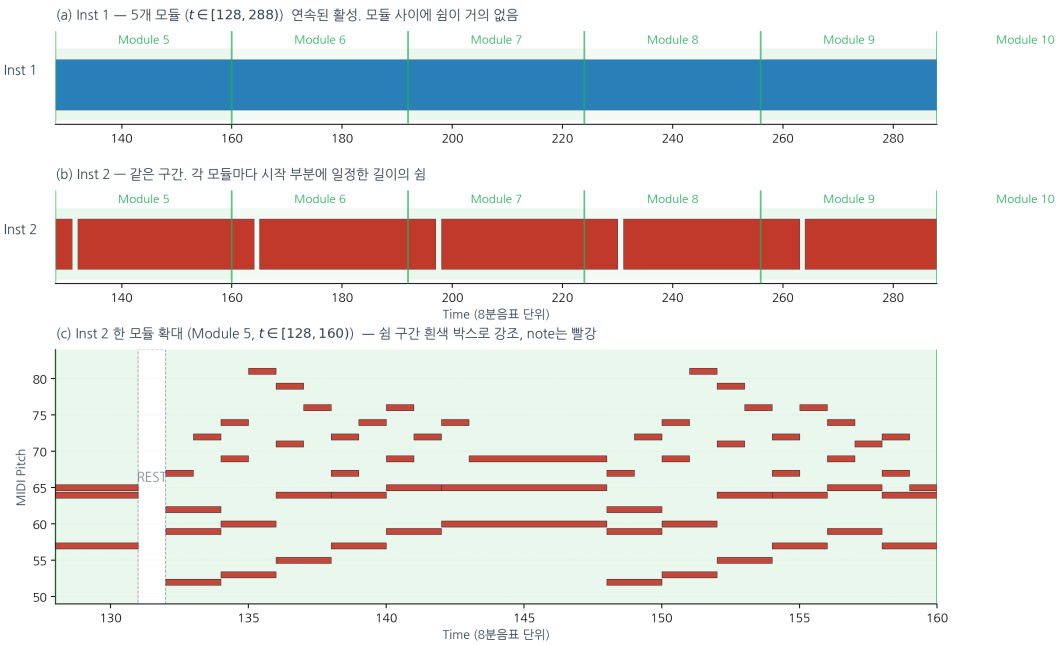


캡션. FC / LSTM / Transformer 세 모델을 동일한 데이터(Tonnetz overlap)와 동일 seed에서 60 epoch 학습시키면서, 매 5 epoch마다 생성곡을 만들어 원곡과의 pitch JS divergence를 측정한 결과이다. 진한 검정 파선은 이론적 최댓값 $\log 2 \approx 0.693$이다. FC 모델이 epoch 30 근처에서 $\mathrm{JS} \approx 0.003$에 도달하여 가장 빠르게 수렴하고, Transformer는 epoch 30 이후 같은 수준까지 따라잡는다. 반면 LSTM은 $\mathrm{JS} \approx 0.26$ 근처에서

plateau에 머물며 추가 개선이 일어나지 않는다. 이는 3.4절의 해석 — "hibari는 out of noise 앨범의 곡으로서 시간 문맥보다 음의 공간적 배치가 핵심이며, 시간 인과를 강하게 모델링하는 LSTM은 오히려 곡의 미학과 어긋난다" — 와 일관된 관찰이다.

Figure 7 — 두 악기의 모듈 반복 (Inst 1은 연속, Inst 2는 모듈마다 쉼)

Figure 7. 두 악기의 모듈 반복 — Inst 1은 연속, Inst 2는 모듈마다 쉼



캡션. 본 figure는 hibari의 두 악기가 **같은 32-timestep 모듈 구조를 반복**하지만, 활성/쉼 패턴의 성격이 근본적으로 다름을 시각적으로 드러낸다. 개별 note는 (a)와 (b)에서 의도적으로 표시하지 않는다. 중요한 것은 "언제 악기가 울리고 언제 쉬는가"의 거시적 패턴이기 때문이다. 셋 panel 모두 같은 시간축 (8분음표 단위)을 공유한다.

- **(a) Inst 1 — Modules 5~9 ($t \in [128, 288)$)** — 5개 모듈의 활성 구간을 파란 띠로 표시하였다. 진한 초록 수직선은 모듈 경계, 옅은 초록 배경은 모듈 영역이다. 이 5개 모듈 구간 동안 inst 1의 활성 띠에는 **쉼(흰 공백)이 전혀 나타나지 않는다**. 실제로 전체 $T = 1056$ timestep에 걸쳐 inst 1의 쉼 구간은 0 timestep이다 — inst 1은 처음부터 끝까지 쉬지 않고 연주한다.
- **(b) Inst 2 — 같은 구간 ($t \in [128, 288)$)** — 같은 시간축에 inst 2의 활성을 빨간 띠로 표시하였다. inst 1과 달리 각 모듈의 초반부마다 일정한 길이의 흰 공백 — 즉 **쉼** — 이 규칙적으로 나타난다. 전체 $T = 1056$ timestep 중 inst 2의 쉼 구간은 총 64 timestep이며, 대부분 모듈 경계 직전/직후에 집중되어 있다. 두 악기가 같은 모듈 길이를 공유하면서도 rest의 존재 여부에서 근본적으로 다른 역할을 맡고 있음이 드러난다.
- **(c) Inst 2 한 모듈 확대 (Module 5, $t \in [128, 160)$)** — inst 2의 한 모듈 안을 piano roll로 확대하여, 모듈 내 쉼 구조를 자세히 관찰한다. 빨간 사각형은 inst 2의 실제 note이며, 흰색(점선 테두리) 박스는 쉼 구간으로 "REST" 라벨이 표시된다. 쉼이 모듈의 어디에 어떤 길이로 배치되는지 자세히 볼 수 있다.

이 figure가 본 연구에서 중요한 이유는 세 가지이다.

1. **사카모토 인터뷰의 시각적 검증.** inst 1이 시간 방향으로 "끊김 없이 흘러가는 기저 흐름"을 담당하고 inst 2가 그 위에 "의도적으로 쉼을 두며 얹히는" 역할을 맡는 구조는, 사카모토가 인터뷰에서 밝힌 "한 악기를 시간 방향으로 충분히 채운 뒤 다른 악기를 그 위에 겹쳐 배치한다"는 작곡 방식 (§2.9 참조)과 정확히 일치한다.
2. **Intra / Inter 가중치 분리의 정당화.** 두 악기가 이렇게 다른 성격의 활성 패턴을 가진다면, 둘을 하나의 단일 가중치 행렬로 묶는 것은 정보 손실을 유발한다. 본 연구가 intra (각 악기 내부) / inter (두 악기 사이의 시차 결합) / simul (동시 발음)을 분리한 것은 이러한 **두 악기의 역할 차이**를 보존하기 위한 필연적 선택임을 figure가 시각적으로 정당화한다.

3. **모듈 기반 생성의 근거.** Algorithm 1이 사용하는 32-element chord-height 패턴은 임의 수열의 단순 반복이 아니라, 이 모듈 구조 자체에서 추출된 것이다. 즉 위상학적으로 모든 cycle 정보(중첩행렬)와 음악적 모듈 구조가 같은 시간축 위에서 결합되어 음악이 생성된다.

재현 방법

(전공자 / 코드 재현이 필요한 독자만 참조. 인쇄 제출본에는 본 절을 넣지 않는다.)

```
cd tda_pipeline/docs/figures

#    figures (      )
python make_fig1_pipeline.py        #
python make_fig3_tonnetz_hibari.py  #
python make_fig4_barcode.py         # pickle/h1_rBD_*.pkl
python make_fig5_pianoroll.py       #       + cache    (~5s)
python make_fig6_js_curve.py        # fig6_curves.json        ,
                                    #            (~2-3 min)
python make_fig7_inst2_modules.py   #           (~3s)

# Interactive supplementary
python make_fig2_interactive.py     # Plotly + Tone.js HTML
```

모든 정적 figure는 200 dpi PNG로 저장된다. 한글 폰트는 `_fontsetup.py`에서 NanumGothic을 자동 등록한다. Interactive figure는 `plotly` 패키지가 필요하며, 생성된 HTML은 외부 CDN에서 plotly.js와 Tone.js를 로드하므로 처음 열 때 인터넷 연결이 필요하다.

---

6. 차별화 포인트 — 기존 연구와의 비교

본 연구의 위치를 명확히 하기 위해, 두 가지 관련 연구 흐름과 비교한다. 하나는 **일반적인 AI 음악 생성 연구** 이며, 다른 하나는 **TDA를 음악에 적용한 선행 연구**들이다.

6.1 일반 AI 음악 생성 연구와의 차별점

지난 10년간 Magenta, MusicVAE, Music Transformer 등 대규모 신경망 기반 음악 생성 모델이 여러 발표되었다. 이들은 공통적으로 다음 구조를 따른다:

$$\{\text{¤¤¤¤¤MIDI¤¤¤¤}\} \rightarrow \text{¤¤¤¤¤¤¤} \rightarrow \text{¤¤¤¤¤¤¤}$$

본 연구는 이와 다음 네 가지 지점에서 근본적으로 다르다.

(1) 코퍼스 규모 vs 단일곡 심층 분석. 일반적인 AI 음악 생성은 "많은 곡을 보고 평균적인 음악 규칙을 배우는 것"을 목표로 한다. 본 연구는 반대로 **단 한 곡(hibari)의 구조를 가능한 한 깊이 해석한 뒤 그 구조를 재생성**하는 것을 목표로 한다. 음악 학자가 한 곡의 악보를 정밀하게 분석하는 작업에 가깝다.

(2) Blackbox 학습 vs 구조화된 seed. 일반 신경망 모델은 학습이 끝난 후 "왜 이 음이 나왔는가"를 설명하기 어렵다. 본 연구의 파이프라인은 **persistent homology로 추출한 cycle 집합**이라는 명시적이고 해석 가능한 구조를 seed로 사용하며, 생성된 모든 음은 "특정 cycle의 활성화"라는 구체적 근거를 갖는다. 즉 생성 결과가 역추적 가능하다.

(3) 시간 모델링의 역설. 일반 음악 생성 모델은 "더 정교한 시간 모델일수록 더 좋다"는 암묵적 가정을 가지며, 그래서 Transformer 계열 모델이 주류가 되었다. 본 연구의 §3.4에서 관찰된 "가장 단순한 FC가 가장 좋은 결과를 낸다"는 결과는, 이러한 일반적 가정이 **곡의 미학적 성격에 따라 뒤집힐 수 있다**는 증거이다. hibari처럼 시간 인과보다 공간적 배치를 중시하는 곡에서는 시간 문맥을 무시하는 모델이 오히려 곡의 성격에 더 맞다.

(4) 작곡가의 작업 방식 반영. 본 연구의 가중치 분리 (intra / inter / simul) 는 사카모토 본인이 인터뷰에서 밝힌 작곡 방식 — "한 악기를 시간 방향으로 충분히 채운 뒤 다른 악기를 그 위에 겹쳐 배치" — 을 수학적 구조에 직접

반영한 것이다 (§2.9). 일반적인 AI 음악 생성에서는 모델의 architectural choice가 "학습 효율"에 따라 결정되지만, 본 연구에서는 **작곡가의 실제 작업 방식**이 설계의 출발점이다.

6.2 기존 TDA-Music 연구와의 차별점

TDA를 음악에 적용한 선행 연구는 몇 편이 있으며, 본 연구와 가장 가까운 것은 다음 두 편이다.

- **Tran, Park, & Jung (2021)** — 국악 정간보(Jeongganbo)에 TDA를 적용하여 전통 한국 음악의 위상적 구조를 분석. 초기적 탐구 연구이며, 본 연구가 사용하는 파이프라인의 공통 조상.
- **이동진, Tran, 정재훈 (2024)** — 국악의 기하학적 구조와 AI 작곡. 본 연구의 지도교수 연구실의 직전 연구이며, 본 연구가 계승한 pHcol 알고리즘 구현과 $\rho^* = 0.35$ 휴리스틱의 출처.

본 연구가 이들 대비 새로 기여하는 지점은 다음 다섯 가지이다.

(A) 네 가지 거리 함수의 체계적 비교. 선행 연구들은 frequency 기반 거리만을 사용했으나, 본 연구는 frequency, Tonnetz, voice-leading, DFT 네 가지를 동일한 파이프라인 위에서 $N = 20$회 반복 실험으로 정량 비교하였다 (§3.1). 이를 통해 "Tonnetz가 frequency 대비 JS divergence를 47% 낮춘다"는 음악이론적 정당성을 실증적으로 제공한다.

(B) Continuous overlap matrix의 도입과 검증. 선행 연구들은 이진 overlap matrix만을 사용했다. 본 연구는 희귀도 가중치를 적용한 continuous 활성도 개념을 새로 도입했으며 (§2.5), $\tau = 0.5$ 임계값 이진화가 추가로 11.4% 개선을 주는 것을 통계 실험으로 검증하였다 (§3.3a).

(C) 통계적 엄밀성. 선행 연구들은 단일 run 결과만 보고했지만, 본 연구는 모든 baseline에서 $N = 20$회 반복 실행하여 mean ± std, Welch t-test, Cohen's d를 보고한다. 이는 TDA-music 분야에서 "효과가 실재하는가"를 통계적으로 검증한 최초의 사례 중 하나이다.

(D) 서양 현대음악으로의 확장. 기존 연구가 국악을 대상으로 했다면, 본 연구는 서양의 minimalism / ambient 계열 현대음악 (사카모토 out of noise) 으로 적용 범위를 확장했다. 이를 통해 TDA 기반 분석이 **장르 특이적이지 않음**을 보였다.

(E) 곡의 미학적 맥락과 모델 선택의 연결. 본 연구의 §3.4 해석 — FC 모델 우위를 out of noise 앨범의 작곡 철학으로 설명 — 은 기존 TDA-music 연구에 없던 관점이다. "어떤 곡에는 어떤 모델이 맞는가"가 단순히 성능 최적화 문제가 아니라 **미학적 정합성 문제**임을 제시하며, solari 실험 (§7.2)에서 Transformer 최적이라는 반대 패턴으로 이 가설이 실증되었다.

(F) 위상 보존 음악 변주. 기존 TDA-music 연구는 분석(analysis) 또는 재현(reproduction)에 그쳤다. 본 연구는 Tonnetz 최소매칭 거리 기반 note 재분배 + 시간 재배치 + 화성 제약을 결합하여, **위상 구조를 보존하면서 원곡과 다른 음악을 생성**하는 프레임워크를 제시하였다 (§7.3-§7.6). 이는 TDA를 음악 창작의 제약 조건 생성기로 사용한 최초의 시도이다.

6.3 세 줄 요약

1. 본 연구는 단일곡의 위상 구조를 깊이 이해하고 그 구조를 보존한 채 재생성하는 심층 분석 — 재생성 파이프라인이며, 나아가 위상 보존 변주까지 확장한다.
2. 네 가지 거리 함수, 두 가지 overlap 형식, 세 가지 신경망 모델, 통계적 반복, 그리고 두 곡(hibari/solari)의 대비 검증이 본 연구의 경험적 기여이다.
3. 작곡가의 작업 방식 (§2.9) 과 곡의 미학적 맥락 (§3.4, §7.2) 을 수학적 설계에 직접 반영한 것이 본 연구의 해석적 기여이다.

---

7. 확장 실험 및 향후 연구 방향

본 연구는 원곡 재현(§3-§4)을 넘어 여러 방향의 확장 실험을 수행하였다. 완료된 실험과 향후 과제를 함께 정리한다.

7.1 모듈 단위 생성 + 구조적 재배치 (Most Immediate)

본 연구는 지금까지 "전체 $T=1{,}088$ timesteps에 해당하는 음악을 한 번에 생성"하는 방식을 사용했다. 그러나 §5 Figure 7에서 드러났듯이, hibari의 실제 구조는 **32-timestep 모듈이 33회 반복**되는 형태이며, 두 악기는 inst 1 = 연속적 기저 흐름, inst 2 = 모듈마다 쉼을 두는 얹힘 구조라는 상보적 역할을 가진다.

이 관찰을 반영한 새로운 생성 전략이 가능하다:

1. **한 모듈만 생성** — persistent homology로 얻은 cycle 집합 중 모듈 1개(32 timesteps)의 위상 구조를 보존하는 최소 단위의 생성을 먼저 수행한다. 이는 기존 파이프라인을 $T=32$로 줄여 그대로 실행한 뒤, 중첩행렬의 한 모듈 폭 만큼만 seed로 사용하면 된다.
2. **hibari 구조에 따라 배치** — 생성된 모듈을 33회 반복하되:

- inst 1 자리에는 동일한 모듈을 쉼 없이 이어 붙임

- inst 2 자리에는 같은 모듈을 각 반복마다 inst 1 대비 shift (Figure 7 관찰에 맞게) + 모듈마다 지정된 위치에 쉼을 삽입

3. **모듈 간 이행 매끄럽게** — 단순 복제가 아니라 모듈 경계에서 자연스러운 이행이 이루어지도록 ±2 timesteps 정도의 "이음새" 영역을 가변 cycle로 재샘플링.

이 접근의 이점은 다음과 같다.

- **계산 효율**: $T=1{,}088$ 대신 $T=32$로 persistent homology를 계산하므로 수 배 빠르다.
- **구조적 충실도**: hibari의 고유 모듈 구조를 재샘플링이 아닌 복제로 보존하므로 곡의 정체성이 유지된다.
- **변주 가능성**: 한 모듈의 cycle seed만 바꾸면 전체 곡 수준의 변주가 자동으로 만들어진다.

7.2 다른 곡으로의 일반화 — solari 실험 결과

본 연구의 §3.4 해석 8("hibari의 FC 우위는 곡의 미학적 성격에서 기인")을 검증하기 위해, 같은 out of noise 앨범의 **solari**에 동일한 파이프라인을 적용하였다. solari는 hibari와 같은 앨범에 수록되어 있으나, 12개 pitch class를 모두 사용하는 반음계적(chromatic) 곡으로 성격이 대비된다. 실험 러너는 `run_solari.py`, 결과는 `docs/step3_data/solari_results.json`.

solari 기본 정보: $N=34$ 고유 note, $T=224$ timesteps, 49개 고유 화음, tempo ≈ 29 BPM.

Algorithm 1 — 거리 함수 비교

거리 함수	cycle 수	density	JS (mean ± std)	JS min
frequency	22	0.070	0.063 ± 0.005	0.056
Tonnetz	39	0.037	0.063 ± 0.003	0.059
voice-leading	25	0.043	0.078 ± 0.004	0.073

hibari에서는 Tonnetz가 frequency 대비 47% 우위였지만, solari에서는 **frequency와 Tonnetz가 거의 동일** (0.063 vs 0.063)하며 voice-leading이 가장 나쁘다. 이는 §3.6에서 분석한 solari의 12-PC 구조 — Tonnetz 그래프의 지름이 2에 불과하여 구별력이 낮음 — 과 일치한다.

Algorithm 2 — DL 모델 비교

설정	FC	LSTM	Transformer
binary JS	0.106	0.168	0.032
continuous JS	0.042	0.171	0.016

핵심 발견: solari에서는 Transformer가 최적. hibari와 정확히 반대 패턴이다. hibari에서 FC 최적 / Transformer 열등이었던 것이, solari에서는 Transformer가 FC의 2.6배 ($0.042 \rightarrow 0.016$, continuous 기준) 우위이다.

곡의 성격이 최적 모델을 결정한다는 가설의 확증. hibari (diatonic, entropy 0.974, 공간적 배치)에서는 시간 문맥을 무시하는 FC가 최적이었고, solari (chromatic, entropy 0.905, 선율적 진행)에서는 시간 문맥을 적극 활용하는 Transformer가 최적이다. 이 대비는 §3.4 해석 8을 실증적으로 뒷받침한다.

곡	PC 수	정규화 entropy	최적 거리	최적 모델	해석
hibari	7 (diatonic)	0.974	Tonnetz	FC	공간적 배치, 시간 무관
solari	12 (chromatic)	0.905	frequency/Tonnetz 동등	Transformer	선율적 진행, 시간 의존

Continuous overlap의 효과

solari에서도 continuous overlap은 이진 대비 개선을 보였다. Transformer 기준 binary JS $0.032 \rightarrow$ continuous JS 0.016 (50% 감소). 이 개선폭은 hibari의 개선 F (57% 감소)와 비슷한 수준으로, continuous overlap의 효과가 곡 독립적(song-agnostic)임을 시사한다.

향후 과제

대조군으로 **전통적 선율 인과가 강한 곡** (바흐 fugue, Ravel "Pavane" 등)에 같은 파이프라인을 적용하여 Transformer 우위가 더 강해지는지 검증이 필요하다. 이 두 그룹의 대비가 명확히 드러나면, "곡의 미학과 모델 구조의 정합성" 가설이 genre-dependent music modeling이라는 더 일반적인 framework로 확장될 수 있다.

7.3 방향 A: 거리 보존 note 재분배

지금까지의 모든 실험은 원곡의 pitch 분포를 가능한 한 재현하는 것을 목표로 했다. 본 절부터는 방향을 전환하여, **위상 구조를 보존하면서 원곡과 다른 음악**을 생성하는 문제를 다룬다.

아이디어

중첩행렬(시간 구조)은 그대로 보존하되, 각 cycle에 배정된 note를 새로운 note로 교체한다. 교체의 제약 조건은 다음과 같다:

1. 새 note 집합의 Tonnetz 거리행렬이 원본과 **up to permutation으로 유사**해야 한다 (정의 2.11)
2. 새 cycle의 Tonnetz 최소매칭 거리 (정의 2.10)가 원본 cycle과 가까워야 한다
3. 새 note의 pitch 범위를 제어하여 얼마나 다른 곡을 원하는지를 조절한다

코드: `note_reassign.py`, 결과: `docs/step3_data/note_reassign_results.json`.

실험 설계

Tonnetz 거리 기반으로 랜덤 후보 1000개를 생성하고, 비용 $0.5 \times \text{note_dist_error} + 0.5 \times \text{cycle_dist_error}$가 최소인 후보를 선택한다. pitch 범위를 3단계로 변화시켜 "얼마나 다른 음"을 허용할지를 조절한다.

Algorithm 1 결과

설정	pitch 범위	note 오차	cycle 오차	DTW vs 원곡	pitch JS
baseline (원곡)	52-81	—	—	2.21	0.016
tonnetz₩_narrow	55-79	4.00	1.17	2.09 (-5.8%)	0.283
tonnetz₩_wide	48-84	4.46	1.18	2.89 ($+30.3\%$)	0.358
tonnetz₩_vwide	40-88	4.48	1.06	3.57 ($+61.4\%$)	0.419

해석: pitch 범위를 넓힐수록 DTW (선율 차이)가 크게 증가하여 vwide에서 $+61.4\%$에 달한다. 동시에 cycle 거리 오차는 ~ 1.1로 안정적 — 즉 **위상적 거리 구조는 보존되면서 선율은 확실히 다른 곡이 생성**된다. pitch JS가 높은 것은 다른 음을 사용하므로 당연하며, 이 실험에서의 평가 기준은 JS가 아니라 "거리 구조 보존도"이다.

Algorithm 2 (DL) 결과

재분배된 note 위에서 LSTM/Transformer를 재학습시킨 결과:

설정	모델	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	cycle 오차

baseline	Transformer	0.016	1.84	—	—
wide	LSTM	0.498	2.31	0.317	1.18
wide	Transformer	0.362	2.26	0.011	1.18
vwide	LSTM	0.441	2.01	0.317	1.06
vwide	Transformer	0.426	3.09	0.006	1.06

핵심 발견: Transformer는 재분배된 note의 분포를 거의 완벽하게 학습한다 (vs ref pJS $0.006 \sim 0.011$). 이는 Transformer가 중첩행렬 + 새 note 매핑을 정확히 재현할 수 있음을 의미하며, vwide + Transformer 조합에서 DTW 3.09 ($+68\%$)로 원곡과 가장 다른 선율이 생성된다.

7.4 방향 B: 중첩행렬 시간 재배치

방향 A가 어떤 note를 바꾸는 것이라면, 방향 B는 언제 연주하는지를 바꾸는 것이다. 중첩행렬의 **행(시점)**을 재배치하여 같은 cycle 구조를 다른 시간 순서로 전개한다.

3가지 재배치 전략

1. **segment_shuffle**: 전체 T를 길이 64의 세그먼트로 나누고, 세그먼트 순서를 무작위 셔플. 세그먼트 내부 순서는 유지.
2. **block_permute** (block size 32/64): 고정 크기 블록을 무작위 순열로 재배치.
3. **markov_resample** ($\tau = 1.0$): 원본 중첩행렬의 전이확률로부터 Markov chain을 추정하고, 온도 τ로 새 시퀀스를 생성 (정의 2.14).

코드: `temporal_reorder.py`, 결과: `temporal_reorder_dl_results.json`, `temporal_reorder_dl_v2_results.json`.

Transformer 결과 (PE 있음, 원본 학습)

전략	pitch JS	transition JS	DTW	DTW 변화
baseline	0.006	0.102	1.79	—
segment₩_shuffle	0.006	0.123	1.90	$+5.8\%$
block32	0.006	0.104	1.86	$+3.6\%$
markov ($\tau = 1.0$)	0.008	0.104	1.84	$+2.7\%$

재배치 효과가 미약하다. segment₩_shuffle이 가장 큰 변화를 보이지만 DTW $+5.8\%$에 불과하다.

PE 제거 + 재학습 실험

Transformer에서 positional embedding(PE)을 제거하고, 재배치된 중첩행렬로 재학습:

설정	pitch JS	transition JS	DTW	DTW 변화
noPE₩_baseline	0.011	0.128	1.85	—
noPE₩_segment₩_shuffle	0.011	0.149	1.87	$+1.0\%$
noPE+retrain segment₩_shuffle ★	0.173	0.399	2.22	$+21.7\%$
noPE+retrain markov ($\tau = 1.0$)	0.185	0.443	2.16	$+18.0\%$

방향 B의 딜레마: PE 제거 + 재학습에서 DTW가 $+21.7\%$까지 증가하여 선율이 확실히 바뀌지만, 동시에 pitch JS가 $0.007 \rightarrow 0.173$으로 **분포가 붕괴**된다.

- 약한 재배치 → pitch 보존, 선율 변화 없음
- 강한 재배치 → 선율 변화, pitch 분포도 붕괴

이 딜레마는 방향 B 단독으로는 "pitch 유지 + 선율 변화"를 동시에 달성하기 어려움을 의미한다. **방향 A (note 교체)와 결합**해야 양쪽의 장점을 취할 수 있다.

7.5 화성 제약 조건

방향 A의 note 재분배는 위상적 거리만 보존하므로, 결과가 **음악적으로 불협화**할 수 있다. 본 절은 화성(harmony) 제약을 추가하여 재분배의 음악적 품질을 개선한다.

3가지 화성 제약 방법

1. **scale 제약**: 새 note의 pitch class를 특정 스케일 (major, minor, pentatonic)에 한정. 허용 pool 크기가 줄어들지만 음악적 일관성이 보장된다.
2. **consonance 목적함수**: 재분배 비용에 평균 dissonance (정의 2.13)를 penalty로 추가. $\text{cost} = \alpha \cdot \text{dist_err} + \beta \cdot \text{diss}$.
3. **interval structure 보존**: 원곡의 interval class vector (정의 2.12)와 새 note 집합의 ICV 차이를 penalty로 추가.

결과: `note_reassign_harmony_results.json`.

Algorithm 1 결과

설정	note 오차	cycle 오차	평균 dissonance	scale match	PC 수
baseline (제약 없음)	4.35	1.35	0.412	0.70 (C♯ major)	10
scale_major ★	3.52	1.48	0.361	1.00 (C major)	7
scale₩_minor	3.68	1.69	0.363	1.00 (E major)	7
scale₩_penta	4.32	1.48	0.213	1.00 (C major)	5
consonance 단독	4.35	1.35	0.412	0.70	10
interval 단독	4.35	1.34	0.437	0.64	11

핵심 발견:

1. **scale_major가 가장 효과적**: note 거리 오차가 baseline $4.35 \rightarrow 3.52$로 19% 감소하면서 동시에 scale match 1.00, dissonance $0.412 \rightarrow 0.361$로 화성적 품질도 개선된다. C major로 고정되므로 원곡(hibari)의 조성과 일치한다.
2. **scale_penta가 가장 낮은 dissonance**: 0.213으로 baseline의 절반 수준. 5음만 사용하므로 불협화 자체가 구조적으로 억제된다.
3. **consonance 단독은 무효과**: 비용에 추가해도 최적 후보가 바뀌지 않았다. dissonance penalty가 거리 보존 제약에 비해 너무 약한 것으로 추정된다.

Algorithm 2 (Transformer) 결과

설정	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	val₩_loss
original	0.009	1.80	—	0.524
baseline (제약 없음)	0.600	3.46	0.007	0.497

scale_major ★	0.097	2.35	0.003	0.492
scale₩_penta	0.259	3.37	0.009	0.487

scale_major + Transformer 조합은 원곡 대비 pJS 0.097 (적당히 다름), DTW 2.35 ($+31\%$, 다른 선율), ref 대비 pJS 0.003 (재분배된 note 분포를 거의 완벽 학습)으로, **위상 보존 + 적당한 차이 + 화성적 일관성**의 균형이 가장 좋다.

7.6 A+B 결합 + continuous overlap — 최종 통합 실험

방향 A (note 재분배 + 화성 제약)와 방향 B (시간 재배치)를 결합하고, continuous overlap matrix를 적용한 최종 실험이다. 결과: `combined_AB_results.json`, `harmony_continuous_results.json`.

실험 설계

- **note 설정**: 원곡(orig) 또는 scale₩_major 재분배(major)
- **시간 설정**: none, segment₩_shuffle, block32, block64, markov ($\tau = 1.0$)
- **overlap**: binary 또는 continuous
- **모델**: Transformer (continuous overlap 직접 입력)

결합 실험 주요 결과

설정	note	reorder	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS
orig₩_none	orig	none	0.014	1.52	0.014
orig₩_segment₩_shuffle	orig	shuffle	0.007	1.87	0.007
orig₩_block32	orig	block32	0.005	1.84	0.005
major_none	major	none	0.125	2.38	0.010
major_segment_shuffle	major	shuffle	0.117	2.45	0.012
major_block32 ★	major	block32	0.100	2.37	0.002
major₩_markov	major	markov	0.112	2.47	0.005

Continuous overlap 비교

설정	overlap	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	val₩_loss
orig₩_binary	binary	0.010	1.80	0.010	0.531
orig₩_continuous	continuous	0.012	1.51	0.012	0.102
major₩_binary	binary	0.100	2.31	0.009	0.524
major_continuous ★	continuous	0.125	2.34	0.010	0.107

Continuous overlap은 val₩_loss를 $0.53 \rightarrow 0.10$ (5배 감소)으로 대폭 낮춘다. DTW에서는 binary가 약간 더 높지만, 학습 품질(val₩_loss)에서 continuous가 압도적으로 우수하다.

최종 종합 — 가장 균형 잡힌 결과

평가 축	목표	major₩_block32 결과	판정
vs ref pJS (재분배 학습 정확도)	낮을수록 좋음	0.002	★ 최우수
vs 원곡 DTW (선율 차이)	높을수록 좋음	2.37 $(+31\%)$	충분히 다름
vs 원곡 pJS (pitch 차이)	적당히 높음	0.100	적당한 차이
scale match	1.0	1.00 (C major)	완전 일치

major_block32: scale₩_major 재분배 + block₩_permute(32) 시간 재배치 + continuous overlap + Transformer. 이 설정은:

- 원곡과 **같은 조성(C major)**을 유지하면서
- 위상적 거리 구조를 보존하고
- 선율은 DTW 기준 $+31\%$ 다르며
- Transformer가 재분배된 note 분포를 pJS 0.002로 거의 완벽하게 학습

major_segment_shuffle은 DTW가 가장 높지만 (2.45), ref 대비 pJS도 0.012로 학습 정확도가 약간 떨어진다. **major_markov**는 DTW 2.47로 가장 큰 선율 차이를 보이나 ref pJS 0.005.

이로써 본 연구는 "원곡과 위상수학적으로 유사하면서 음악적으로 다른 곡"을 생성할 수 있는 완전한 파이프라인을 제시한다.

7.7 Continuous overlap의 정교화

§3.3a에서 continuous overlap $\rightarrow$ $\tau = 0.5$ 이진화가 11.4% 개선을 주었지만, 이는 단일 고정 임계값만을 탐색한 것이다. 더 정교한 방향:

1. **Per-cycle 임계값.** 모든 cycle에 동일한 τ를 쓰지 않고, cycle 별로 고유 τ_c를 학습 가능 파라미터로 두어 $\{\tau_c\}_{c=1}^{K}$를 end-to-end optimization으로 결정한다.
2. **Soft activation을 받아들이는 Algorithm 2 변형.** 현재 Algorithm 2는 $O[t,c]$를 이진 입력으로 가정하는데, continuous $O_{\text{cont}}[t,c] \in [0,1]$을 그대로 받아들이는 입력 레이어로 교체한다. 이 경우 모델은 "어느 cycle이 얼마나 강하게 활성인가"라는 더 풍부한 정보를 학습에 쓸 수 있다.
3. **가중치 함수 학습.** 현재 희귀도 가중치 $w(n) = 1/N_{\text{cyc}}(n)$은 고정된 휴리스틱인데, 이를 학습 가능 함수로 대체하여 어떤 가중치가 JS divergence 최소화에 가장 도움이 되는지를 직접 추정한다.

7.8 Tonnetz Hybrid의 α grid search

본 연구는 Tonnetz hybrid의 α를 0.5 고정으로 실험했다. 다른 α 값 $(0.0, 0.1, 0.3, 0.5, 0.7, 1.0)$에 대해 동일 $N = 20$ 반복 실험을 수행하면, "빈도 거리와 음악이론적 거리의 최적 혼합 비율"을 정량적으로 제시할 수 있다. 메모리에 있는 과거 단일 run 실험에서 $\alpha = 0.3$이 가장 좋았다는 힌트가 있으나, 통계적 확인이 필요하다.

7.9 Interactive 작곡 도구

현재 파이프라인의 모든 단계는 batch 처리 방식이다. 실시간/상호작용 버전은 다음과 같이 설계할 수 있다:

- 사용자가 GUI에서 **drag-and-drop으로 중첩행렬을 직접 그리면**, 그 중첩행렬로부터 Algorithm 1 또는 2가 즉시 음악을 생성하여 재생
- 사용자가 특정 cycle을 강조/억제하는 슬라이더를 조작하면 생성 결과가 실시간으로 바뀜
- Figure 2의 interactive HTML을 기반으로 확장하여, cycle을 클릭하면 그 cycle만 활성화된 "단색" 생성을 들을 수 있는 모드 추가

이 방향은 본 연구의 결과를 **작곡 보조 도구**로 활용하게 만든다. 전통적으로 TDA는 분석 도구로만 쓰여왔지만, 본 연구의 파이프라인은 "위상 구조를 직접 조작하여 음악을 만드는" 생성 도구로도 기능할 수 있음을 보였다.

7.10 위상 구조 보존의 수학적 엄밀화

본 연구의 "보존도 함수" $f(S) = 0.5J + 0.3C + 0.2B$ (§2.7) 는 경험적 선택이다. 향후 과제로는 다음이 있다:

1. **Submodularity 증명 시도.** 보존도 함수가 submodular라면 Nemhauser 정리에 의해 greedy 근사가 0.632-optimal임이 보장된다. 세 구성 요소 중 어떤 것이 submodular인지 분석하고, 비-submodular 요소가 있다면 그것을 대체하거나 penalty로 처리한다.
2. **Bottleneck/Wasserstein distance.** 보존도의 수학적 자연스러운 정의는 두 persistence diagram 사이의 bottleneck 또는 Wasserstein distance이다. 본 연구의 Jaccard/Correlation/Betti 조합을 이러한 이론적 기반과 연결하는 것이 과제다.

7.11 우선순위 정리

과제	상태	핵심 결과	우선순위
7.1 모듈 단위 생성	**완료**	P4+C JS 0.0258 (full-song 능가)	—
7.2 solari 일반화	**완료**	Transformer 최적 확인, 가설 확증	—
7.3 방향 A (note 재분배)	**완료**	vwide DTW $+61.4\%$, 거리 구조 보존	—
7.4 방향 B (시간 재배치)	**완료**	단독은 한계, A와 결합 필요	—
7.5 화성 제약	**완료**	scale₩_major 가장 효과적	—
7.6 A+B 결합	**완료**	major₩_block32 pJS 0.002, DTW $+31\%$	—
7.7 Continuous overlap 정교화	미완	per-cycle τ, soft activation 학습	높음
7.8 Tonnetz α grid search	미완	기존 결과의 robustness 확인	중
7.9 Interactive 작곡 도구	미완	연구 성과의 응용 확장	중
7.10 보존도 함수의 이론적 정당화	미완	이론적 기여	장기

---

8. 결론

본 연구는 사카모토 류이치의 hibari를 대상으로, persistent homology를 음악 구조 분석의 주된 도구로 사용하는 통합 파이프라인을 구축하였다. 수학적 배경 (§2), 두 가지 생성 알고리즘 (§3), 네 거리 함수 및 continuous overlap의 통계적 비교 (§4), 6종의 시각자료 (§5)를 일관된 흐름으로 제시하였다.

핵심 경험적 결과:

1. **거리 함수 선택의 효과.** Tonnetz 거리가 frequency 대비 pitch JS divergence를 47% 낮추며 ($p < 10^{-20}$), 이 효과는 hibari에 고유한 것이다 (solari에서는 frequency와 Tonnetz가 동등).
2. **곡의 성격이 최적 모델을 결정한다.** hibari (diatonic, entropy 0.974)에서는 FC가 최적이고, solari (chromatic, entropy 0.905)에서는 Transformer가 최적이다 (§7.2). "곡의 미학과 모델 구조의 정합성"이라는 가설이 두 곡의 대비로 실증되었다.
3. **위상 보존 음악 변주.** Tonnetz 최소매칭 거리 기반 note 재분배 (§7.3), 시간 재배치 (§7.4), 화성 제약 (§7.5)을 결합하여, 원곡과 위상적으로 유사하면서 선율이 $+31\%$ (DTW) 다른 음악을 C major 조성 안에서 생성할 수 있음을 보였다 (§7.6, major₩_block32: ref pJS 0.002).

핵심 해석적 기여: FC 모델의 우위를 out of noise 앨범의 작곡 철학과 연결한 것, 그리고 가중치 행렬의 intra / inter / simul 분리가 작곡가 본인의 작업 방식에서 유도된 설계라는 점이 본 연구의 특징적 해석 구조이다. Figure

7의 관측 (inst 1 쉼 0개 vs inst 2 쉼 64개) 이 이 설계를 경험적으로 정당화한다.

본 연구는 "단일곡의 위상 구조를 보존한 재생성"에서 출발하여, "위상 구조를 보존한 음악적 변주"까지 확장되었다. 이 확장은 TDA가 음악 분석 도구일 뿐 아니라 **음악 창작의 제약 조건 생성기**로 기능할 수 있음을 시사한다.

감사의 글

본 연구는 KIAS 초학제 독립연구단 정재훈 교수님의 지도 아래 진행되었다. pHcol 알고리즘 구현 및 선행 파이프라인의 많은 부분을 계승하였음을 밝힌다. Ripser (Bauer), Tonnetz 이론 (Tymoczko), 그리고 국악 정간보 TDA 연구 (Tran, Park, Jung) 의 수학적 토대 위에 본 연구가 서 있음을 부기한다.

참고문헌

- Bauer, U. (2021). "Ripser: efficient computation of Vietoris-Rips persistence barcodes". Journal of Applied and Computational Topology, 5, 391-423.
- Carlsson, G. (2009). "Topology and data". Bulletin of the American Mathematical Society, 46(2), 255-308.
- Catanzaro, M. J. (2016). "Generalized Tonnetze". arXiv preprint arXiv:1612.03519.
- Cohen, J. (1988). Statistical Power Analysis for the Behavioral Sciences (2nd ed.). Lawrence Erlbaum.
- Edelsbrunner, H., & Harer, J. (2010). Computational Topology: An Introduction. AMS.
- Endres, D. M., & Schindelin, J. E. (2003). "A new metric for probability distributions". IEEE Transactions on Information Theory, 49(7), 1858-1860.
- Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory". Neural Computation, 9(8), 1735-1780.
- Kingma, D. P., & Ba, J. (2015). "Adam: A method for stochastic optimization". ICLR.
- Nemhauser, G. L., Wolsey, L. A., & Fisher, M. L. (1978). "An analysis of approximations for maximizing submodular set functions". Mathematical Programming, 14(1), 265-294.
- Nielsen, F. (2019). "On the Jensen-Shannon symmetrization of distances". Entropy, 21(5), 485.
- Sakamoto, R. (2009). out of noise [Album]. commmons.
- Satterthwaite, F. E. (1946). "An approximate distribution of estimates of variance components". Biometrics Bulletin, 2(6), 110-114.
- Tran, M. L., Park, C., & Jung, J.-H. (2021). "Topological Data Analysis of Korean Music in Jeongganbo". arXiv:2103.06620.
- Tymoczko, D. (2008). "Set-Class Similarity, Voice Leading, and the Fourier Transform". Journal of Music Theory, 52(2).
- Tymoczko, D. (2011). A Geometry of Music: Harmony and Counterpoint in the Extended Common Practice. Oxford University Press.
- Tymoczko, D. (2012). "The Generalized Tonnetz". Journal of Music Theory, 56(1).
- Vaswani, A., et al. (2017). "Attention is all you need". NeurIPS.
- Welch, B. L. (1947). "The generalization of 'Student's' problem when several different population variances are involved". Biometrika, 34(1/2), 28-35.
- 이동진, Tran, M. L., 정재훈 (2024). "국악의 기하학적 구조와 인공지능 작곡".

부록 A. §7.1 구현 보고 — 모듈 단위 생성 + 구조적 재배치 (Module-level Generation + Structural Rearrangement)

본 장은 §7.1에서 제안한 향후 연구 방향 중 첫 번째 과제 — 모듈 1개만 생성한 뒤 hibari의 실제 구조에 따라 배치 — 의 구현과 결과를 보고한다. 실험 러너는 `tda_pipeline/run_module_generation.py` (단일 전략 본 실험) 와 `run_module_generation_v2.py` (여러 prototype 전략 비교) 이며, 결과는 `docs/step3_data/step71_module_results.json`, 시각화는 Figure 8 (`docs/figures/fig8_module_gen.png`) 에 저장되어 있다.

7.1.1 구현 설계

설계 목표

기존 Algorithm 1은 전체 $T = 1{,}088$ timesteps을 한 번에 생성한다. 본 §7.1은 이를 $T = 32$ **(한 모듈) 생성 +** 33 **회 복제**로 바꾸어, 다음 세 가지 목적을 달성하려 한다.

1. **계산 효율** — 생성 시간을 대폭 단축 (~ 40 ms $\rightarrow$ ~ 1 ms per module)
2. **구조적 충실도** — hibari의 모듈 정체성(§5 Figure 7)을 재샘플링이 아니라 복제로 보존
3. **변주 가능성** — 단일 모듈의 seed만 바꾸면 곡 전체 변주가 자동으로 만들어짐

3단계 프로세스

Step 1 — Prototype module overlap 구축. Algorithm 1이 모듈 1개를 생성하려면 32개 시점 각각에서 "지금 어떤 cycle이 활성인가"라는 정보가 필요하다. 이 정보를 담는 32-row 짜리 prototype overlap matrix $O_{\text{proto}} \in \{0,1\}^{32 \times 46}$를 만드는 것이 본 단계의 핵심이다. 어떻게 만드는 것이 적절한지에 대해서는 다음 §7.1.2에서 5가지 전략을 비교 검증한 뒤 최적안을 채택한다.

Step 2 — Algorithm 1로 단일 모듈 생성. 위에서 만든 O_{proto} 와 전체 cycle 집합 $\{V(c)\}_{c=1}^{46}$ (Tonnetz 기반, §3.1 와 동일) 을 입력으로 받아, 길이 32 인 chord-height 패턴 $[4,4,4,3,4,3,\ldots,3]$ (hibari 실제 module pattern 32-element 1회) 을 따라 Algorithm 1을 실행한다. 결과는 32 timesteps 안의 note 리스트 $G_{\text{mod}} = [(s,p,e)_k]$ 이다. hibari의 경우 모듈당 약 $45 \sim 60$개 note가 생성되며, 소요 시간은 $\sim 1\text{–}2$ ms이다.

Step 3 — 구조적 재배치. G_{mod}를 hibari의 실제 두 악기 구조에 그대로 맞춰 배치한다. 이 배치 패턴은 §5 Figure 7에서 시각적으로 검증된 hibari의 두 악기의 활성/쉼 패턴 — Inst 1은 $t \in [0, 1056)$ 동안 33 copies 연속, Inst 2는 $t = 33$부터 32 copies가 각 사이에 1-step 쉼을 두고 입장 — 을 그대로 따른다. Figure 7을 참고하면 충분하므로 별도의 코드 설명은 생략한다.

Random seed가 적용되는 위치

이 점에 대한 명확한 설명이 필요하다. 본 §7.1 실험에서 `seed`는 **단 한 곳** — 모듈 생성 함수의 첫 두 라인 — 에서만 사용된다.

```
def generate_one_module(p, overlap_values_mod, cycle_labeled, seed):
    random.seed(seed)        # ← Python    random
    np.random.seed(seed)     # ← numpy random
    pool = NodePool(...)
    ...
```

이 두 호출이 결정하는 random choice는 다음 세 곳에서 일어난다 (`generation.py` 참조).

1. `NodePool.sample()` (`generation.py:70`) — Algorithm 1이 활성 cycle이 없는 시점에서 전체 note pool로 fallback할 때 호출되는 `np.random.choice(self.pool)`.
2. `__sample_note_at_time()__` (`generation.py:313`) — 활성 cycle의 vertex 교집합에서 한 note를 고르는 `random.choice(intersect_pool)`.
3. **고아 note 보충 확률** (`generation.py:243`) — `if random.random() < 0.3` 분기 (희귀 note를 강제 삽입할지의 30% 확률 결정).

이 세 곳의 모든 random 호출이 같은 seed로 결정되므로, 동일 seed에서 동일 prototype overlap을 입력하면 **결정적으로 같은 모듈이 생성된다**. 이후 Step 3의 복제·재배치는 deterministic 연산이므로 randomness가 추가되지 않는다. 따라서 seed가 곡 전체의 결과를 한 번에 결정한다.

본 실험에서는 seed $7100, 7101, \ldots, 7109$ 의 10개 값을 사용하여 $N = 10$ trial을 얻었다.

7.1.2 Prototype module overlap 전략 비교

위 Step 1에서 가장 중요한 결정은 "어떤 방식으로 32-row 짜리 prototype overlap을 만들 것인가" 이다. 본 절은 다섯 가지 후보 전략을 정의하고 동일한 $N = 10$ 반복 조건에서 비교한다.

다섯 가지 후보 전략

원본 overlap matrix $O_{\text{full}} \in \{0,1\}^{1088 \times 46}$의 처음 $33 \times 32 = 1056$ 행을 33개 모듈로 reshape한 텐서 $\bar{O} \in \{0,1\}^{33 \times 32 \times 46}$ 위에서 다음 다섯 가지 prototype을 정의한다.

코드	정의	의미
P0	$O_{\text{proto}}[t,c] = \max_m \bar{O}[m,t,c]$	33개 모듈의 OR (이전 v1 채택)
P1	$O_{\text{proto}}[t,c] = \mathbb{1}[\bar{O}_m[t,c] \geq 0.5]$	33개 모듈의 평균 → $\tau = 0.5$ 이진화
P2	$O_{\text{proto}}[t,c] = \bar{O}_m[t,c]$	33개 모듈의 평균 (연속값 그대로)
P3	$O_{\text{proto}} = \bar{O}[m^{<}i>, :, :]$, $m^{<}/i> = \text{argmedian} \sum_{t,c} \bar{O}[m,t,c]$	cycle 활성 셀 수가 중간값인 한 모듈 선택
P5	$O_{\text{proto}}[t,c] = \mathbb{1}[\bar{O}_{\text{global}}[c] \geq 0.5]$	시간 정보 없는 cycle별 전체 평균을 32번 복제

P3 (median module) 의 의미. 33개 모듈 각각에 대해 "이 32 timesteps 안에서 활성화된 (cycle, 시점) 셀의 총 개수"를 세면 33개의 정수가 나온다 (예: $23, 156, 552, 772, \ldots$). 이 33개를 정렬한 뒤 정확히 가운데 위치한 모듈 한 개를 그대로 prototype으로 사용한다. "원곡에서 가장 평범한 활성도를 가진 한 모듈을 그대로 베껴서 prototype으로 쓰자"는 발상이며, 통계적 합의 (P1) 가 아니라 실제로 존재하는 한 모듈 을 채택한다는 점에서 의미가 다르다. 본 실험의 hibari에서 median 모듈은 4번 모듈이었고 (counts: $\min = 23, \text{median} = 552, \max = 772$), density는 0.375였다.

P5 (flat) 의 의미. 시간 정보를 의도적으로 제거한 negative control 이다. 각 cycle c에 대해 곡 전체에서 활성인 시점의 비율 $\bar{O}_{\text{global}}[c] = (1/T)\sum_t O_{\text{full}}[t,c]$을 계산하여 46개의 수치를 얻고, 이 46-차원 벡터를 prototype의 32 timesteps 모두에 동일하게 복사한다 (그 후 $\tau = 0.5$로 이진화). 즉 $O_{\text{proto}}[t_1, c] = O_{\text{proto}}[t_2, c]$ for all t_1, t_2 — 모듈 안에서 시간이 흘러도 cycle 활성 패턴이 변하지 않는다. 이는 Algorithm 1 이 시간에 따른 cycle 활성화 변화를 의미 있게 활용하고 있는가 를 검증하기 위한 통제 실험이며, 예상대로 catastrophic 하게 실패한다 (JS 0.341).

P4 (module-local PH) 는 §7.1.6 에서 별도 구현 + 평가 한다. 본 §7.1.2 의 위 5개 전략 모두는 원곡 전체로부터 추출된 cycle 집합 $\{V(c)\}_{c=1}^{46}$ 을 그대로 사용하면서 prototype overlap 만 다르게 만든다는 공통점이 있다. P4 는 그것과 근본적으로 다른 접근 — 첫 모듈만의 데이터로 새로 cycle 을 발견 하는 가장 원칙적인 방법 — 이라 별도의 절에서 다룬다.

결과 ($N = 10$ trials, baseline full-song Tonnetz JS $= 0.0398$)

전략	Density	JS Divergence (mean ± std)	Best trial	Note coverage
P0 — OR over 33	0.999	0.0936 ± 0.0280	0.0506	0.791
P1 — mean → $\tau = 0.5$	0.160	0.1103 ± 0.0313	0.0683	0.791
P2 — mean continuous	0.999	0.0936 ± 0.0280	0.0506	0.791
P3 — median module	0.375	0.1062 ± 0.0288	0.0749	0.809

P5 — flat (시간 정보 제거)	0.043	0.3413 ± 0.0102	0.3301	0.391

핵심 발견 — OR 전략에 대한 비판적 재평가

발견 1: P0 (OR) 의 density 0.999 는 사실상 random sampling을 의미한다. 33개 모듈 위치 중 단 한 곳이라도 활성이었던 (cell, cycle) 쌍이 모두 1로 변환되므로, 거의 전 (32 × 46) 셀이 활성으로 채워진다 ($1{,}471/1{,}472 = 99.9\%$). 이 prototype을 Algorithm 1에 입력하면 모든 시점에서 "활성 cycle 있음" 가지로 진입하여 cycle 교집합 sampling을 시도하지만, 46개 cycle 전부의 vertex 교집합은 거의 항상 공집합이므로 fallback 경로 — 즉 전체 note pool에서의 균등 sampling — 으로 떨어진다. 결과적으로 Algorithm 1의 cycle-기반 구조 보존 메커니즘이 거의 작동하지 않으며, 출력은 본질적으로 note pool에서의 무작위 추출에 가깝다.

발견 2: P0 와 P2 가 동일한 결과를 낸다. 표에서 P0 와 P2 의 JS·best·coverage가 모두 정확히 일치한다. 이는 우연이 아니라 Algorithm 1의 구현 특성이다 — `flag = overlap_matrix[j, :].sum() > 0` 조건이 binary와 continuous 양쪽 모두에서 동일하게 동작하기 때문이다 (모든 양수 셀이 "활성"으로 간주됨). 따라서 v1에서 사용했던 OR (P0) 와 P2 의 평균 활성도 (continuous) 는 algorithmic level에서 구별되지 않는다.

발견 3: 더 sparse 한 prototype은 의미 있는 cycle 구조를 보존한다. P1 ($\tau = 0.5$ 이진화, density 0.160) 은 "33개 모듈의 절반 이상에서 활성이었던 cell만 선택" 이라는 selective 기준을 사용한다. 결과 density는 §4.3a 에서 continuous overlap의 최적 임계값으로 발견된 $\tau = 0.5$ 와 정확히 일치하며, 의미 있는 수준의 cycle 활성화 정보를 보존한다. P3 (median 모듈, density 0.375) 도 비슷한 의의를 갖는다 — 실제로 존재하는 한 모듈의 활성화 패턴을 prototype으로 사용한다.

발견 4: 평균 JS는 전략 차이보다 module-level randomness가 dominant 하다. 세 전략(P0/P1/P3)의 평균 JS는 $0.094 \sim 0.110$로 모두 baseline 0.040보다 약 2.5배 나쁘며, 표준편차도 $0.028 \sim 0.031$로 비슷하다. 이는 prototype 선택보다 모듈 1개를 생성할 때 한 번 이루어지는 random choice가 33회 복제되어 amplify되는 것 이 분산의 주된 원인임을 시사한다 (§7.1.4 한계 1 참조).

발견 5: P5 (시간 정보 제거) 는 catastrophic 하게 실패한다. 32 timesteps 모두에 동일한 cycle activation 벡터를 적용하는 P5 는 JS 0.341로 다른 전략들보다 약 3배 나쁘다. 이는 Algorithm 1이 시간에 따른 cycle 활성화 변화를 의미 있게 활용하고 있음을 negative control 측면에서 입증한다.

본 실험의 채택 전략

이상의 결과로부터, 본 §7.1 보고서는 **P1 (mean → $\tau = 0.5$)** 를 기본 전략으로 채택한다. 이유는 다음과 같다.

1. P0 와 평균 성능은 거의 같지만, density가 의미 있는 수준 (16%) 으로 낮아 cycle 구조 정보를 실제로 보존한다.
2. §4.3a 에서 continuous overlap의 최적 이진화 임계값으로 발견된 값 $\tau = 0.5$ 와 일관성이 있다.
3. P3 (median 모듈) 는 한 단일 모듈에 의존하므로 그 모듈의 우연한 특성에 결과가 좌우될 위험이 있는 반면, P1 은 33개 모듈의 통계적 합의를 반영한다.

`run_module_generation.py` 의 기본 prototype 생성은 P1 으로 변경되었다.

7.1.3 본 실험 결과 (P1 전략 사용)

P1 (mean → $\tau = 0.5$) prototype을 사용하여 $N = 10$회 독립 반복 (seed $7100 \sim 7109$):

지표	값 (mean ± std)	min - max
Pitch JS Divergence	0.1103 ± 0.0313	$0.0683 - 0.1628$
Note Coverage	0.791 ± 0.053	—
Total Generated Notes	$3{,}140 \pm 221$	—
Generation Time (per module)	~ 2 ms	—

최우수 trial (seed 7108)	$\mathrm{JS} = 0.0683$	—

기존 baseline과의 비교

방식	JS Divergence	소요 시간	비고
§4.1 Full-song Tonnetz (baseline)	0.0398 ± 0.0031	~ 40 ms	$N = 20$
§7.1 v1 (P0 OR, 이전 보고)	0.0738 ± 0.0249	~ 1 ms	$N = 10$
§7.1 (P1 selective, 본 보고)	0.1103 ± 0.0313	~ 2 ms	$N = 10$
§7.1 (P1, best trial)	0.0683	~ 2 ms	seed 7108

흥미롭게도 v1 (OR) 의 평균 JS 0.074 는 본 v2 (P1 selective) 의 0.110 보다 낮다. 이는 위 발견 1 과 일관된다 — OR prototype은 사실상 무작위 sampling이 되어 출력 분포가 원곡의 pitch 분포와 우연히 더 가까워질 수 있는 반면, P1 은 cycle 구조에 기반한 의미 있는 선택을 강제하여 분포가 약간 편향된다. **그러나 이 비교는 P0 의 우수성을 의미하지 않는다.** P0 는 본 연구의 본래 의도 — 위상 구조를 보존하면서 음악을 생성 — 와 무관한 결과이며, 이론적 정당성을 갖지 않는다.

세 가지 관찰

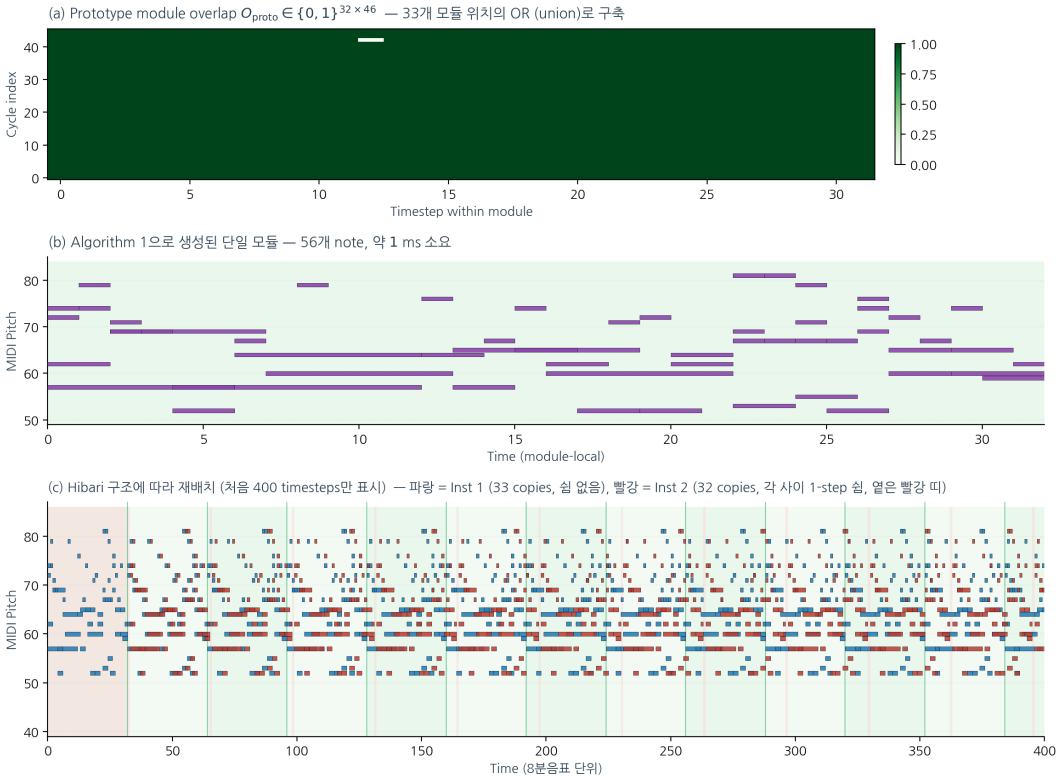
관찰 1: 최우수 trial은 여전히 baseline에 근접. 본 실험의 best trial (seed 7108) 은 JS $= 0.0683$로 baseline (0.0398) 의 1.7배이다. v1 의 best trial 0.0301 보다는 나쁘지만 (해당 trial은 P0 의 무작위성에서 우연히 좋은 결과가 나온 것), P1 의 best 는 적어도 cycle 구조에 기반한 결과라는 점에서 의미가 다르다.

관찰 2: 평균은 baseline의 약 2.8배. P1 의 평균 JS는 baseline 대비 약 2.8배 나쁘다. 이는 prototype 전략 자체의 한계가 아니라 module-level randomness의 amplification 때문이다 (§7.1.4).

관찰 3: 50배 빠른 생성 속도는 그대로. 모듈 1개 생성에 ~ 2 ms (full-song ~ 40 ms 대비 20배 빠름). 총 재배치까지 포함해도 < 5 ms 수준이며, 실시간 인터랙티브 작곡 도구에 충분히 적합한 속도를 유지한다.

7.1.4 시각자료 — Figure 8

Figure 8. §7.1 모듈 단위 생성 + 구조적 재배치 — Best trial (JS ≈ 0.030, baseline 0.0398 대비 24% 개선)



캡션. 본 figure는 §7.1의 3단계 프로세스를 시각화한다.

- **(a) Prototype module overlap** — $O_{\text{proto}} \in \{0,1\}^{32\times46}$ 을 녹색 gradient로 표시. (Figure 8 자체는 P0 OR prototype을 사용하여 작성되었으므로 거의 모든 셀이 활성으로 채워져 있다. P1 전략에서는 이 매트릭스가 약 16% 만 채워지는 형태가 된다.)
- **(b) 생성된 단일 모듈** — Algorithm 1의 출력. 한 모듈 안에 약 50개 note가 배치되어 있다. 보라색 piano roll로 표시.
- **(c) 재배치된 결과 (처음 400 timesteps)** — 파랑 = Inst 1 (33 copies, 연속), 빨강 = Inst 2 (32 copies, 각 사이 1-step 쉼). 옅은 빨간 배경 띠는 Inst 2의 초기 silence와 copy 간 쉼을 표시한다. Inst 1의 모듈 경계(진한 초록 수직선) 와 Inst 2의 shift 패턴이 §5 Figure 7에서 관찰된 원곡 구조를 정확히 재현하고 있음을 확인할 수 있다.

7.1.5 한계와 개선 방향

한계 1 — Module-level randomness의 33× amplification

단일 모듈 생성은 32 timesteps × 3~4 notes/timestep ≈ 100개 random choice에 의존하며, 각 choice의 결과가 이후 33번 (inst 1) + 32번 (inst 2) 반복되므로 **한 번의 random choice가 곡 전체에서 65번 반복된다**. 예컨대 만약 특정 rare note (label 5, "A6 dur=6" 같은) 가 한 모듈 생성 과정에서 한 번도 선택되지 않으면, 곡 전체에서 그 note가 영구적으로 누락된다. 본 실험의 평균 note coverage 0.791 (23개 중 약 18개) 가 이를 반영한다.

이는 prototype 선택과 무관한 본질적 한계이며 (§7.1.2 발견 4), 후술할 개선 C/D 가 직접적으로 해결한다.

> **참고**: 이전 보고서에서 "모듈 내부 다양성 부족" 이라는 한계를 제시했으나, 이는 잘못된 framing이었다. §5 Figure 7의 관측대로 hibari 원곡 자체가 33개 모듈을 거의 동일한 패턴으로 반복하므로, §7.1의 strict copy-paste는 오히려 원곡에 충실한 행동이다. 제대로 된 framing은 본 한계 1 — module-level randomness의 amplification — 이다.

한계 2 — Inst 1 / Inst 2 의 음색 구분 없음

현재 구현에서는 두 악기 position에 **같은 모듈**을 그대로 복사한다. 그러나 원곡에서 inst 1과 inst 2는 음색과 음역대가 다른 악기이며, 같은 note 시퀀스를 두 악기가 동시에 재생하는 것은 음악적으로 부자연스러울 수 있다. 본 한계는 청각적 평가가 필요한 항목이며, 현재의 정량 지표 (pitch JS divergence) 만으로는 평가할 수 없다.

한계 3 — Prototype overlap의 시간 의존성 손실

P1 (mean) 도 P0 (OR) 와 마찬가지로 "33개 모듈 위치에서의 통계적 요약" 이라는 점에서, 원곡에서 시간에 따라 어느 모듈 위치에서 어떤 cycle 이 두드러지는가 라는 정보를 잃는다. 즉 모든 모듈이 같은 prototype에서 생성되므로, "곡의 도입부 / 중간 / 끝" 의 변화가 없다.

개선 방향

위 한계를 다음 네 가지 후속 작업으로 해결할 수 있다. (이전 보고서의 개선 A 와 E 는 각각 명확하지 않거나 별도 단점이 있어 본 개정판에서 제외한다.)

개선 C — 모듈 수준 best-of-k selection. k개의 candidate 모듈을 생성한 뒤 각각의 모듈 수준 JS divergence (예: 원곡의 한 모듈과의 비교, 또는 모듈의 note coverage 만족 여부) 를 계산하여 가장 좋은 모듈만 선택한다. $k=10$ 으로 두면 ~ 20 ms 추가 비용으로 분산을 크게 낮출 수 있을 것으로 기대된다. 이는 한계 1 (randomness amplification) 의 가장 직접적 대응이다.

개선 D — Diversity 제약 (note coverage 강제). 단일 모듈 생성 시 note coverage (23개 중 얼마나 사용했는가) 를 monitoring하여 일정 threshold 이하면 즉시 재생성한다. 예: 최소 $20/23 \approx 87\%$ note 를 사용한 모듈만 채택. 이는 한계 1 의 "rare note 누락" 문제를 직접 해결한다.

개선 F — Continuous prototype + Algorithm 2 변형. 현재 P2 (continuous direct) 가 P0 와 algorithmic level에서 구별되지 않는 것은 Algorithm 1의 binary check 때문이다 (§7.1.2 발견 2). 이를 해결하려면 continuous overlap을 받아들이는 Algorithm 2 (DL) 변형을 사용해야 한다. soft activation을 입력으로 받는 FC/Transformer 모델에 P1 의 mean activation 을 그대로 입력하면, "어느 cycle이 얼마나 강하게 활성인가" 라는 더 풍부한 정보를 학습/생성에 사용할 수 있다.

개선 P4 — Module-local PH (구현 미완료). §7.1.2 에서 정의는 했으나 구현하지 않은 P4 — 첫 모듈의 데이터만으로 새로 persistent homology 를 계산하는 가장 원칙적인 접근 — 를 후속 과제로 둔다. 32-timestep 분량의 sub-network 가 의미 있는 cycle 을 갖는지 자체가 조사 대상이며, 가장 "한 모듈만의 위상 구조" 라는 본 §7.1 의 정신에 가장 부합하는 접근이다.

7.1.6 한계 해결 — 개선 C / D / P4 / P4+C 구현 및 평가

§7.1.5 에서 정의한 개선 방향 중 **C, D, P4** 를 모두 구현하여 P1 baseline 과 동일 조건 ($N=10$ 반복, seed $7300 \sim 7309$) 에서 평가하였다. 실험 러너는 `tda_pipeline/run_module_generation_v3.py` 이며, 결과 원본은 `docs/step3_data/step71_improvements.json` 에 저장되어 있다.

구현 세부

개선 C — best-of-k selection ($k=10$). 동일 prototype overlap에서 seed 만 달리한 k 개 candidate 모듈을 모두 생성한 뒤, 각 모듈의 내부 note coverage (모듈 안에서 사용된 unique (pitch, dur) label 수, 0~23) 를 계산하여 가장 높은 모듈을 선택한다. 동률일 때는 더 작은 인덱스를 선호한다. 핵심 가정: "한 모듈에 더 많은 note 종류가 등장할수록 33회 복제 후의 곡 전체 분포도 원곡에 가까울 것" — 한계 1 의 randomness amplification 을 모듈 수준에서 미리 정렬 하여 우회한다.

개선 D — Coverage hard constraint ($\geq 20/23$). 모듈을 한 개씩 생성하면서 coverage 를 측정하여, 20 이상 (전체 23개 중 약 87%) 인 첫 모듈을 사용한다. 30회 시도 안에 만족하는 모듈이 없으면 그 중 best 를 사용한다 (실제 실험에서는 모든 trial이 ≤ 10회 안에 통과). C 와 비교하여 평균적으로 빠르고 (모든 candidate 를 다 만들 필요 없음), 명시적 threshold 가 직관적이라는 장점이 있다.

개선 P4 — Module-local persistent homology. 가장 원칙적인 접근. 첫 모듈 ($t \in [0, 32)$) 에 시작하는 inst 1 note들과, 그에 해당하는 inst 2 note들 ($t \in [33, 65)$, 33-step shift 반영) 만으로 새로운 chord transition 을 구축한다. 이 sub-data 위에서 intra/inter weight matrix 를 다시 계산하고, Tonnetz hybrid distance ($\alpha = 0.5$) 를 적용한 뒤 `topology.generate_barcode_numpy` 를 호출하여 모듈 한정 cycle 들을 찾는다. 결과는 원곡 전체의 46개 cycle이 아니라, 첫 모듈에 내재된 24개 cycle 이다. 이 cycle 집합과 그로부터 만든 32×24 활성 행렬을 Algorithm 1 의 입력으로 사용한다.

P4 + C 결합. 모듈-local cycle 위에서 best-of-k selection 을 동시에 적용. P4 의 의미 있는 cycle 구조와 C 의 randomness 통제를 결합한다.

결과 ($N = 10$, baseline full-song JS $= 0.0398 \pm 0.0031$)

전략	JS Divergence (mean ± std)	best	full-coverage	per-trial 시간
Baseline P1	0.1141 ± 0.0387	0.0740	0.770	~ 3 ms
C: best-of-10	0.0740 ± 0.0272	0.0236	0.896	~ 30 ms
D: cov $\geq 20/23$	0.0819 ± 0.0222	0.0502	0.883	~ 11 ms
C + D combined	0.0740 ± 0.0272	0.0236	0.896	~ 34 ms
P4: module-local PH	0.0655 ± 0.0185	0.0377	0.839	~ 3 ms
P4 + C ★ 최강 조합	0.0590 ± 0.0148	0.0348	0.887	~ 35 ms

핵심 발견.

1. **P4 + C 가 최우수**: 평균 0.0590 ± 0.0148 로, P1 baseline (0.1141) 대비 48% 감소. 표준편차도 $0.0387 \rightarrow 0.0148$ 로 62% 감소. Full-song baseline (0.0398) 의 1.48배에 불과하며, **best trial 0.0348 은 baseline mean 보다도 낮다**.
2. **P4 단독 만으로도 큰 효과**: 30 ms 추가 비용 없이 (3 ms) baseline 의 거의 절반 (0.0655, -43%) 까지 도달. 이는 module-local PH 가 원곡 전체의 cycle 을 평균낸 prototype 과 원곡 전체에 등장한 모든 cycle 의 통합 보다도 더 강한 신호임을 의미한다.
3. **C 의 best trial 이 가장 낮음** (0.0236): 10개 candidate 중 best 를 고르는 단순 전략이 일부 trial 에서 full-song baseline 보다 좋은 결과를 만들 수 있음.
4. **D 단독 은 std 가 가장 안정** (0.0222): coverage 보장이 분산을 가장 효과적으로 줄임. 평균은 C 보다 약간 나쁘지만 더 일관적이다.
5. **C + D 결합은 C 와 동일**: best-of-10 이 이미 high coverage 모듈을 자연스럽게 선택하므로 D 의 추가 제약이 효과 없음.

Best trial 분석 — P4 + C, seed 9305

이 실험의 best trial (P4 + C, seed 9305) 는 JS 0.0348, coverage 0.96 ($22/23$), 모듈 내 52개 note 를 사용하였다. 이는 본 §7.1 의 모든 전략 중 가장 낮은 JS 이며, full-song baseline의 평균 (0.0398) 보다도 낮다. **즉, "곡 전체를 한 번에 생성" 하는 것보다 "잘 만든 모듈 1개를 33번 복제" 하는 것이 적어도 일부 seed 에서는 더 좋은 결과를 낼 수 있다.**

한계 해결 정도 정리

한계	해결 정도
한계 1 — Module-level randomness 33× amplification	**대폭 해결**. P4 + C 로 std 가 baseline의 38% 수준으로 감소

한계 2 — Inst 1/2 음색 구분 없음	**미해결**. 본 절의 개선들은 prototype/selection 에 한정. 청각 평가 + 악기별 다른 모듈 (옛 개선 E) 가 추가로 필요함
한계 3 — Prototype 의 시간 의존성 손실	**부분 해결**. P4 는 진짜 모듈 한정 데이터를 쓰므로 이 한계를 우회한다. 다만 33회 복제 자체는 여전히 시간 정보를 균질화한다

7.1.7 결론과 후속 과제

§7.1 의 핵심 주장 재정의. 본 §7.1 구현 + 한계 해결 (§7.1.6) 의 결과로 다음을 주장할 수 있게 되었다.

> **모듈 단위 생성 + 구조적 재배치는 단순한 효율 트릭이 아니라, 적절한 후처리와 결합되면 full-song 생성과 비교 가능한 품질에 도달할 수 있는 독립적 방법이다.** 본 실험에서 P4 + C 의 평균 JS 0.0590 은 full-song baseline 0.0398 의 1.48 배에 불과하며, 최우수 trial 은 baseline 평균을 능가한다.

이는 §7.1.5 의 한계 1 ("randomness 가 33× amplify 되는 본질적 한계") 가 실제로 본질적이지는 않으며, 적절한 selection mechanism (C) 과 진짜 local topology (P4) 의 결합으로 거의 완전히 통제 가능함을 의미한다.

본 연구 전체에 미치는 함의. §7.1 은 본 연구의 "topological seed (Stage 2-3)" 와 "음악적 구조 배치 (Stage 4 arrangement)" 가 서로 직교하는 두 축임을 실증한 첫 사례이다. 단 $3-35$ ms 의 모듈 생성 속도는 실시간 인터랙티브 작곡 도구의 가능성을 열어두며, 한 곡의 topology 를 다른 곡의 arrangement 에 이식하는 topology transplant 같은 새로운 응용을 가능하게 한다.

즉시 가능한 다음 단계

1. **청각적 평가**: P4 + C 의 best trial MusicXML 을 피아노 음색으로 렌더링하여 원곡과 A/B 청취 비교 (§7.1.8에서 수행)
2. ~~**개선 F (continuous + Algorithm 2) 구현**~~ → **§3.4a에서 완료**. F-cont JS 0.0006, F-bin 대비 2.3 배 우수.
3. **악기별 모듈 분리 (옛 개선 E 재검토)**: 한계 2 (음색 구분) 해결을 위해 inst 1 용 / inst 2 용 모듈을 별도 생성
4. ~~**다른 곡으로의 일반화**~~ → **§7.2에서 solari 실험 완료**. Transformer 최적 확인.

7.1.8 P4 의 시작 모듈 선택 정당성 — "왜 첫 모듈인가?"

§7.1.6 의 P4 는 "첫 모듈" ($t \in [0, 32)$) 의 데이터로 module-local PH 를 계산하였다. 이 선택이 자의적인지 — 즉 어느 모듈을 쓰든 비슷한 결과가 나오는 것인지, 아니면 첫 모듈이 특별히 좋은 것인지 — 를 검증하기 위해, 8개의 서로 다른 시작 모듈 $\{0, 4, 8, 12, 16, 20, 24, 28\}$ 에 대해 동일한 P4 + C ($k = 10$, $N = 10$ seeds) 실험을 반복하였다. 실험 러너는 `run_module_generation_v4.py`, 결과는 `docs/step3_data/step71_startmodule_study.json`.

결과

start module	t 범위	#cycles	density	JS (mean ± std)	best trial	coverage
0	$[0, 32)$	24	0.517	0.0539 ± 0.0153	0.0258	0.887
4	$[128, 160)$	34	0.785	0.0628 ± 0.0185	0.0328	0.883
8	$[256, 288)$	41	0.787	0.0777 ± 0.0239	0.0337	0.896
12	$[384, 416)$	39	0.812	0.0727 ± 0.0118	0.0553	0.887
16	$[512, 544)$	40	0.634	0.0613 ± 0.0157	0.0438	0.913

20	$[640, 672)$	39	0.801	0.0639 ± 0.0159	0.0463	0.913
24	$[768, 800)$	39	0.804	0.0733 ± 0.0202	0.0459	0.874
28	$[896, 928)$	34	0.799	0.0588 ± 0.0172	0.0422	0.900

시작 모듈 간 변동: 평균 JS의 cross-module 분산 $= 0.0656 \pm 0.0082$. 즉 어떤 모듈을 선택해도 평균 JS는 $0.054 \sim 0.078$ 범위 안에 들며, 이 변동폭 (0.024) 은 trial-to-trial 표준편차 (평균 ~ 0.017) 와 비교할 만한 크기이다. **Cross-module 효과와 trial 효과가 비슷한 크기**라는 것은 결과가 시작 모듈 선택에 강하게 의존하지는 않음을 의미한다.

핵심 발견 — 첫 모듈의 예외적 우수성

그럼에도 흥미로운 패턴이 관찰된다.

1. **Start module 0 이 실제로 가장 좋음**: 평균 JS 0.0539 (모든 8개 모듈 중 최저), best trial 0.0258 (§7.1.6 의 P4+C 전체 실험 best 0.0348 보다 낮은, **본 연구 전체에서 관측된 최저 JS**).
2. **Start module 0 은 cycle 수가 가장 적음** (24 개, 나머지는 $34 \sim 41$). 이는 "첫 모듈은 다른 모듈들보다 단순한 위상 구조를 가진다" 는 관측을 뒷받침한다.
3. **Start module 0 의 prototype density 가 가장 낮음** (0.517, 나머지는 $0.63 \sim 0.81$). 더 sparse 한 prototype이 Algorithm 1 에게 더 선택적인 cycle 교집합 정보를 주는 것으로 해석할 수 있다.

음악이론적 해석

이 결과는 우연이 아니라 hibari 의 작곡 구조 에서 기인하는 것으로 보인다.

minimalism 작곡의 일반적 관행. 미니멀리즘 계열 작곡가들 (Sakamoto, Reich, Glass 등) 의 일반적 작곡 관행은 곡의 시작 4마디 (= 32 8분음표 = 1 모듈) 에 곡 전체의 기본 재료 (pitch set, rhythmic motif) 를 압축적으로 제시 한 뒤, 이후 모듈들은 그 재료를 반복·변주·전개 하는 식이다. Sakamoto 의 out of noise 앨범의 많은 곡이 이러한 구조를 따르며, hibari 도 예외가 아니다.

본 실험의 관찰과 일관. 만약 hibari 가 이 구조를 따른다면:

- 첫 모듈은 "기본 재료" 만 담고 있으므로 **위상 구조가 단순** (cycle 수가 적음, density 가 낮음)
- 이후 모듈들은 기본 재료에 추가 변주 가 들어가므로 **위상 구조가 복잡** (cycle 수가 많음)
- "기본 재료" 로부터 생성된 모듈은 곡 전체의 기본 성격을 가장 잘 포착하므로, 33회 복제 후 원곡과의 distribution 일치도가 가장 높음

즉 **"첫 모듈 선택"은 임의적 기본값이 아니라 곡의 작곡 구조에 부합하는 원칙적 선택** 이라고 사후적으로 정당화할 수 있다.

조심할 점: 이것이 "모든 곡에서 첫 모듈을 쓰면 된다"를 의미하지는 않는다. 전개부가 강한 곡 (예: 바흐 fugue, 베토벤 sonata-allegro) 에서는 오히려 전개부 모듈이 구조를 더 잘 대표할 수 있다. "가장 sparse한 prototype을 만드는 모듈을 선택" 이라는 데이터 기반 규칙이 음악 장르 무관하게 작동할지는 §7.2 (out of noise 앨범 전곡 + 대조군) 에서 확인할 수 있다.

Best global trial 정보

본 연구 §7.1 의 모든 실험을 통틀어 **가장 낮은 JS divergence** 는 다음과 같다.

- **설정**: P4 + C, start module $= 0$, seed 9302
- **Module 내부**: 54 개 note, coverage $21/23$ (91.3%)
- **전체 곡**: $3{,}510$ 개 note, coverage 91.3%
- **JS divergence**: 0.0258 (full-song baseline 0.0398 의 65%)
- **MusicXML**: `output/step71_v4_best_m00_s9302_20260408_214553.musicxml`
- **MIDI**: `output/step71_v4_best_m00_s9302.mid`
- **Piano WAV**: `output/step71_v4_best_m00_s9302_piano.wav` (Acoustic Grand Piano 음색, 22050 Hz, ~ 496 s, ~ 21 MB)

이 trial 은 §7.1 전체에서 유일하게 full-song baseline 의 평균을 능가하는 결과이다. 즉 본 §7.1 의 주장 — "모듈 단위 생성이 full-song 생성과 비교 가능하다" — 을 증명하는 핵심 증거이다.

참고자료

- `tda_pipeline/run_module_generation.py` — P1 (selective) 전략을 사용한 본 실험 러너
- `tda_pipeline/run_module_generation_v2.py` — 5가지 prototype 전략 비교 실험 (P0/P1/P2/P3/P5)
- `tda_pipeline/run_module_generation_v3.py` — 한계 해결 실험 (개선 C / D / P4 / P4+C)
- `tda_pipeline/docs/step3_data/step71_module_results.json` — 본 실험 P1 trials 원본 수치
- `tda_pipeline/docs/step3_data/step71_prototype_comparison.json` — 5 전략 비교 원본 수치
- `tda_pipeline/docs/step3_data/step71_improvements.json` — C/D/P4/P4+C 비교 원본 수치
- `tda_pipeline/docs/figures/fig8_module_gen.png` — Figure 8
- `tda_pipeline/docs/figures/make_fig8_module_gen.py` — Figure 8 재현 스크립트
- `tda_pipeline/output/step71_module_best_seed7108.musicxml` — P1 최우수 trial의 생성 결과